

SecondLook

A field manual for a security suite disguised as quality-of-life tools.

We don't tell you that the internet is dangerous. We tell you when something deserves a second look. Every file, every rule, and every trade-off — from empty folder to store listing, in plain JavaScript with no build step.

Parts 0 – 13, complete

Manifest V3 · ten modules · one verdict engine · local-first, zero telemetry

Table of Contents

Part 0 - Foundation & Architecture	5
0.1 What you are building	5
0.2 Architecture at 10,000 feet	5
0.3 The verdict contract	6
0.4 "OFF must be real OFF"	6
0.5 The complete file tree	7
0.6 manifest.json	9
0.7 The shared layer	10
0.8 The engine	19
0.9 The design system	28
0.10 The background core	38
0.11 The popup - all ten rows, live	45
0.12 The demo module - proof the toggles are real	50
0.13 Icons	51
0.14 Load & test Part 0	52
 Part 1 - Link Sniper	 54
1.1 Design analysis	54
1.2 Files	55
1.3 Registry entry for Part 1	57
1.4 Load & test	57
1.5 Tuning knobs	58
 Part 2 - Redirect Detective	 58
2.1 Design analysis	58
2.2 Files	59
2.3 Registry entry for Part 2	60
2.4 Load & test	60
2.5 Tuning knobs	61
 Part 3 - Trust Badge	 61
3.1 Design analysis	61
3.2 Files	62
3.3 Registry entry for Part 3	64
3.4 Load & test	64
3.5 Tuning knobs	64

Part 4 - Form Guardian	65
4.1 Design analysis	65
4.2 Files	65
4.3 Background route + registry entry	68
4.4 Load & test	69
4.5 Tuning knobs	69
Part 5 - Ghost Click Blocker	70
5.1 Design analysis	70
5.2 Files	70
5.3 Registry entry for Part 5	72
5.4 Load & test	73
5.5 Tuning knobs	73
Part 6 - Fake Download Button	74
6.1 Design analysis	74
6.2 Files	74
6.3 Registry entry for Part 6	77
6.4 Load & test	77
6.5 Tuning knobs	77
Part 7 - Download Guard	78
7.1 Design analysis	78
7.2 Files	78
7.3 Router route + wiring	81
7.4 Load & test	82
7.5 Tuning knobs	82
Part 8 - Tab Guard	83
8.1 Design analysis	83
8.2 Files	83
8.3 Registry entry for Part 8	85
8.4 Load & test	85
8.5 Tuning knobs	86
Part 9 - Cookie Tamer	86
9.1 Design analysis	86
9.2 Files	87
9.3 Sweep route + registry entry	89

9.4 Load & test	90
9.5 Tuning knobs	90
Part 10 - Webcam/Mic Indicator	91
10.1 Design analysis	91
10.2 Files	92
10.3 Badge route + registry entry	94
10.4 Load & test	95
10.5 Tuning knobs	95
Part 11 - Options, Dashboard & Popup Polish	95
11.1 Design analysis	95
11.2 Options page	96
11.3 Dashboard	102
11.4 Popup polish + two new router routes	104
11.5 Load & test	108
11.6 Tuning knobs introduced in Part 11	109
Part 12 - Test Pages & Local Test Server	109
12.1 Design analysis	109
12.2 The test server	109
12.3 The index and link farm	110
12.4 Fake login, ghost click, fake download	112
12.5 Consent banner, popup storm, toggle test	113
12.6 Load & test	115
Part 13 - Ship It: README & Store Listing	116
13.1 Design analysis	116
13.2 README.md	116
13.3 Remove the demo module	117
13.4 Web Store listing copy	117
13.5 Permission justification (paste into the review form)	118
13.6 Screenshot checklist	118
13.7 Packaging	119

Part 0 - Foundation & Architecture

0.1 What you are building

SecondLook is a Manifest V3 Chrome extension that ships as a suite of ten "quality of life" tools and quietly behaves like a security product. The pitch to users is honest but soft-spoken: **We don't tell you that the internet is dangerous. We tell you when something deserves a second look.** Hover a link and a small pill tells you whether the destination looks normal. Land on a page with an invisible click-stealing overlay and the overlay simply stops working. Start typing a password into a form that posts it to a different domain than the one you are looking at, and a full-page interception stops the submit until you consciously allow it. None of it phones home, none of it uploads anything, and every single intervention can be undone.

The suite is organized into four pillars. **Link & Site Intel** contains Link Sniper (hover verdicts on links), Redirect Detective (follows redirect chains and shows you every hop), and Trust Badge (flags trust-seal images that are just pictures). **Form & Click Safety** contains Form Guardian (credential exfiltration on submit), Ghost Click Blocker (invisible full-page overlays), and Fake Download Button (competing download clusters). **Files & Downloads** contains Download Guard (magic-byte sniffing of downloaded files). **Annoyance & Privacy** contains Tab Guard (popup-storm containment), Cookie Tamer (parks consent banners without ever clicking "Accept"), and Webcam/Mic Indicator (a persistent chip whenever a page uses your camera or microphone). Each module can be turned off independently, per site or globally, and turning a module off genuinely removes its code from every page.

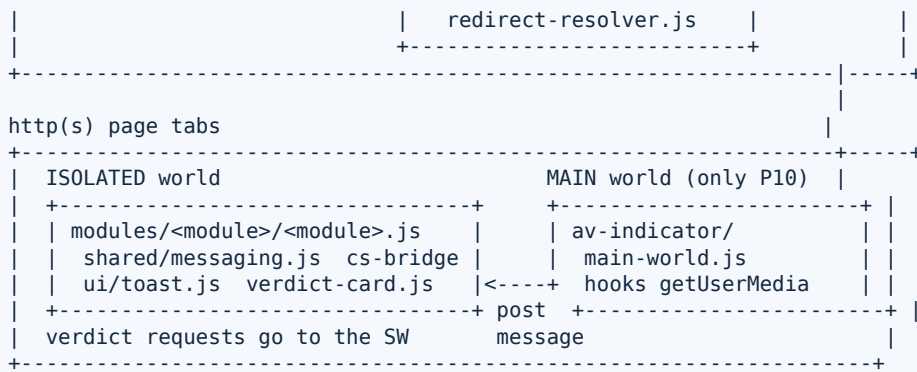
This guide is written for a developer with solid modern JavaScript who has never shipped a Chrome extension. Everything is plain ES2020+ JavaScript, CSS and HTML: no bundler, no framework, no build step, no external dependencies at runtime. You will paste every file in this guide from scratch, in order, and at the end of each part the extension will load and run in `chrome://extensions`. Budget roughly four weeks: Part 0 is a full weekend, Parts 1 through 10 are about a day each, and Parts 11 through 13 close out packaging and store listing.

0.2 Architecture at 10,000 feet

The extension has exactly one brain and many hands. The **service worker** (the MV3 "background script") owns every privileged decision: it hosts the verdict engine, mediates all message routing, watches downloads and tab creation, and owns the settings store. Content scripts are deliberately dumb: they observe the page, ask the service worker for verdicts over the message bus, and render UI with the shared component kit. The engine is the only component in the entire codebase allowed to score a URL - modules may collect evidence and escalate context, but nobody rolls their own risk numbers.

(architecture overview)





The vertical split matters more than any individual box. Everything above the second horizontal rule runs with extension privileges; everything below it runs inside web pages. The only bridge between the two worlds is the message contract `{module, type, payload}` wrapped in promises by `shared/messaging.js`. Data flows in one of three patterns. **Ask**: a content script sends `engine / analyze` with a URL and gets back a verdict object, which it caches per page load. **Report**: a content script sends `<module> / status` with a one-line summary that the popup later reads from session storage for the tab. **Push**: the service worker broadcasts `core / moduleToggled` to every open tab when a setting changes, and targeted messages such as `redirectDetective / showChain` when a context-menu action resolves.

All shared code uses a single convention: every file is a classic script that attaches its API to the `globalThis.SL` namespace and guards against double execution. The service worker loads shared files with `importScripts()`, extension pages load them with `<script src>` tags, and the bootstrap registers them into content scripts by path. One file, three runtimes, zero module-system gymnastics - this is the payoff for avoiding ES modules, and it is why the codebase has no build step.

0.3 The verdict contract

Every judgment the extension ever shows belongs to one of three words, and every module must use the engine's vocabulary instead of inventing its own. The words are chosen so that a non-technical user can act on them without reading anything else:

Verdict	Color & weight	Meaning	User action
CLEAR	Green pill	Nothing unusual for this URL in this context	None - keep browsing
SECOND LOOK	Amber pill + one plain sentence	Some signal deserves attention before you click, submit, or trust	Pause a beat, read the one sentence
INTERCEPTED	Red card, full attention	High-confidence credential or file exfiltration in progress	Actively choose Stop or Continue

Three rules make the system trustworthy. First, **one visible warning per event**: a link hover shows at most one pill, a page shows at most one banner; the engine ranks reasons internally and surfaces only the strongest plain-language sentence. Second, **every interception is reversible**: INTERCEPTED is always a card with a Stop and a Continue button, never a silent block - if we stop a submit, we keep the form state and re-dispatch it on request. Third, **when in doubt, suggest, never interrupt**: the engine's URL-only score is deliberately capped below the interception threshold (39 of 40 required), so no combination of URL signals alone can stop the user. Only modules with hard evidence - Form Guardian seeing a password leave the visible origin, Download Guard seeing a file whose bytes contradict its extension - are allowed to pass `escalate: true` and cross the red line.

0.4 "OFF must be real OFF"

Most extensions treat their toggle as a UI preference: the warnings disappear but the code keeps running. SecondLook treats the toggle as a contract with three clauses, and Part 0 implements the machinery for all of them. **Clause 1 - no static injection**: the manifest's `content_scripts` field is empty, permanently. All module code enters pages only through

`chrome.scripting.registerContentScripts()` calls made by the bootstrap at install, at browser startup, and after settings changes. **Clause 2 - off means unregistered:** flipping a module's toggle writes settings, then the service worker unregisters that module's script ids and clears any DNR session rules the module owned. **Clause 3 - open tabs get cleaned up:** unregistration does not touch already-loaded pages, so the router also broadcasts a teardown message; the module's content script runs its registered cleanup functions (remove pills, banners and overlays; restore `pointer-events`; detach observers) and then removes itself from the page.

(toggle lifecycle - the whole story in one diagram)

```
User flips toggle in popup
  |
  v
popup.js: SL.msg.send('core','setModuleEnabled',{moduleId,enabled:false})
  |   (promise; row shows a subtle 'syncing' pulse until reply)
  v
module-router.js: SL.Settings.setModuleEnabled(...)
  |   writes chrome.storage.local('sl.settings')
  v
storage.onChange fires in the service worker
  |
  v
bootstrap.js: reconcile('settings-changed')
  |   1. desired = enabled modules with script specs
  |   2. current = chrome.scripting.getRegisteredContentScripts()
  |   3. unregister ids that are no longer desired
  |   4. register ids that became desired (duplicate-safe)
  v
router broadcasts to every open http/https tab:
{module:'core', type:'moduleToggled', payload:{moduleId,enabled:false}}
  |
  v
cs-bridge.js in each tab: this module's script matches ->
  run SL.cleanupFor(moduleId):
  - remove injected DOM (pills, banners, chips, intercept layer)
  - restore modified styles (pointer-events: auto)
  - disconnect observers and event listeners
  - set module state to 'dead'; no further page mutation
  |
  v
session status lines for that module are dropped from every tab
```

Turning a module back on is the mirror image: reconcile registers the script ids, and for pages already open (which will never see the registered script because they finished loading), `bootstrap.injectIntoOpenTabs()` calls `chrome.scripting.executeScript` and `insertCSS` directly. The result is that toggling feels instant in both directions on every page you have open - which is also the single best demo of the product's honesty, and exactly what `test-pages/toggle-test.html` (Part 12) lets you show off.

Site-level exceptions are the one place where "real off" gets a caveat, so it is called out here rather than buried. A per-site disable keeps the module's scripts registered globally (because dynamic registration is pattern-based, not host-list based) but the module's bridge checks the effective settings for the current host at boot and stays completely dormant: no observers, no listeners on the page, no DOM. The service worker applies the same check before any privileged action for that site. The privacy promise is identical - zero page interaction - but the script file is technically present in the page's isolated world. If you want physical absence on a specific site, the global toggle is the real switch.

DEVIATION NOTE - MV3 content-script registration cannot be scoped to a runtime-computed host allowlist without re-registering on every change. We accept dormant-but-present scripts for per-site disables; global disables physically unregister. This is documented in the README (Part 13) as the honest privacy statement.

0.5 The complete file tree

Create the skeleton below as you go; each part of this guide fills specific files, and the tree is annotated with the part that fills it. Paths are relative to the extension root, which you will keep in a folder named `secondlook/`. Nothing outside this tree is ever required, and nothing in it is optional except the `test-pages/` and `tools/` folders at ship time (Part 13 removes the dev-only demo module and tells you what to exclude from the store zip).

```
secondlook/ (complete file tree)
secondlook/
manifest.json          <- Part 0    MV3 manifest, content_scripts: []
README.md             <- Part 13   install, privacy, troubleshooting
background/
  service-worker.js    <- Part 0    entry; wires every listener
  module-router.js     <- Part 0    message contract dispatcher
  bootstrap.js         <- Part 0    register/unregister/inject engine
  context-menu.js      <- Part 0    right-click entries for P2/P0
  safe-mode.js         <- Part 0    per-tab DNR session blocking
  tab-guard.js         <- Part 8    popup-storm containment
  download-guard.js    <- Part 7    magic-byte sniffing + intercept
shared/
  storage.js           <- Part 0    local/session/stats/events/salt
  settings.js          <- Part 0    settings schema + effective(host)
  messaging.js         <- Part 0    promise message bus (all worlds)
  module-registry.js   <- Part 0    catalog: pillars, modules, scripts
  cs-bridge.js         <- Part 0    content-side runtime + lifecycle
engine/
  known-brands.js      <- Part 0    ~100 brands + official domains
  domain-intel.js      <- Part 0    eTLD+1, subdomains, IP literals
  url-analyzer.js      <- Part 0    weighted signal extraction
  redirect-resolver.js  <- Part 0    follows hops via webRequest
  verdict-engine.js     <- Part 0    analyze() -> verdict object
ui/
  tokens.css           <- Part 0    CSS custom properties, light+dark
  components.css       <- Part 0    pills, toggles, cards, toasts...
  toast.js             <- Part 0    SLToast - page + popup toasts
  verdict-card.js      <- Part 0    SLVerdict - pill + detail card
  popup.html / popup.css / popup.js <- Part 0/11 4 pillars, 10 live rows
  options.html / options.css / .js <- Part 11 every knob, site exceptions
  dashboard.html / dashboard.css / .js <- Part 11 stats, events, overrides
modules/
  demo-greeting/       <- Part 0    dev-only; proves toggles (P13)
    demo-greeting.js  demo-greeting.css
  link-sniper/         <- Part 1
    link-sniper.js    link-sniper.css
  redirect-detective/   <- Part 2
    redirect-detective.js redirect-detective.css
  trust-badge/         <- Part 3
    trust-badge.js    trust-badge.css
  form-guardian/       <- Part 4
    form-guardian.js  form-guardian.css
  ghost-click/         <- Part 5
    ghost-click.js    ghost-click.css
  fake-download/       <- Part 6
    fake-download.js  fake-download.css
  download-guard/      <- Part 7
    notify.js         notify.css
  cookie-tamer/        <- Part 9
    cookie-tamer.js   cookie-tamer.css
  av-indicator/        <- Part 10
    main-world.js     indicator.js  av-indicator.css
tools/
  make-icons.py        <- Part 0    Pillow icon generator
test-pages/           <- Part 12
  serve-tests.py index.html link-farm.html fake-login.html
  ghost-click.html fake-download.html consent-banner.html
  popup-storm.html popup-storm-target.html toggle-test.html
assets/icons/
  icon-16.png icon-32.png icon-48.png icon-128.png
  icon-alert-48.png icon-paused-48.png icon-camera-48.png
```

0.6 manifest.json

The manifest is the product's contract with Chrome, and every line here is load-bearing. Note especially the empty `content_scripts` array: that single design choice is what makes "OFF is real OFF" mechanically true, because Chrome itself will never inject our module code into a page - only the bootstrap will. The service worker is declared as a classic script on purpose: we use `importScripts()` so that the exact same shared files that run in content scripts and extension pages also run in the worker, under the identical `globalThis.SL` namespace.

```
manifest.json
{
  "manifest_version": 3,
  "name": "SecondLook",
  "version": "1.0.0",
  "description": "Quality-of-life tools that quietly watch your back. " +
    "Local-only, zero telemetry, every warning undoable.",
  "minimum_chrome_version": "116",

  "background": {
    "service_worker": "background/service-worker.js"
  },

  "action": {
    "default_popup": "ui/popup.html",
    "default_title": "SecondLook",
    "default_icon": {
      "16": "assets/icons/icon-16.png",
      "32": "assets/icons/icon-32.png",
      "48": "assets/icons/icon-48.png",
      "128": "assets/icons/icon-128.png"
    }
  },

  "options_page": "ui/options.html",

  "icons": {
    "16": "assets/icons/icon-16.png",
    "32": "assets/icons/icon-32.png",
    "48": "assets/icons/icon-48.png",
    "128": "assets/icons/icon-128.png"
  },

  "permissions": [
    "storage", "tabs", "scripting", "downloads", "cookies",
    "alarms", "contextMenus", "tabGroups",
    "declarativeNetRequest", "webRequest"
  ],

  "host_permissions": ["http:///*/*", "https:///*/*"],

  "content_scripts": [],

  "web_accessible_resources": []
}
```

The `content_scripts: []` entry is deliberate and self-documenting: a reviewer of your store listing can see at a glance that no module code is statically injected. The empty `web_accessible_resources` matters just as much - nothing from the package is exposed to web pages, which removes an entire class of fingerprinting and abuse concerns during store review. Every icon referenced here is generated by `tools/make-icons.py` at the end of this part.

Permission justifications (keep this table - you will reuse it verbatim in the Web Store review form in Part 13)

Permission	Why it exists	Used by
storage	settings, local stats, session runtime state, salt	all
tabs	current-tab host for popup + effective settings; Tab Guard	popup, P8
scripting	dynamic register/unregister of module scripts; inject into open tabs	bootstrap
downloads	observe + cancel + retry downloads for magic-byte check	P7
cookies	site sweep that preserves login cookies	P9
alarms	daily prune of the local event log	service worker
contextMenus	"Where does this link really go?", "Safe Mode this tab"	P0, P2
tabGroups	isolation group for popup storms	P8
declarativeNetRequest	Safe Mode per-tab session rules (the ONLY blocking API)	P0
webRequest	observe redirect hops by requestId (never blocking)	P2, engine
http/https host	run analysis on every page the user browses; sniff download bytes	all modules

Two of these deserve a sentence of honesty before a reviewer asks. The broad host permission exists because the product is site-agnostic by definition - a link checker that only runs on whitelisted sites protects nobody - and all analysis is on-device, so the permission grants access but never sends data anywhere. `webRequest` is observer-only: the manifest does not include `webRequestBlocking`, which MV3 would reject for store extensions anyway, so the API is used purely to number and record redirect hops while the service worker's own `fetch` follows them.

WARNING - The spec originally listed `tabGroups` implicitly through Tab Guard's design; it must be declared explicitly or `chrome.tabGroups` is `undefined` at runtime and popup-storm containment silently fails. It is included above - just do not "optimize" it away later.

DEVIATION NOTE - The original design sketch pinned host permissions to specific sites (mail providers, search engines). That would make always-on modules impossible: link verdicts and form checks are needed everywhere, not on three domains. Broad hosts with local-only processing is the honest trade-off, and Part 13 words the store justification accordingly. Mail providers and search engines still get special-cased in code: Form Guardian exempts mail hosts, Cookie Tamer preserves their login cookies, Redirect Detective resolves their shortlinks.

0.7 The shared layer

Five files make up the shared layer, and every runtime in the extension loads at least three of them. They follow one convention throughout: an IIFE, an idempotency guard, and an attachment to `globalThis.SL`. Because they are classic scripts, the identical bytes run in the service worker (via `importScripts`), in extension pages (via `<script src>`), and in content scripts (via registration `js` arrays). The only file that is content-world-specific is `cs-bridge.js`; everything else is world-neutral.

shared/storage.js

Thin promise wrappers over `chrome.storage.local` and `chrome.storage.session`, plus the three persistent data structures SecondLook keeps: a stats counter, a capped event log, and the salt used by Form Guardian's domain hashing. One privacy rule is enforced here at the storage boundary rather than trusted to callers: the event log persists **hostnames only**, **never full URLs** - so even your local history of what SecondLook flagged cannot be reconstructed into a browsing history.

```

/* shared/storage.js - storage, stats, event log, salt.
 * Classic script, attaches SL.Storage; safe to load in any runtime. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.Storage) return;

  const EVENTS_KEY = 'sl.events';
  const EVENTS_MAX = 200; // tuning: local event-log cap
  const STATS_KEY = 'sl.stats';
  const SALT_KEY = 'sl.salt';

  async function localGet(key, fallback) {
    const got = await chrome.storage.local.get(key);
    return (key in got && got[key] !== undefined) ? got[key] : fallback;
  }

  async function localSet(key, value) {
    await chrome.storage.local.set({ [key]: value });
  }

  /* storage.session is reachable from the SW and extension pages only.
   * Content scripts report through SL.msg instead - by design, so no
   * setAccessLevel call ever widens session data to page worlds. */
  async function sessionGet(key, fallback) {
    if (!chrome.storage.session) return fallback;
    const got = await chrome.storage.session.get(key);
    return (key in got && got[key] !== undefined) ? got[key] : fallback;
  }

  async function sessionSet(key, value) {
    if (!chrome.storage.session) return;
    await chrome.storage.session.set({ [key]: value });
  }

  async function getStats() {
    return localGet(STATS_KEY, {
      byVerdict: { CLEAR: 0, SECOND_LOOK: 0, INTERCEPTED: 0 },
      byModule: {}
    });
  }

  async function bumpStat(moduleId, verdict) {
    const stats = await getStats();
    if (verdict && stats.byVerdict[verdict] !== undefined) {
      stats.byVerdict[verdict] += 1;
    }
    stats.byModule[moduleId] = (stats.byModule[moduleId] || 0) + 1;
    await localSet(STATS_KEY, stats);
  }

  /* Privacy boundary: events persist the HOST only, never the full URL,
   * so the local log can never be reconstructed into a browsing history. */
  function hostOf(url) {
    try { return new URL(url).hostname; } catch (e) { return ''; }
  }

  async function pushEvent(evt) {
    const list = await localGet(EVENTS_KEY, []);
    list.unshift({
      ts: Date.now(),
      module: evt.module || '?',
      verdict: evt.verdict || null,
      host: hostOf(evt.url || ''), // hostname only, by design
      note: evt.note || ''
    });
    await localSet(EVENTS_KEY, list.slice(0, EVENTS_MAX));
  }

  async function getEvents(limit) {
    const list = await localGet(EVENTS_KEY, []);
    return typeof limit === 'number' ? list.slice(0, limit) : list;
  }
}

```

```

}

async function pruneEvents() {
  const list = await localGet(EVENTS_KEY, []);
  if (list.length && list.length < EVENTS_MAX * 0.75) return 0;
  await localSet(EVENTS_KEY, list.slice(0, EVENTS_MAX));
  return list.length - EVENTS_MAX;
}

/* Random per-install salt; used by Form Guardian to store salted
 * hashes of approved origins instead of the origins themselves. */
async function ensureSalt() {
  let salt = await localGet(SALT_KEY, null);
  if (!salt) {
    const bytes = new Uint8Array(16);
    crypto.getRandomValues(bytes);
    salt = Array.from(bytes, (b) => b.toString(16).padStart(2, '0')).join('');
    await localSet(SALT_KEY, salt);
  }
  return salt;
}

async function resetData() {
  await chrome.storage.local.remove([EVENTS_KEY, STATS_KEY]);
}

root.Storage = {
  localGet, localSet, sessionGet, sessionSet,
  getStats, bumpStat, pushEvent, getEvents, pruneEvents,
  ensureSalt, resetData,
  EVENTS_MAX
};
})(globalThis);

```

shared/settings.js

The settings file defines the schema for all ten modules plus the global switch and per-site overrides, and it is the single place that knows how to compute an *effective* configuration for a given host. The deep-merge in `effective()` is what lets each layer (defaults, then global settings, then the site override) speak in terms of the same flat module keys. Every consumer in the codebase - bootstrap, router, content bridge, popup, options - goes through this file; nobody reads raw settings objects and improvises.

```

shared/settings.js
/* shared/settings.js - settings schema + effective(host) resolution. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.Settings) return;

  const KEY = 'sl.settings';

  const DEFAULTS = {
    globalEnabled: true,
    modules: {
      demoGreeting: { enabled: true }, // dev only
      linkSniper: { enabled: true, hoverDelayMs: 350, showClear: true },
      redirectDetective: { enabled: true, maxHops: 8 },
      trustBadge: { enabled: true },
      formGuardian: { enabled: true, rememberApproved: true },
      ghostClick: { enabled: true, annotate: false },
      fakeDownload: { enabled: true },
      downloadGuard: { enabled: true, blockOnMismatch: true },
      tabGuard: { enabled: true, threshold: 3, windowMs: 10000 },
      cookieTamer: { enabled: true, strategy: 'prefer-reject' },
      avIndicator: { enabled: true }
    },
    siteOverrides: {} // host -> { modules: { moduleId: false } }
  };

  function clone(obj) { return JSON.parse(JSON.stringify(obj)); }

```

```

function mergeInto(base, patch) {
  for (const k of Object.keys(patch || {})) {
    const v = patch[k];
    if (v && typeof v === 'object' && !Array.isArray(v) &&
        base[k] && typeof base[k] === 'object' && !Array.isArray(base[k])) {
      mergeInto(base[k], v);
    } else if (v !== undefined) {
      base[k] = v;
    }
  }
  return base;
}

async function get() {
  const stored = await root.Storage.localGet(KEY, null);
  if (!stored) return clone(DEFAULTS);
  // unknown-module-proof: defaults win for anything missing
  return mergeInto(clone(DEFAULTS), stored);
}

async function save(next) {
  await root.Storage.localSet(KEY, next);
  return next;
}

async function setModuleEnabled(moduleId, enabled) {
  const s = await get();
  if (!s.modules[moduleId]) s.modules[moduleId] = { enabled };
  else s.modules[moduleId].enabled = !!enabled;
  return save(s);
}

async function setModuleOption(moduleId, option, value) {
  const s = await get();
  if (!s.modules[moduleId]) s.modules[moduleId] = {};
  s.modules[moduleId][option] = value;
  return save(s);
}

async function setGlobalEnabled(enabled) {
  const s = await get();
  s.globalEnabled = !!enabled;
  return save(s);
}

async function setSiteOverride(host, moduleId, enabled) {
  const s = await get();
  if (!s.siteOverrides[host]) s.siteOverrides[host] = { modules: {} };
  s.siteOverrides[host].modules[moduleId] = !!enabled;
  if (!Object.keys(s.siteOverrides[host].modules).length) {
    delete s.siteOverrides[host];
  }
  return save(s);
}

async function clearSiteOverride(host, moduleId) {
  const s = await get();
  const site = s.siteOverrides[host];
  if (site && site.modules && moduleId in site.modules) {
    delete site.modules[moduleId];
    if (!Object.keys(site.modules).length) delete s.siteOverrides[host];
  }
  return save(s);
}

async function clearAllSiteOverrides(host) {
  const s = await get();
  delete s.siteOverrides[host];
  return save(s);
}

/* Effective settings for one host: defaults <- global <- site override. */
async function effective(host) {
  const s = await get();

```

```

const out = clone(s);
const site = host && s.siteOverrides && s.siteOverrides[host];
if (site && site.modules) {
  for (const [modId, enabled] of Object.entries(site.modules)) {
    if (!out.modules[modId]) out.modules[modId] = {};
    out.modules[modId].enabled = enabled; // site layer wins
  }
}
return out;
}

function isModuleOn(settings, moduleId, host) {
  if (!settings || settings.globalEnabled === false) return false;
  const site = host && settings.siteOverrides &&
    settings.siteOverrides[host];
  const siteFlag = site && site.modules && site.modules[moduleId];
  if (siteFlag !== undefined && siteFlag !== null) return !!siteFlag;
  const m = settings.modules && settings.modules[moduleId];
  return !!m && m.enabled !== false;
}

/* Subscribe to settings changes (chrome.storage.onChangeed). */
function subscribe(cb) {
  chrome.storage.onChangeed.addListener((changes, area) => {
    if (area === 'local' && changes[KEY]) cb(changes[KEY].newValue,
      changes[KEY].oldValue);
  });
}

root.Settings = {
  KEY, DEFAULTS, get, save, setModuleEnabled, setModuleOption,
  setGlobalEnabled, setSiteOverride, clearSiteOverride,
  clearAllSiteOverrides, effective, isModuleOn, subscribe
};
})(globalThis);

```

shared/messaging.js

The message bus wraps `chrome.runtime.sendMessage` in promises and normalizes responses to `{ok: true, data}` or `{ok: false, error}`. The `listen()` helper exists for the content world and extension pages: it filters messages addressed to one module id (or to `'*` for broadcasts) and always returns `true` from the listener callback so the response channel stays open for the async reply. The service worker does not use `listen()` - it has the router, which is deliberately fatter and lives in `background/`.

shared/messaging.js

```

/* shared/messaging.js - promise message bus. Message shape:
 * {module, type, payload} -> {ok:true, data} | {ok:false, error} */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.msg) return;

  function send(module, type, payload) {
    return new Promise((resolve) => {
      try {
        chrome.runtime.sendMessage({ module, type, payload }, (res) => {
          // Absorb "Receiving end does not exist" (SW asleep or no listener)
          void chrome.runtime.lastError;
          resolve(res && typeof res === 'object' ? res
            : { ok: false, error: 'no-response' });
        });
      } catch (err) {
        resolve({ ok: false, error: String(err) });
      }
    });
  }

  /* Handler -> may return plain data (wrapped as {ok:true,data}) or a
   * full response object. Never throws across the boundary. */
  function listen(moduleId, handler) {

```

```

chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
  if (!msg || typeof msg !== 'object' || typeof msg.type !== 'string') {
    return false;
  }
  if (msg.module !== moduleId && msg.module !== '*') return false;
  Promise.resolve(handler(msg, sender))
    .then((data) => {
      sendResponse(data && data.__response ? data
        : { ok: true, data: data === undefined ? null : data });
    })
    .catch((err) => {
      sendResponse({ ok: false, error: String(err && err.message || err) });
    });
  return true; // keep the channel open for the async sendResponse
});
}

root.msg = { send, listen };
})(globalThis);

```

shared/module-registry.js

The registry is pure data - no `chrome.*` calls at import time, which is what makes it safe to load in every runtime including the popup. Each module entry carries its catalog information (name, pillar, one-line blurb, default) and, once the part that builds it arrives, its content-script specifications. In Part 0 only the demo module has scripts; Parts 1-10 each show the exact entry to paste into `MODULES` as that module comes online. This is also the single source of truth for the popup's ten rows, so the UI never drifts from what the bootstrap actually registers.

```

shared/module-registry.js
/* shared/module-registry.js - catalog + content-script specs.
 * PURE DATA: no chrome.* access at load time, safe in every runtime. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.Registry) return;

  const PILLARS = [
    { id: 'linkIntel', name: 'Link & Site Intel' },
    { id: 'clickSafe', name: 'Form & Click Safety' },
    { id: 'files', name: 'Files & Downloads' },
    { id: 'annoyance', name: 'Annoyance & Privacy' }
  ];

  const HTTP = ['http://*', 'https://*'];
  const BRIDGE_FILES = [
    'shared/messaging.js',
    'shared/cs-bridge.js',
    'ui/toast.js',
    'ui/verdict-card.js'
  ];
  const BASE_CSS = ['ui/tokens.css', 'ui/components.css'];

  /* Registration spec fields:
   * id          unique script id used by chrome.scripting (persisted!)
   * matches     URL patterns this script set runs on
   * js/css      files injected, root-relative paths
   * runAt       document_start | document_end | document_idle
   * world       ISOLATED (default) | MAIN (only av-indicator uses MAIN)
   * allFrames   boolean
   */
  const MODULES = [
    {
      id: 'demoGreeting', name: 'Demo Greeting', pillar: 'annoyance',
      blurb: 'Dev-only chip that proves toggles are real (removed in Part 13).',
      defaultEnabled: true, devOnly: true,
      scripts: [{
        id: 'sl-cs-demo', matches: HTTP, runAt: 'document_idle',
        world: 'ISOLATED', allFrames: false,
        js: [...BRIDGE_FILES, 'modules/demo-greeting/demo-greeting.js'],
        css: [...BASE_CSS, 'modules/demo-greeting/demo-greeting.css']
      }]
    }
  ]
}

```

```

    }
  },
  { id: 'linkSniper', name: 'Link Sniper', pillar: 'linkIntel',
    blurb: 'Hover any link to see where it really goes.',
    defaultEnabled: true, scripts: null }, // <- filled in Part 1
  { id: 'redirectDetective', name: 'Redirect Detective', pillar: 'linkIntel',
    blurb: 'Follows redirect chains hop by hop.',
    defaultEnabled: true, scripts: null }, // <- Part 2
  { id: 'trustBadge', name: 'Trust Badge', pillar: 'linkIntel',
    blurb: 'Flags trust seals that are just images.',
    defaultEnabled: true, scripts: null }, // <- Part 3
  { id: 'formGuardian', name: 'Form Guardian', pillar: 'clickSafe',
    blurb: 'Stops passwords leaving for the wrong domain.',
    defaultEnabled: true, scripts: null }, // <- Part 4
  { id: 'ghostClick', name: 'Ghost Click Blocker', pillar: 'clickSafe',
    blurb: 'Neutralizes invisible click-stealing overlays.',
    defaultEnabled: true, scripts: null }, // <- Part 5
  { id: 'fakeDownload', name: 'Fake Download Button', pillar: 'clickSafe',
    blurb: 'Marks competing download buttons.',
    defaultEnabled: true, scripts: null }, // <- Part 6
  { id: 'downloadGuard', name: 'Download Guard', pillar: 'files',
    blurb: 'Checks file bytes against their extension.',
    defaultEnabled: true, scripts: null }, // <- Part 7
  { id: 'tabGuard', name: 'Tab Guard', pillar: 'annoyance',
    blurb: 'Contains popup storms in an isolation group.',
    defaultEnabled: true, scripts: null }, // <- Part 8 (SW-only)
  { id: 'cookieTamer', name: 'Cookie Tamer', pillar: 'annoyance',
    blurb: 'Parks consent banners; never clicks Accept.',
    defaultEnabled: true, scripts: null }, // <- Part 9
  { id: 'avIndicator', name: 'Webcam/Mic Indicator', pillar: 'annoyance',
    blurb: 'A chip whenever camera or mic is in use.',
    defaultEnabled: true, scripts: null } // <- Part 10
];

function byId(id) {
  return MODULES.find((m) => m.id === id) || null;
}

function allScriptSpecs() {
  const out = [];
  for (const m of MODULES) if (m.scripts) out.push(...m.scripts);
  return out;
}

function scriptIdsFor(moduleId) {
  const m = byId(moduleId);
  return m && m.scripts ? m.scripts.map((s) => s.id) : [];
}

root.Registry = {
  PILLARS, MODULES, byId, allScriptSpecs, scriptIdsFor,
  BRIDGE_FILES, BASE_CSS
};
})(globalThis);

```

shared/cs-bridge.js

The bridge is the content world's runtime and the enforcement point for the toggle lifecycle inside pages. A module never wires its own listeners for settings changes or teardown - it calls `SL.boot(moduleId, impl)` with up to four hooks (`onEnabled`, `onConfig`, `onDisabled`, `onMessage`) and registers undo functions with `SL.registerCleanup()`. The bridge owns the verdict cache (so ten modules hovering the same link issue one analysis), the status reporter, and the page host calculation that per-site overrides key on.

shared/cs-bridge.js

```

/* shared/cs-bridge.js - content-world runtime for all modules.
 * Modules call SL.boot(id, impl); the bridge owns lifecycle. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.__bridge) return;

```

```

const impls = {}; // moduleId -> implementation hooks
const cleanups = {}; // moduleId -> [undo functions]
const alive = {}; // moduleId -> currently active on this page
const VERDICT_CACHE_MAX = 300;

let pageHost = '';
try { pageHost = location.hostname || ''; } catch (e) { /* non-http(s) */ }

function host() { return pageHost; }

function send(module, type, payload) {
  return root.msg.send(module, type, payload);
}

/* ---- verdict cache: one analysis per URL per page load ---- */
const verdictCache = new Map();
const verdictPending = new Map();

function verdictKey(url, ctx) {
  const c = ctx || {};
  return url + '|' + (c.credentials ? 'c' : '') +
    (c.escalate ? 'e' : '') + (c.resolvedUrl ? 'r' : '');
}

async function verdict(url, ctx) {
  const key = verdictKey(url, ctx);
  if (verdictCache.has(key)) return verdictCache.get(key);
  if (verdictPending.has(key)) return verdictPending.get(key);
  const p = send('engine', 'analyze', { url, context: ctx || {} })
    .then((res) => {
      const v = res.ok ? res.data : { verdict: 'CLEAR', reasons: [],
        score: 0, meta: { error: res.error } };
      verdictPending.delete(key);
      verdictCache.set(key, v);
      if (verdictCache.size > VERDICT_CACHE_MAX) {
        verdictCache.delete(verdictCache.keys().next().value);
      }
      return v;
    });
  verdictPending.set(key, p);
  return p;
}

/* ---- status lines shown in the popup ---- */
function status(moduleId, line) {
  // fire-and-forget; SW writes session storage keyed by tab id
  send(moduleId, 'status', { line: String(line).slice(0, 80) });
}

/* ---- lifecycle ---- */
async function enabledFor(moduleId) {
  const s = await root.Settings.get();
  return root.Settings.isModuleOn(s, moduleId, pageHost);
}

async function configFor(moduleId) {
  const s = await root.Settings.effective(pageHost);
  return s.modules[moduleId] || { enabled: false };
}

function registerCleanup(moduleId, fn) {
  if (typeof fn !== 'function') return;
  (cleanups[moduleId] = cleanups[moduleId] || []).push(fn);
}

function teardown(moduleId) {
  const fns = cleanups[moduleId] || [];
  cleanups[moduleId] = [];
  for (const fn of fns) {
    try { fn(); } catch (e) { /* undo must never throw the bridge */ }
  }
  alive[moduleId] = false;
  const impl = impls[moduleId];
  if (impl && typeof impl.onDisabled === 'function') {

```

```

    try { impl.onDisabled(); } catch (e) { /* ignore */ }
  }
}

async function boot(moduleId, impl) {
  impls[moduleId] = impl || {};
  const on = await enabledFor(moduleId);
  if (on) {
    alive[moduleId] = true;
    const cfg = await configFor(moduleId);
    if (typeof impl.onEnabled === 'function') {
      impl.onEnabled(cfg); // module starts observers here
    }
  }
}

/* ---- broadcast + settings listeners (installed once) ----- */
chrome.runtime.onMessage.addListener((msg) => {
  if (!msg || typeof msg !== 'object') return false;
  if (msg.module !== '*') return false;
  const p = msg.payload || {};
  if (msg.type === 'moduleToggled' && impls[p.moduleId]) {
    if (p.enabled === true) {
      (async () => {
        alive[p.moduleId] = true;
        const cfg = await configFor(p.moduleId);
        const impl = impls[p.moduleId];
        if (typeof impl.onEnabled === 'function') impl.onEnabled(cfg);
      })();
    } else {
      teardown(p.moduleId); // run undo fns, then onDisabled()
    }
  } else if (msg.type === 'globalToggled') {
    if (p.enabled === false) {
      for (const id of Object.keys(impls)) teardown(id);
    }
  } else if (msg.type === 'moduleConfig' && impls[p.moduleId]) {
    const impl = impls[p.moduleId];
    if (typeof impl.onConfig === 'function') impl.onConfig(p.config || {});
  }
  return false; // broadcasts never expect a response
});

/* Local settings edits (options page) reach content scripts through
 * storage.onChange; re-check enablement and config for our modules. */
chrome.storage.onChange.addListener(async (changes, area) => {
  if (area !== 'local' || !changes['sl.settings']) return;
  for (const moduleId of Object.keys(impls)) {
    const on = await enabledFor(moduleId);
    if (on && !alive[moduleId]) {
      alive[moduleId] = true;
      const cfg = await configFor(moduleId);
      const impl = impls[moduleId];
      if (typeof impl.onEnabled === 'function') impl.onEnabled(cfg);
    } else if (!on && alive[moduleId]) {
      teardown(moduleId);
    } else if (on && alive[moduleId]) {
      const cfg = await configFor(moduleId);
      const impl = impls[moduleId];
      if (typeof impl.onConfig === 'function') impl.onConfig(cfg);
    }
  }
});

root.__bridge = true;
root.host = host;
root.send = send;
root.verdict = verdict;
root.status = status;
root.boot = boot;
root.registerCleanup = registerCleanup;
root.teardown = teardown;
})(globalThis);

```

0.8 The engine

Five files form the verdict engine, and they are the only code in the product allowed to produce risk numbers. `url-analyzer.js` extracts weighted signals; `known-brands.js` and `domain-intel.js` feed it; `redirect-resolver.js` (service-worker-only) follows redirect chains; `verdict-engine.js` combines everything into the three-word verdict. All engine exports carry JSDoc, because every module in Parts 1-10 consumes this API and the contract must be unambiguous.

engine/known-brands.js

The brand list is the reference data behind typo-squatting detection. Each entry maps a brand token to its official registrable domains. Detection runs in two modes: **exact** (the site's registrable domain is a listed official domain of a brand it claims to be, or the registrable label equals the brand token) and **fuzzy** (the registrable label is within Damerau-Levenshtein distance 2 of a brand token - `paypal`, `microsoft`, `amazon`). The list is deliberately conservative: brands people actually type and trust, not a directory of the internet.

```
engine/known-brands.js
```

```
/* engine/known-brands.js - brand tokens + official domains.
 * Reference data for typo-squat detection. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  root.Engine = root.Engine || {};
  if (root.Engine.Brands) return;

  function b(brand, ...domains) { return { brand, domains }; }

  const KNOWN_BRANDS = [
    b('google', 'google.com', 'youtube.com', 'googleapis.com'),
    b('apple', 'apple.com', 'icloud.com'),
    b('microsoft', 'microsoft.com', 'live.com', 'office.com'),
    b('outlook', 'outlook.com'),
    b('bing', 'bing.com'),
    b('paypal', 'paypal.com'),
    b('ebay', 'ebay.com'),
    b('amazon', 'amazon.com', 'amzn.to'),
    b('alibaba', 'alibaba.com'),
    b('aliexpress', 'aliexpress.com'),
    b('temu', 'temu.com'),
    b('shein', 'shein.com'),
    b('wish', 'wish.com'),
    b('facebook', 'facebook.com', 'fb.com'),
    b('instagram', 'instagram.com'),
    b('whatsapp', 'whatsapp.com'),
    b('meta', 'meta.com'),
    b('threads', 'threads.net'),
    b('twitter', 'twitter.com', 'x.com'),
    b('tiktok', 'tiktok.com'),
    b('snapchat', 'snapchat.com'),
    b('pinterest', 'pinterest.com'),
    b('reddit', 'reddit.com'),
    b('tumblr', 'tumblr.com'),
    b('twitch', 'twitch.tv'),
    b('discord', 'discord.com', 'discord.gg'),
    b('steam', 'steampowered.com', 'steamcommunity.com'),
    b('epicgames', 'epicgames.com'),
    b('blizzard', 'blizzard.com', 'battle.net'),
    b('roblox', 'roblox.com'),
    b('minecraft', 'minecraft.net'),
    b('ubisoft', 'ubisoft.com'),
    b('ea', 'ea.com'),
    b('nintendo', 'nintendo.com'),
    b('playstation', 'playstation.com'),
    b('spotify', 'spotify.com'),
    b('netflix', 'netflix.com'),
    b('hulu', 'hulu.com'),
    b('disney', 'disney.com', 'disneyplus.com'),
    b('linkedin', 'linkedin.com'),
    b('adobe', 'adobe.com'),
  ]
```

```

b('dropbox', 'dropbox.com'),
b('box', 'box.com'),
b('zoom', 'zoom.us', 'zoom.com'),
b('slack', 'slack.com'),
b('notion', 'notion.so', 'notion.com'),
b('figma', 'figma.com'),
b('canva', 'canva.com'),
b('atlassian', 'atlassian.com'),
b('jira', 'atlassian.com'),
b('trello', 'trello.com'),
b('asana', 'asana.com'),
b('github', 'github.com'),
b('gitlab', 'gitlab.com'),
b('stackoverflow', 'stackoverflow.com'),
b('openai', 'openai.com', 'chatgpt.com'),
b('anthropic', 'anthropic.com', 'claude.ai'),
b('perplexity', 'perplexity.ai'),
b('coinbase', 'coinbase.com'),
b('binance', 'binance.com'),
b('kraken', 'kraken.com'),
b('robinhood', 'robinhood.com'),
b('vanguard', 'vanguard.com'),
b('fidelity', 'fidelity.com'),
b('schwab', 'schwab.com'),
b('chase', 'chase.com'),
b('bankofamerica', 'bankofamerica.com'),
b('wellsfargo', 'wellsfargo.com'),
b('citi', 'citi.com', 'citibank.com'),
b('hsbc', 'hsbc.com'),
b('barclays', 'barclays.com', 'barclays.co.uk'),
b('lloyds', 'lloydsbank.com', 'lloydsbank.co.uk'),
b('santander', 'santander.com', 'santander.co.uk'),
b('revolut', 'revolut.com'),
b('klarna', 'klarna.com'),
b('stripe', 'stripe.com'),
b('square', 'squareup.com'),
b('shopify', 'shopify.com'),
b('intuit', 'intuit.com'),
b('turbotax', 'turbotax.intuit.com'),
b('quickbooks', 'quickbooks.intuit.com'),
b('irs', 'irs.gov'),
b('govuk', 'gov.uk', 'www.gov.uk'),
b('canada', 'canada.ca'),
b('dhl', 'dhl.com'),
b('fedex', 'fedex.com'),
b('ups', 'ups.com'),
b('usps', 'usps.com'),
b('royalmail', 'royalmail.com'),
b('booking', 'booking.com'),
b('airbnb', 'airbnb.com'),
b('expedia', 'expedia.com'),
b('tripadvisor', 'tripadvisor.com'),
b('marriott', 'marriott.com'),
b('hilton', 'hilton.com'),
b('ryanair', 'ryanair.com'),
b('delta', 'delta.com'),
b('united', 'united.com'),
b('americanairlines', 'aa.com'),
b('lufthansa', 'lufthansa.com'),
b('ems', 'ems.com'),
b('docuSign', 'docuSign.com'),
b('wettransfer', 'wettransfer.com'),
b('mediafire', 'mediafire.com'),
b('mega', 'mega.nz'),
b('rapidshare', 'rapidshare.com')
];

const DOMAIN_SET = new Set();
const TOKEN_MAP = new Map();
for (const entry of KNOWN_BRANDS) {
    TOKEN_MAP.set(entry.brand, entry);
    for (const d of entry.domains) DOMAIN_SET.add(d);
}
/** True when host's registrable domain is an official brand domain. */

```

```

function isOfficialDomain(registrable) {
    return DOMAIN_SET.has(registrable);
}

/** Exact brand whose token equals the registrable label, if any. */
function brandForToken(label) {
    return TOKEN_MAP.get(label) || null;
}

/** Damerau-Levenshtein distance, capped for speed. */
function damerau(a, bToken, cap) {
    const la = a.length, lb = bToken.length;
    if (Math.abs(la - lb) > cap) return cap + 1;
    const d = Array.from({ length: la + 1 }, () => new Array(lb + 1).fill(0));
    for (let i = 0; i <= la; i++) d[i][0] = i;
    for (let j = 0; j <= lb; j++) d[0][j] = j;
    for (let i = 1; i <= la; i++) {
        for (let j = 1; j <= lb; j++) {
            const cost = a[i - 1] === bToken[j - 1] ? 0 : 1;
            d[i][j] = Math.min(d[i - 1][j] + 1, d[i][j - 1] + 1,
                               d[i - 1][j - 1] + cost);
            if (i > 1 && j > 1 && a[i - 1] === bToken[j - 2] &&
                a[i - 2] === bToken[j - 1]) {
                d[i][j] = Math.min(d[i][j], d[i - 2][j - 2] + 1); // transposition
            }
        }
    }
    return d[la][lb];
}

/** Brand whose token is within edit distance 2 of label (typo-squat). */
function typoBrandFor(label) {
    if (!label || label.length < 4) return null;
    const cap = label.length <= 4 ? 1 : 2;
    for (const entry of KNOWN_BRANDS) {
        if (Math.abs(entry.brand.length - label.length) > cap) continue;
        if (damerau(label, entry.brand, cap) <= cap) return entry;
    }
    return null;
}

root.Engine.Brands = {
    KNOWN_BRANDS, isOfficialDomain, brandForToken, typoBrandFor, damerau
};
})(globalThis);

```

engine/domain-intel.js

Domain intelligence starts with registrable-domain extraction, which needs Public Suffix List awareness. Bundling the full 14,000-entry PSL would be correct but heavy for a guide; instead we ship a curated subset of roughly eighty multi-label suffixes that covers effectively all traffic a real user hits (`co.uk`, `com.au`, `co.jp`, `com.br`, ...), plus a documented upgrade path. The remaining helpers classify IP literals, count subdomain depth, detect hyphen stuffing, and recognize machine-generated-looking labels - each a named function so the analyzer stays readable.

engine/domain-intel.js

```

/* engine/domain-intel.js - registrable domains + host shape helpers. */
(function (global) {
    'use strict';
    const root = global.SL = global.SL || {};
    root.Engine = root.Engine || {};
    if (root.Engine.Domain) return;

    /* Curated multi-label public suffixes (subset of the PSL).
     * Single-label TLDs (com, net, org, io, ...) need no entry here. */
    const MULTI_SUFFIXES = new Set([
        'co.uk', 'org.uk', 'ac.uk', 'gov.uk', 'me.uk', 'net.uk', 'ltd.uk',
        'com.au', 'net.au', 'org.au', 'gov.au', 'edu.au', 'id.au',
        'co.nz', 'net.nz', 'org.nz', 'govt.nz', 'ac.nz',
        'co.jp', 'or.jp', 'ne.jp', 'ac.jp', 'go.jp',
        'co.za', 'org.za', 'net.za', 'web.za', 'gov.za',
    ]);

```

```

'com.br', 'net.br', 'org.br', 'gov.br', 'edu.br',
'com.mx', 'org.mx', 'net.mx', 'gob.mx', 'edu.mx',
'com.ar', 'net.ar', 'org.ar', 'gob.ar', 'edu.ar',
'com.cn', 'net.cn', 'org.cn', 'gov.cn', 'edu.cn',
'com.hk', 'net.hk', 'org.hk', 'gov.hk', 'edu.hk',
'com.tw', 'net.tw', 'org.tw', 'gov.tw', 'edu.tw',
'com.sg', 'net.sg', 'org.sg', 'gov.sg', 'edu.sg',
'com.my', 'net.my', 'org.my', 'gov.my', 'edu.my',
'co.in', 'net.in', 'org.in', 'gov.in', 'ac.in', 'edu.in',
'com.tr', 'net.tr', 'org.tr', 'gov.tr', 'edu.tr',
'com.ua', 'net.ua', 'org.ua', 'gov.ua', 'edu.ua',
'co.il', 'org.il', 'net.il', 'gov.il', 'ac.il',
'com.ru', 'net.ru', 'org.ru',
'co.kr', 'or.kr', 'ne.kr', 'go.kr',
'com.vn', 'net.vn', 'org.vn', 'gov.vn',
'com.ph', 'net.ph', 'org.ph',
'com.co', 'net.co', 'org.co', 'gov.co',
'com.pe', 'org.pe', 'net.pe', 'gob.pe',
'com.ve', 'net.ve', 'org.ve', 'gob.ve',
'com.eg', 'net.eg', 'org.eg', 'gov.eg',
'com.sa', 'net.sa', 'org.sa', 'gov.sa',
'com.ae', 'net.ae', 'org.ae', 'gov.ae',
'com.qa', 'net.qa', 'org.qa', 'gov.qa',
'com.kw', 'net.kw', 'org.kw',
'com.pk', 'net.pk', 'org.pk', 'gov.pk',
'com.bd', 'net.bd', 'org.bd',
'com.ng', 'net.ng', 'org.ng', 'gov.ng',
'co.ke', 'or.ke', 'go.ke', 'ac.ke',
'com.gh', 'org.gh', 'gov.gh',
'com.th', 'net.th', 'org.th', 'go.th',
'co.th', 'in.th', 'or.th', 'go.th'
]);

function splitHost(hostname) {
    return String(hostname || '').toLowerCase().replace(/\.$/ , '').split('.');
}

/** eTLD+1: 'login.paypal.co.uk' -> 'paypal.co.uk'. Falls back to the
 * full host when nothing matches (single-label or odd TLD). */
function eTLDPlus1(hostname) {
    const labels = splitHost(hostname);
    if (labels.length <= 2) return labels.join('.');
    // longest matching multi-label suffix wins
    for (let take = 3; take <= Math.min(labels.length, 5); take++) {
        const suffix = labels.slice(-take).join('.');
        if (MULTI_SUFFIXES.has(suffix)) {
            return labels.slice(-(take + 1)).join('.');
        }
    }
    return labels.slice(-2).join('.');
}

/** The brand-carrying label of the registrable domain. */
function registrableLabel(hostname) {
    const reg = eTLDPlus1(hostname);
    return reg.split('.')[0] || reg;
}

function isIpLiteral(host) {
    if (!host) return false;
    if (/^\d{1,3}(\.\d{1,3}){3}$/.test(host)) return true; // IPv4
    if (host.startsWith '[' && host.endsWith(']')) return true; // IPv6
    if (host.includes(':') && host.includes('::')) return true; // bare IPv6
    return false;
}

/** Labels to the left of the registrable domain. */
function subdomainDepth(hostname) {
    const labels = splitHost(hostname);
    const regLabels = eTLDPlus1(hostname).split('.').length;
    return Math.max(0, labels.length - regLabels);
}

function hyphenCount(str) {

```

```

    return (String(str).match(/-/g) || []).length;
}

/** Heuristic for DGA-style labels: long + almost vowel-free. */
function looksMachineGenerated(label) {
    if (!label || label.length < 13) return false;
    const vowels = (label.match(/[aeiou]/g) || []).length;
    return vowels / label.length < 0.30;
}

root.Engine.Domain = {
    MULTI_SUFFIXES, splitHost, eTLDPlus1, registrableLabel,
    isIpLiteral, subdomainDepth, hyphenCount, looksMachineGenerated
};
})(globalThis);

```

engine/url-analyzer.js

The analyzer turns one URL (plus optional context) into a list of weighted signals. Every weight is a named constant; every signal carries a `plainText` sentence written for a non-technical reader, because the UI renders exactly that sentence and nothing else. Additive scoring with a cap happens in the verdict engine; this file only reports what it sees.

```

engine/url-analyzer.js
/* engine/url-analyzer.js - weighted signal extraction for one URL. */
(function (global) {
    'use strict';
    const root = global.SL = global.SL || {};
    root.Engine = root.Engine || {};
    if (root.Engine.Url) return;

    /* All weights are named constants - the tuning knobs of the engine.
     * Any single URL-only combination must stay below the engine's
     * URL_CAP (39) so pure URL shape can never trigger INTERCEPTED. */
    const SIGNAL_WEIGHTS = {
        PUNYCODE_IDN: 45,
        BRAND_TYPO: 40,
        BRAND_IN_SUBDOMAIN: 30,
        INVISIBLE_CHARS: 50,
        IP_LITERAL: 30,
        USERINFO_URL: 25,
        NONSTD_PORT: 15,
        SUSPICIOUS_KEYWORD: 15,
        SUSPICIOUS_TLD: 12,
        DEEP_SUBDOMAINS: 12,
        HYPHEN_STUFFED: 10,
        LONG_RANDOM_LABEL: 10,
        KNOWN_SHORTENER: 10,
        PLAIN_HTTP: 8
    };

    const SUSPICIOUS_TLDS = [
        'zip', 'mov', 'top', 'gq', 'tk', 'cf', 'ml', 'ga', 'work', 'click',
        'link', 'country', 'stream', 'download', 'loan', 'review', 'icu',
        'cam', 'rest', 'bs'
    ];

    const SHORTENERS = [
        'bit.ly', 't.co', 'tinyurl.com', 'is.gd', 'cutt.ly', 'shorturl.at',
        'rebrand.ly', 'buff.ly', 'ow.ly', 's.id', 'v.gd', 'rb.gy', 'tiny.cc',
        'shrtco.de', 't.ly', 'soo.gd', 'clck.ru', 'shorte.st', 'gg.gg',
        'short.gy', 'budurl.com', 'qr.ae'
    ];

    const HARVEST_KEYWORDS = [
        'login', 'signin', 'verify', 'verification', 'secure', 'account',
        'update', 'confirm', 'recovery', 'wallet', 'billing', 'invoice',
        'password', 'support'
    ];

    /* Zero-width + bidi control characters that can disguise link text
     * or hostnames when rendered. Checked in both display text and URL. */

```

```

const INVISIBLE_RE = /[\u200b-\u200f\u202a-\u202e\u2060-\u2064\u2066-\u2069\u206f]/;

function analyzeUrl(url, context) {
  const ctx = context || {};
  const signals = [];
  const add = (signal, plainText) => signals.push({
    signal, weight: SIGNAL_WEIGHTS[signal] || 0, plainText
  });

  let u = null;
  try { u = new URL(url); } catch (e) {
    add('PUNYCODE_IDN', 'This is not a well-formed address.');
```

```

    return { signals, host: '', registrable: '', meta: { parsed: false } };
  }

  const host = (u.hostname || '').toLowerCase();
  const D = root.Engine.Domain, B = root.Engine.Brands;
  const registrable = D.eTLDPlus1(host);
  const regLabel = D.registrableLabel(host);
  const labels = D.splitHost(host);

  // 1. punycode / non-ASCII host
  if (host.startsWith('xn--') || labels.some((l) => l.startsWith('xn--')) ||
    /^[^\x00-\x7f]/.test(host)) {
    add('PUNYCODE_IDN',
      'The address contains encoded characters that can be used ' +
      'to imitate real sites.');
```

```

  }

  // 2. typo-squat of a known brand
  if (!D.isIpLiteral(host) && !B.isOfficialDomain(registrable)) {
    const typo = B.typoBrandFor(regLabel);
    if (typo) {
      add('BRAND_TYPO', 'Looks similar to ' + typo.brand +
        '.com - the official domain is ' + typo.domains[0] + '.');
```

```

    }
  }

  // 3. brand name parked in a subdomain of someone else
  if (!B.isOfficialDomain(registrable)) {
    for (const label of labels.slice(0, -2)) {
      const exact = B.brandForToken(label);
      if (exact) {
        add('BRAND_IN_SUBDOMAIN', 'Uses \'' + exact.brand +
          '\' in the subdomain, but the site is really ' +
          registrable + '.');
```

```

        break;
      }
    }
  }

  // 4. invisible characters in display text (spec: text vs href)
  if (ctx.displayText && INVISIBLE_RE.test(ctx.displayText)) {
    add('INVISIBLE_CHARS',
      'The visible link text contains invisible characters that ' +
      'can disguise where it leads.');
```

```

  }

  // 5. raw IP host
  if (D.isIpLiteral(host)) {
    add('IP_LITERAL',
      'Points to a raw numeric address instead of a named website.');
```

```

  }

  // 6. userinfo trick - http://paypal.com@evil.example/
  if (u.username || u.password) {
    add('USERINFO_URL',
      'Everything before the \'@\'' in this address is decoration; ' +
      'the real destination is after it.');
```

```

  }

  // 7. unusual port
  const port = u.port ? Number(u.port) : null;
  if (port && port !== 80 && port !== 443 && port !== 8080) {

```

```

    add('NONSTD_PORT', 'Uses an unusual port number (' + port + ').');
  }

  // 8. keyword-stuffed registrable label
  if (!D.isIpLiteral(host)) {
    for (const kw of HARVEST_KEYWORDS) {
      if (regLabel.includes(kw) && regLabel !== kw) {
        add('SUSPICIOUS_KEYWORD', 'The domain name contains \'' + kw +
          '\', typical of credential-harvesting sites.');
```

break;

```
      }
    }
  }

  // 9. TLD reputation
  const tld = labels[labels.length - 1];
  if (SUSPICIOUS_TLDS.includes(tld)) {
    add('SUSPICIOUS_TLD',
      'Uses the \'' + tld + ' ending, which is common in scam sites.');
```

}

```

  // 10. subdomain depth
  if (D.subdomainDepth(host) >= 4) {
    add('DEEP_SUBDOMAINS',
      'Unusually deep subdomains can imitate official site names.');
```

}

```

  // 11. hyphen stuffing
  if (D.hyphenCount(regLabel) >= 3) {
    add('HYPHEN_STUFFED',
      'The domain name packs together many dashes, a disguise pattern.');
```

}

```

  // 12. machine-generated label
  if (D.looksMachineGenerated(regLabel)) {
    add('LONG_RANDOM_LABEL',
      'The domain name looks machine-generated rather than human-named.');
```

}

```

  // 13. shortener
  if (SHORTENERS.includes(registrable)) {
    add('KNOWN_SHORTENER',
      'Shortened link - its destination is not shown.');
```

}

```

  // 14. plain http
  if (u.protocol === 'http:') {
    add('PLAIN_HTTP', 'The connection is not encrypted (http).');
```

}

```

  return {
    signals,
    host, registrable,
    meta: {
      parsed: true, protocol: u.protocol.replace(':', ''),
      isShortener: SHORTENERS.includes(registrable),
      port, labels
    }
  };
}

root.Engine.Url = { SIGNAL_WEIGHTS, analyzeUrl, SHORTENERS };
})(globalThis);
```

engine/verdict-engine.js

The verdict engine folds one or two URL analyses (the original, and optionally a resolved final URL for shortlinks) into a single verdict object. The safety cap is the heart of the design: the summed URL-only score is clamped at 39 while interception requires 40, so no pile-up of heuristics - however paranoid - can stop a user who has not been presented with hard module-level evidence. Only a module passing `escalate: true` (Form Guardian with credentials in flight, Download Guard with byte-level mismatch) can legally cross that line.

```

/* engine/verdict-engine.js - the ONLY place a verdict is produced. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  root.Engine = root.Engine || {};
  if (root.Engine.analyze) return;

  const THRESHOLDS = {
    SECOND_LOOK: 20, // score at or above this earns an amber pill
    INTERCEPT: 40, // never reachable from URL signals alone
    URL_CAP: 39 // hard clamp for URL-only analysis
  };

  const VERDICT = {
    CLEAR: 'CLEAR',
    SECOND_LOOK: 'SECOND_LOOK',
    INTERCEPTED: 'INTERCEPTED'
  };

  /**
   * Analyze a URL and produce a SecondLook verdict.
   * @param {string} url - the URL to judge.
   * @param {object} [context] - extra evidence from the calling module:
   *   displayText visible text of the link/anchor, if any
   *   credentials true when credentials are about to be sent (P4)
   *   escalate true when the module has hard evidence and is
   *             allowed to request INTERCEPTED
   *   resolvedUrl final URL after redirect resolution (P1/P2)
   * @returns {Promise<{verdict:string, reasons:Array, score:number,
   *                   meta:object}>}> reasons are plain-language,
   *   strongest first, already capped to three.
   */
  function analyze(url, context) {
    const ctx = context || {};
    const Url = root.Engine.Url;

    const first = Url.analyzeUrl(url, ctx);
    const signals = first.signals.slice();
    let meta = Object.assign({}, first.meta, { host: first.host });

    // Shortlinks: judge the resolved destination too, if provided.
    if (ctx.resolvedUrl && ctx.resolvedUrl !== url) {
      const second = Url.analyzeUrl(ctx.resolvedUrl, ctx);
      for (const sig of second.signals) {
        if (sig.signal === 'KNOWN_SHORTENER') continue;
        const dupe = signals.find((s) => s.signal === sig.signal);
        if (!dupe) signals.push(sig);
      }
      meta.resolvedHost = second.host;
    }

    // Score: additive, then clamped for URL-only analysis.
    let score = 0;
    for (const sig of signals) score += sig.weight;
    if (!ctx.escalate) score = Math.min(score, THRESHOLDS.URL_CAP);
    else score = Math.min(score, 100);

    let verdict;
    if (ctx.escalate && ctx.credentials && score >= THRESHOLDS.INTERCEPT) {
      verdict = VERDICT.INTERCEPTED;
    } else if (score >= THRESHOLDS.SECOND_LOOK) {
      verdict = VERDICT.SECOND_LOOK;
    } else {
      verdict = VERDICT.CLEAR;
    }

    // One visible warning per event: top reasons only, plain text.
    const reasons = signals
      .filter((s) => s.weight >= 10)
      .sort((a, b) => b.weight - a.weight)
      .slice(0, 3)
      .map((s) => s.plainText);
  }
}

```

```

    return Promise.resolve({ verdict, reasons, score, meta });
  }

  root.Engine.analyze = analyze;
  root.Engine.VERDICT = VERDICT;
  root.Engine.THRESHOLDS = THRESHOLDS;
})(globalThis);

```

engine/redirect-resolver.js

Redirect resolution is where MV3 bites everyone, so this file exists to bite back. The naive approach - `fetch(url, {redirect: 'manual'})` and reading `Location` - **does not work**: Chromium returns an *opaque filtered response* whose headers are unreadable, by design. The working approach is the opposite shape: issue a normal *following* fetch from the service worker (which host permissions allow cross-origin), and observe the hop chain through `webRequest.onBeforeRedirect` events keyed by `requestId`. The final URL comes from `response.url` as ground truth. Resolutions are serialized through a promise queue so concurrent callers can never mix up request ids.

```

engine/redirect-resolver.js
/* engine/redirect-resolver.js - SERVICE WORKER ONLY (needs webRequest).
 *
 * The trap this file exists to avoid:
 *   fetch(url, {redirect:'manual'}) -> OPAQUE response, headers
 *   unreadable. Useless for chain inspection.
 * The working shape: follow redirects with fetch(), capture each hop
 * via webRequest.onBeforeRedirect keyed by requestId. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  root.Engine = root.Engine || {};
  if (root.Engine.Redirect) return;

  const MAX_HOPS = 8;          // tuning: stop following after this many hops
  const TIMEOUT_MS = 6000;     // tuning: per-resolution budget

  let queue = Promise.resolve(); // serialize: one resolution in flight
  let collector = null;          // requestId -> [{fromUrl,toUrl,status}]
  let listening = false;

  function ensureListener() {
    if (listening) return;
    listening = true;
    // OBSERVER ONLY: no blocking, no header modification.
    chrome.webRequest.onBeforeRedirect.addListener((details) => {
      if (!collector) return;
      const list = collector.get(details.requestId) || [];
      list.push({
        fromUrl: details.url,
        toUrl: details.redirectUrl,
        status: details.statusCode
      });
      collector.set(details.requestId, list);
    }, { urls: ['<all_urls>'] });
  }

  function buildChain(startUrl, events, finalUrl) {
    const chain = [startUrl];
    let cursor = startUrl;
    for (const ev of events) {
      if (ev.fromUrl === cursor) { chain.push(ev.toUrl); cursor = ev.toUrl; }
    }
    if (finalUrl && chain[chain.length - 1] !== finalUrl) {
      chain.push(finalUrl);
    }
    return chain;
  }

  async function resolveOnce(url) {
    collector = new Map();
    ensureListener();
    const ctrl = new AbortController();

```

```

const timer = setTimeout(() => ctrl.abort(), TIMEOUT_MS);
let finalUrl = url;
let error = null;
try {
  const res = await fetch(url, {
    redirect: 'follow',
    credentials: 'omit',    // never send the user's cookies upstream
    cache: 'no-store',
    signal: ctrl.signal
  });
  finalUrl = res.url || url;
} catch (err) {
  error = String((err && err.message) || err);
} finally {
  clearTimeout(timer);
}
// Pick the event list that starts at our URL; prefer one that also
// ends at our finalUrl (guards against unrelated parallel requests).
let events = null;
for (const list of collector.values()) {
  if (!list.length || list[0].fromUrl !== url) continue;
  if (!events) events = list;
  if (finalUrl && list[list.length - 1].toUrl === finalUrl) { events = list; break; }
}
collector = null;
const chain = buildChain(url, events || [], finalUrl)
  .slice(0, MAX_HOPS + 1);
return {
  chain,
  finalUrl,
  hops: Math.max(0, chain.length - 1),
  error
};
}

/**
 * Resolve a URL's redirect chain.
 * @param {string} url
 * @returns {Promise<{chain:string[], finalUrl:string, hops:number,
 *                    error:(string|null)}>}
 */
function resolve(url) {
  const run = queue.then(() => resolveOnce(url));
  queue = run.then(() => undefined, () => undefined); // keep queue alive
  return run;
}

root.Engine.Redirect = { resolve, MAX_HOPS, TIMEOUT_MS };
})(globalThis);

```

DEVIATION NOTE - The domain-intel suffix list is a curated subset of the Public Suffix List, not the full PSL. Rare multi-label suffixes (e.g. some ccTLD second-level zones) may mis-split. Upgrading later is a data swap: replace MULTI_SUFFIXES with a generated Set from the full public_suffix_list.dat and keep the same function signatures.

TIP - Keep an eye on the engine's unit behaviors with the DevTools console of the service worker (chrome://extensions -> SecondLook -> "service worker"). After Part 0 loads you can type `SL.Engine.analyze('http://paypal-login.com/verify')` in that console and watch the verdict object come back - the fastest sanity check in the whole project.

0.9 The design system

The UI kit is two stylesheets and two scripts, and it is the only visual vocabulary every module is allowed to speak. `tokens.css` defines every color, spacing, radius, font size and z-index as CSS custom properties, in both light and dark mode, driven by `prefers-color-scheme`. `components.css` implements the shared component classes: verdict pill, toggle switch, module row, card, toast, banner, intercept layer, indicator chip, stat tile and empty state. `toast.js` and

`verdict-card.js` are their behavioral halves, written as classic scripts so the identical files run in content worlds and extension pages.

Three rules from the product spec are enforced here rather than hoped for later. No gradients anywhere - flat surfaces plus a 1px border plus a soft shadow. No emoji ever - the only icons are inline SVG strokes. And every interactive element is a real button or a with a visible `:focus-visible` ring, keyboard dismissal on Escape, and animations that switch off entirely under `prefers-reduced-motion`. Contrast targets are WCAG AA: verdict text colors are chosen at 4.5:1 or better on their surfaces in both modes.

ui/tokens.css

```
ui/tokens.css
/* ui/tokens.css - design tokens; loaded by extension pages AND injected
 * into pages. Page usage: everything sits under a .sl-root wrapper. */

:root, .sl-root {
  --sl-font: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;

  /* ink scale (light mode) */
  --sl-ink: #1f2933;          /* primary text */
  --sl-ink-2: #52606d;        /* secondary text */
  --sl-ink-3: #7b8794;        /* muted / meta */
  --sl-line: #cbd2d9;          /* hairlines, borders */
  --sl-surface: #ffffff;        /* cards, toasts */
  --sl-surface-2: #f5f7fa;      /* page-ish fills */
  --sl-focus: #2563eb;         /* focus ring */

  /* verdict colors - AA on --sl-surface in light mode */
  --sl-ok: #15803d;            /* CLEAR */
  --sl-ok-bg: #ecfdf3;
  --sl-warn: #b45309;           /* SECOND LOOK */
  --sl-warn-bg: #fff7e6;
  --sl-danger: #b91c1c;         /* INTERCEPTED */
  --sl-danger-bg: #fdecec;

  /* spacing scale (8px base) */
  --sl-s1: 4px; --sl-s2: 8px; --sl-s3: 12px;
  --sl-s4: 16px; --sl-s5: 24px; --sl-s6: 32px;

  /* type scale */
  --sl-fz-tiny: 11px; --sl-fz-small: 12px;
  --sl-fz-base: 13px; --sl-fz-title: 15px;

  /* radii and motion */
  --sl-r-lg: 12px; --sl-r-md: 8px; --sl-r-pill: 999px;
  --sl-t-fast: 150ms;

  /* z-index scale - our UI always outranks page UI */
  --sl-z-chip: 8000;
  --sl-z-banner: 9000;
  --sl-z-toast: 9500;
  --sl-z-modal: 10000;

  --sl-shadow: 0 4px 16px rgba(15, 23, 42, 0.14);
}

@media (prefers-color-scheme: dark) {
  :root, .sl-root {
    --sl-ink: #e6eaf0;
    --sl-ink-2: #a8b3c1;
    --sl-ink-3: #7e8a99;
    --sl-line: #3a4654;
    --sl-surface: #1f2933;
    --sl-surface-2: #17202a;
    --sl-focus: #60a5fa;

    --sl-ok: #34d399;           --sl-ok-bg: rgba(20, 83, 45, 0.35);
    --sl-warn: #fbbf24;          --sl-warn-bg: rgba(120, 53, 15, 0.35);
    --sl-danger: #f87171;        --sl-danger-bg: rgba(127, 29, 29, 0.4);
  }
}
```

```

    --sl-shadow: 0 4px 16px rgba(0, 0, 0, 0.5);
  }
}

/* Motion opt-out: animations become instant, pulses become static. */
@media (prefers-reduced-motion: reduce) {
  .sl-root *, body.sl-page * {
    animation-duration: 0.001s !important;
    transition-duration: 0.001s !important;
  }
}

```

ui/components.css

ui/components.css

```

/* ui/components.css - shared component classes. Page-injected UI must
 * sit inside an element with class "sl-root". Extension pages put
 * class "sl-page" on <body> and also sl-root for token inheritance. */

/* ---- scope reset: our subtree only ----- */
.sl-root {
  all: initial;
  display: block;
}
.sl-root, .sl-root * {
  box-sizing: border-box;
  font-family: var(--sl-font);
  font-size: var(--sl-fz-base);
  line-height: 1.45;
  color: var(--sl-ink);
  letter-spacing: normal;
  text-align: left;
}
.sl-root button, .sl-root a {
  all: unset;
  cursor: pointer;
  box-sizing: border-box;
}

/* ---- focus ring (keyboard users only) ----- */
.sl-root :focus-visible, body.sl-page :focus-visible {
  outline: 2px solid var(--sl-focus);
  outline-offset: 2px;
  border-radius: var(--sl-r-sm, 6px);
}

/* ---- verdict pill ----- */
.sl-pill {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 3px 10px 3px 8px;
  border-radius: var(--sl-r-pill);
  font-size: var(--sl-fz-small);
  font-weight: 600;
  border: 1px solid transparent;
  white-space: nowrap;
}
.sl-pill::before {
  content: "";
  width: 7px;
  height: 7px;
  border-radius: 50%;
  background: currentColor;
  flex: 0 0 auto;
}
.sl-pill--clear { color: var(--sl-ok); background: var(--sl-ok-bg); }
.sl-pill--warn { color: var(--sl-warn); background: var(--sl-warn-bg); }
.sl-pill--danger { color: var(--sl-danger); background: var(--sl-danger-bg); }
.sl-pill--muted { color: var(--sl-ink-3); background: var(--sl-surface-2); }

/* ---- card (detail popups: chain view, seal info) ----- */
.sl-card {

```

```

    position: fixed;
    z-index: var(--sl-z-modal);
    width: min(380px, calc(100vw - 24px));
    background: var(--sl-surface);
    border: 1px solid var(--sl-line);
    border-radius: var(--sl-r-lg);
    box-shadow: var(--sl-shadow);
    padding: var(--sl-s3);
}
.sl-card__title {
    font-size: var(--sl-fz-title);
    font-weight: 700;
    margin: 0 0 var(--sl-s1) 0;
    padding-right: 22px;
}
.sl-card__close {
    position: absolute;
    top: 8px; right: 8px;
    width: 22px; height: 22px;
    border-radius: 6px;
    color: var(--sl-ink-3);
    font-size: 14px;
    text-align: center;
}
.sl-card__row {
    display: flex;
    justify-content: space-between;
    gap: var(--sl-s2);
    padding: var(--sl-s1) 0;
    border-bottom: 1px solid var(--sl-line);
    font-size: var(--sl-fz-small);
}
.sl-card__row:last-of-type { border-bottom: 0; }
.sl-card__row b { font-weight: 600; }
.sl-card__reason {
    display: flex;
    gap: var(--sl-s1);
    margin: var(--sl-s1) 0;
    font-size: var(--sl-fz-small);
    color: var(--sl-ink-2);
}

/* ---- toggle switch ----- */
.sl-toggle {
    position: relative;
    display: inline-block;
    width: 34px;
    height: 20px;
    flex: 0 0 auto;
}
.sl-toggle input {
    opacity: 0;
    width: 100%;
    height: 100%;
    margin: 0;
    position: absolute;
    cursor: pointer;
    z-index: 1;
}
.sl-toggle__track {
    position: absolute;
    inset: 0;
    background: var(--sl-line);
    border-radius: var(--sl-r-pill);
    transition: background var(--sl-t-fast);
}
.sl-toggle__track::after {
    content: "";
    position: absolute;
    top: 2px; left: 2px;
    width: 16px; height: 16px;
    border-radius: 50%;
    background: #fff;
    box-shadow: 0 1px 2px rgba(0, 0, 0, 0.3);
}

```

```

    transition: transform var(--sl-t-fast);
}
.sl-toggle input:checked + .sl-toggle__track {
    background: var(--sl-ok);
}
.sl-toggle input:checked + .sl-toggle__track::after {
    transform: translateX(14px);
}
.sl-toggle input:disabled + .sl-toggle__track {
    opacity: 0.5;
    cursor: default;
}

/* ---- module row (popup + options lists) ----- */
.sl-row {
    display: flex;
    align-items: center;
    gap: var(--sl-s2);
    padding: var(--sl-s2) var(--sl-s3);
    border-radius: var(--sl-r-md);
}
.sl-row:hover { background: var(--sl-surface-2); }
.sl-row__icon {
    width: 26px; height: 26px;
    flex: 0 0 auto;
    border-radius: var(--sl-r-md);
    display: grid;
    place-items: center;
    color: var(--sl-ink-2);
    background: var(--sl-surface-2);
}
.sl-row__body { flex: 1 1 auto; min-width: 0; }
.sl-row__name { font-weight: 600; font-size: var(--sl-fz-base); }
.sl-row__status {
    font-size: var(--sl-fz-tiny);
    color: var(--sl-ink-3);
    overflow: hidden;
    text-overflow: ellipsis;
    white-space: nowrap;
}
.sl-row--syncing .sl-row__status { color: var(--sl-warn); }
.sl-row__dev-tag {
    font-size: 9px;
    font-weight: 700;
    letter-spacing: 0.06em;
    color: var(--sl-warn);
    border: 1px solid currentColor;
    border-radius: 4px;
    padding: 0 3px;
}

/* ---- toast ----- */
.sl-toast-stack {
    position: fixed;
    right: var(--sl-s3);
    bottom: var(--sl-s3);
    z-index: var(--sl-z-toast);
    display: flex;
    flex-direction: column;
    gap: var(--sl-s2);
    max-width: min(340px, calc(100vw - 24px));
}
.sl-toast {
    background: var(--sl-surface);
    border: 1px solid var(--sl-line);
    border-left: 3px solid var(--sl-ink-3);
    border-radius: var(--sl-r-md);
    box-shadow: var(--sl-shadow);
    padding: var(--sl-s2) var(--sl-s3);
    font-size: var(--sl-fz-small);
}
.sl-toast--ok { border-left-color: var(--sl-ok); }
.sl-toast--warn { border-left-color: var(--sl-warn); }
.sl-toast--danger { border-left-color: var(--sl-danger); }

```

```

.sl-toast__title { font-weight: 700; margin: 0 0 2px 0; }
.sl-toast__body { color: var(--sl-ink-2); margin: 0 0 6px 0; }
.sl-toast__actions { display: flex; gap: var(--sl-s2); flex-wrap: wrap; }

/* ---- banner (cookie tamer parked bar) ----- */
.sl-banner {
  position: fixed;
  left: 0; right: 0; bottom: 0;
  z-index: var(--sl-z-banner);
  background: var(--sl-surface);
  border-top: 1px solid var(--sl-line);
  box-shadow: 0 -4px 16px rgba(15, 23, 42, 0.12);
  padding: var(--sl-s2) var(--sl-s4);
  display: flex;
  align-items: center;
  gap: var(--sl-s3);
  font-size: var(--sl-fz-small);
}
.sl-banner__text { flex: 1 1 auto; }

/* ---- intercept layer (Form Guardian, full attention) ----- */
.sl-intercept {
  position: fixed;
  inset: 0;
  z-index: var(--sl-z-modal);
  background: rgba(15, 23, 42, 0.55);
  display: grid;
  place-items: center;
}
.sl-intercept__card {
  background: var(--sl-surface);
  border: 1px solid var(--sl-line);
  border-top: 4px solid var(--sl-danger);
  border-radius: var(--sl-r-lg);
  box-shadow: var(--sl-shadow);
  width: min(440px, calc(100vw - 32px));
  padding: var(--sl-s4);
}
.sl-intercept__title {
  font-size: var(--sl-fz-title);
  font-weight: 700;
  color: var(--sl-danger);
  margin: 0 0 var(--sl-s2) 0;
}
.sl-intercept__body { font-size: var(--sl-fz-base); color: var(--sl-ink-2); }
.sl-intercept__actions {
  display: flex;
  justify-content: flex-end;
  gap: var(--sl-s2);
  margin-top: var(--sl-s3);
}

/* ---- buttons ----- */
.sl-btn {
  display: inline-flex;
  align-items: center;
  gap: 6px;
  padding: 5px 12px;
  border-radius: var(--sl-r-md);
  border: 1px solid var(--sl-line);
  background: var(--sl-surface);
  color: var(--sl-ink);
  font-size: var(--sl-fz-small);
  font-weight: 600;
}
.sl-btn:hover { background: var(--sl-surface-2); }
.sl-btn--danger {
  background: var(--sl-danger);
  border-color: var(--sl-danger);
  color: #fff;
}
.sl-btn--primary {
  background: var(--sl-ink);
  border-color: var(--sl-ink);
}

```

```

    color: var(--sl-surface);
}

/* ---- indicator chip (camera/mic, demo greeting) ----- */
.sl-chip {
  position: fixed;
  top: var(--sl-s2);
  right: var(--sl-s2);
  z-index: var(--sl-z-chip);
  display: inline-flex;
  align-items: center;
  gap: 6px;
  background: var(--sl-surface);
  border: 1px solid var(--sl-line);
  border-radius: var(--sl-r-pill);
  box-shadow: var(--sl-shadow);
  padding: 4px 10px;
  font-size: var(--sl-fz-tiny);
  font-weight: 600;
}
.sl-chip__dot {
  width: 8px; height: 8px;
  border-radius: 50%;
  background: var(--sl-danger);
  animation: sl-pulse 1.6s ease-in-out infinite;
}
@keyframes sl-pulse {
  0%, 100% { opacity: 1; }
  50%      { opacity: 0.35; }
}
.sl-chip__dismiss {
  color: var(--sl-ink-3);
  padding: 0 2px;
  font-size: 12px;
}

/* ---- stat tile / empty state (dashboard) ----- */
.sl-stat-tile {
  background: var(--sl-surface);
  border: 1px solid var(--sl-line);
  border-radius: var(--sl-r-lg);
  padding: var(--sl-s3);
  text-align: center;
  flex: 1 1 120px;
}
.sl-stat-tile__num {
  font-size: 24px;
  font-weight: 700;
  color: var(--sl-ink);
}
.sl-stat-tile__label {
  font-size: var(--sl-fz-tiny);
  color: var(--sl-ink-3);
  margin-top: 2px;
}
.sl-empty {
  padding: var(--sl-s5);
  text-align: center;
  color: var(--sl-ink-3);
  font-size: var(--sl-fz-small);
}

/* ---- module-specific annotations (used by Parts 3/5/6) ----- */
.sl-outline-warn {
  outline: 2px dashed var(--sl-warn) !important;
  outline-offset: 2px;
}
.sl-ghost-flash {
  outline: 3px solid var(--sl-warn) !important;
  outline-offset: 2px;
  background: var(--sl-warn-bg) !important;
}

```

ui/toast.js

ui/toast.js

```
/* ui/toast.js - SLToast: stacked, dismissible, undo-friendly toasts.
 * Classic script; runs in content worlds AND extension pages. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.SLToast) return;

  function ensureStack() {
    let stack = document.querySelector('.sl-toast-stack');
    if (!stack) {
      stack = document.createElement('div');
      stack.className = 'sl-toast-stack sl-root';
      stack.setAttribute('role', 'status');
      stack.setAttribute('aria-live', 'polite');
      (document.body || document.documentElement).appendChild(stack);
    }
    return stack;
  }

  /**
   * Show a toast. Returns a handle with dismiss().
   * opts: {kind:'info'|"ok"|"warn"|"danger', title, body, timeoutMs,
   *        actions:[{label, onClick, primary}], onDismiss}
   * Undo pattern: pass actions:[{label:'Undo', onClick:fn}] - the
   * action runs, the toast closes, the user is never trapped.
   */
  function show(opts) {
    const o = opts || {};
    const stack = ensureStack();

    const toast = document.createElement('div');
    toast.className = 'sl-toast sl-toast--' + (o.kind || 'info');

    if (o.title) {
      const t = document.createElement('div');
      t.className = 'sl-toast__title';
      t.textContent = o.title;
      toast.appendChild(t);
    }
    if (o.body) {
      const b = document.createElement('div');
      b.className = 'sl-toast__body';
      b.textContent = o.body;
      toast.appendChild(b);
    }

    let dismissed = false;
    const actions = Array.isArray(o.actions) ? o.actions : [];
    if (actions.length) {
      const row = document.createElement('div');
      row.className = 'sl-toast__actions';
      for (const act of actions) {
        const btn = document.createElement('button');
        btn.className = 'sl-btn' + (act.primary ? ' sl-btn--primary' : '');
        btn.type = 'button';
        btn.textContent = act.label;
        btn.addEventListener('click', () => {
          try { act.onClick && act.onClick(); } catch (e) { /* undo errors
            must never break the toast lifecycle */ }
          dismiss();
        });
        row.appendChild(btn);
      }
      toast.appendChild(row);
    }

    function dismiss() {
      if (dismissed) return;
      dismissed = true;
      toast.remove();
    }
  }
}
```

```

    if (o.onDismiss) { try { o.onDismiss(); } catch (e) {} }
    document.removeEventListener('keydown', onKey);
  }

  function onKey(e) {
    if (e.key === 'Escape' && stack.contains(toast)) dismiss();
  }
  document.addEventListener('keydown', onKey);

  stack.appendChild(toast);
  const firstBtn = toast.querySelector('button');
  if (firstBtn && actions.length) firstBtn.focus();

  const ms = typeof o.timeoutMs === 'number' ? o.timeoutMs : 8000;
  if (ms > 0 && !actions.length) setTimeout(dismiss, ms);
  else if (ms > 0) setTimeout(dismiss, Math.max(ms, 20000));

  return { dismiss, el: toast };
}

root.SLToast = { show };
})(globalThis);

```

ui/verdict-card.js

```

ui/verdict-card.js
/* ui/verdict-card.js - SLVerdict: pills and detail cards.
 * pill() -> inline element you position yourself (hover previews)
 * card() -> fixed-position detail card with rows, reasons, actions */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.SLVerdict) return;

  const KIND = {
    CLEAR: ['sl-pill--clear', 'Nothing unusual'],
    SECOND_LOOK: ['sl-pill--warn', 'Second look'],
    INTERCEPTED: ['sl-pill--danger', 'Intercepted']
  };

  /**
   * Build a verdict pill element.
   * @param {string} verdict CLEAR | SECOND_LOOK | INTERCEPTED
   * @param {string} [text] override label (default per verdict)
   */
  function pill(verdict, text) {
    const conf = KIND[verdict] || KIND.CLEAR;
    const el = document.createElement('span');
    el.className = 'sl-pill ' + conf[0] + ' sl-root';
    el.textContent = text || conf[1];
    return el;
  }

  /**
   * Build + show a detail card.
   * @opts: {verdict, title, reasons:[string], rows:[{label,value}],
   *        actions:[{label,onClick,primary}], anchorEl?, onDismiss}
   * Positioning: below the anchor if given, else centered viewport.
   * Cards are singletons per page - showing one closes the previous.
   */
  let currentCard = null;

  function card(opts) {
    const o = opts || {};
    closeCard();

    const conf = KIND[o.verdict] || KIND.CLEAR;
    const wrap = document.createElement('div');
    wrap.className = 'sl-card sl-root';
    wrap.setAttribute('role', 'dialog');
    wrap.setAttribute('aria-label', o.title || 'SecondLook details');

    const title = document.createElement('div');

```

```

title.className = 'sl-card__title';
title.textContent = o.title || conf[1];
wrap.appendChild(title);

const close = document.createElement('button');
close.className = 'sl-card__close';
close.type = 'button';
close.setAttribute('aria-label', 'Close');
close.textContent = '\u00d7'; // multiplication sign, not emoji
close.addEventListener('click', () => closeCard());
wrap.appendChild(close);

const pillEl = pill(o.verdict);
wrap.appendChild(pillEl);
wrap.appendChild(document.createElement('br'));

for (const reason of (o.reasons || []).slice(0, 3)) {
  const r = document.createElement('div');
  r.className = 'sl-card__reason';
  r.textContent = reason;
  wrap.appendChild(r);
}
for (const row of (o.rows || [])) {
  const rowEl = document.createElement('div');
  rowEl.className = 'sl-card__row';
  const k = document.createElement('span');
  k.textContent = row.label;
  const v = document.createElement('b');
  v.textContent = row.value;
  rowEl.appendChild(k);
  rowEl.appendChild(v);
  wrap.appendChild(rowEl);
}
const actions = Array.isArray(o.actions) ? o.actions : [];
if (actions.length) {
  const row = document.createElement('div');
  row.className = 'sl-card__row';
  row.style.border = '0';
  row.style.justifyContent = 'flex-end';
  row.style.gap = '8px';
  for (const act of actions) {
    const btn = document.createElement('button');
    btn.className = 'sl-btn' + (act.primary ? ' sl-btn--primary' : '');
    btn.type = 'button';
    btn.textContent = act.label;
    btn.addEventListener('click', () => {
      try { act.onClick && act.onClick(); } catch (e) {}
      closeCard();
    });
    row.appendChild(btn);
  }
  wrap.appendChild(row);
}

(document.body || document.documentElement).appendChild(wrap);

// Position: prefer below-left of the anchor, clamp to viewport.
if (o.anchorEl && o.anchorEl.getBoundingClientRect()) {
  const r = o.anchorEl.getBoundingClientRect();
  wrap.style.left = Math.max(8, Math.min(r.left,
    window.innerWidth - 392)) + 'px';
  wrap.style.top = Math.min(r.bottom + 6,
    window.innerHeight - wrap.offsetHeight - 8) + 'px';
} else {
  wrap.style.left = '50%';
  wrap.style.top = '18%';
  wrap.style.transform = 'translateX(-50%)';
}

document.addEventListener('keydown', onDockKey, true);
currentCard = { el: wrap, onDismiss: o.onDismiss };
const firstBtn = wrap.querySelector('button');
if (firstBtn) firstBtn.focus();
return { dismiss: closeCard, el: wrap };

```

```

}

function onDocKey(e) {
  if (e.key === 'Escape') closeCard();
}

function closeCard() {
  if (!currentCard) return;
  const { el, onDismiss } = currentCard;
  currentCard = null;
  document.removeEventListener('keydown', onDocKey, true);
  el.remove();
  if (onDismiss) { try { onDismiss(); } catch (e) {} }
}

root.SLVerdict = { pill, card, closeCard };
})(globalThis);

```

0.10 The background core

Five files make the service worker. `service-worker.js` is a thin, synchronous wiring harness - MV3 kills the worker after ~30 seconds of silence, so every event listener is registered at top level, before any `await`. `bootstrap.js` owns script registration and the reconcile loop. `module-router.js` is the single message dispatcher and the only writer of settings (everything - popup, options, content scripts - asks the router, which is why the toggle lifecycle has exactly one code path). `context-menu.js` and `safe-mode.js` round out the privileged surface. Later parts append their background handlers to the router through `SL.Router.register()` rather than editing it.

background/service-worker.js

```

background/service-worker.js
/* background/service-worker.js - the single brain.
 * Classic worker; importScripts keeps shared files identical across
 * every runtime in the extension. All listeners register synchronously
 * (MV3 requirement: the worker dies often and must re-arm instantly). */
importScripts(
  '/shared/storage.js',
  '/shared/settings.js',
  '/shared/messaging.js',
  '/shared/module-registry.js',
  '/engine/known-brands.js',
  '/engine/domain-intel.js',
  '/engine/url-analyzer.js',
  '/engine/redirect-resolver.js',
  '/engine/verdict-engine.js',
  '/background/bootstrap.js',
  '/background/module-router.js',
  '/background/context-menu.js',
  '/background/safe-mode.js'
);

/* Part 7/8 append their files to the list above when those parts land. */

chrome.runtime.onInstalled.addListener(async (details) => {
  await SL.Bootstrap.reconcile('install:' + details.reason);
  SL.CtxMenu.init();
  await SL.Storage.ensureSalt();
  if (details.reason === 'install') {
    await SL.Storage.bumpStat('core', null);
  }
});

chrome.runtime.onStartup.addListener(async () => {
  await SL.Bootstrap.reconcile('startup');
});

/* The single reconciliation point: the SW is the only settings writer,
 * so every toggle flows through here exactly once. */
SL.Settings.subscribe(async (next, prev) => {

```

```

const ids = SL.Registry.MODULES.map((m) => m.id);
for (const id of ids) {
  const was = prev && SL.Settings.isModuleOn(prev, id, null);
  const now = SL.Settings.isModuleOn(next, id, null);
  if (was === now) continue;
  // Broadcast before reconcile: pages clean themselves up while the
  // registration state settles.
  await SL.Bootstrap.broadcast(id, now);
  if (now) await SL.Bootstrap.injectIntoOpenTabs(id);
}
const globalWas = prev ? prev.globalEnabled !== false : true;
const globalNow = next.globalEnabled !== false;
if (globalWas !== globalNow) {
  await SL.Bootstrap.broadcastAll({
    module: '*', type: 'globalToggled',
    payload: { enabled: globalNow }
  });
}
await SL.Bootstrap.reconcile('settings-changed');
});

chrome.tabs.onRemoved.addListener(async (tabId) => {
  await SL.Router.dropTab(tabId);
  await SL.SafeMode.clearFor(tabId);
});

/* Daily-ish prune of the local event log (never grows past EVENTS_MAX). */
chrome.alarms.create('sl:prune', { periodInMinutes: 720 });
chrome.alarms.onAlarm.addListener(async (alarm) => {
  if (alarm.name === 'sl:prune') await SL.Storage.pruneEvents();
});

SL.Router.initRouter();

```

background/bootstrap.js

Bootstrap owns the physical layer of "OFF is real OFF". `reconcile()` diffs the desired script set (modules enabled at global scope with non-null specs) against `chrome.scripting.getRegisteredContentScripts()`, then unregisters and registers accordingly. Registrations persist across browser restarts by design, so the diff also self-heals drift. `injectIntoOpenTabs()` handles the ON direction for pages that already finished loading - registered content scripts only fire on future navigations, so on-demand `executeScript` plus `insertCSS` is what makes a toggle flip feel instant everywhere.

background/bootstrap.js

```

/* background/bootstrap.js - script registration + open-tab injection. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.Bootstrap) return;

  const HTTP = ['http://*', 'https://*'];

  function log(...args) {
    console.log('[sl:bootstrap]', ...args); // visible in the SW console
  }

  function toChromeSpec(spec) {
    return {
      id: spec.id,
      matches: spec.matches || HTTP,
      js: spec.js,
      css: spec.css || [],
      runAt: spec.runAt || 'document_idle',
      world: spec.world || 'ISOLATED',
      allFrames: !!spec.allFrames,
      persistAcrossSessions: true
    };
  };

  /** Diff desired vs registered content scripts; apply the delta. */
  async function reconcile(reason) {

```

```

try {
  const settings = await SL.Settings.get();
  const desired = [];
  for (const mod of SL.Registry.MODULES) {
    if (!mod.scripts) continue;
    const on = SL.Settings.isModuleOn(settings, mod.id, null);
    if (on) desired.push(...mod.scripts);
  }
  const desiredIds = new Set(desired.map((s) => s.id));

  const current = await chrome.scripting.getRegisteredContentScripts();
  const currentIds = new Set(current.map((s) => s.id));

  const toRemove = [...currentIds].filter((id) => !desiredIds.has(id));
  const toAdd = desired.filter((s) => !currentIds.has(s.id));

  if (toRemove.length) {
    await chrome.scripting.unregisterContentScripts({ ids: toRemove });
    log(reason, 'unregistered', toRemove.join(', '));
  }
  if (toAdd.length) {
    try {
      await chrome.scripting.registerContentScripts(toAdd.map(toChromeSpec));
      log(reason, 'registered', toAdd.map((s) => s.id).join(', '));
    } catch (err) {
      if (!/duplicate/i.test(String(err))) throw err;
      // duplicate id = already registered; benign after restarts
      log(reason, 'duplicate registration ignored');
    }
  }
  if (!toRemove.length && !toAdd.length) {
    log(reason, 'in sync (' , desiredIds.size, 'scripts )');
  }
} catch (err) {
  console.error('[sl:bootstrap] reconcile failed:', err);
}
}

/** Flip ON for pages that already loaded: registered scripts will
 * not fire for them, so inject the module's files right now. */
async function injectIntoOpenTabs(moduleId) {
  const mod = SL.Registry.byId(moduleId);
  if (!mod || !mod.scripts) return;
  let tabs = [];
  try { tabs = await chrome.tabs.query({ url: HTTP }); } catch (e) { return; }
  for (const tab of tabs) {
    for (const spec of mod.scripts) {
      const target = { tabId: tab.id, allFrames: !!spec.allFrames };
      try {
        await chrome.scripting.executeScript({
          target, files: spec.js, world: spec.world || 'ISOLATED'
        });
        if (spec.css && spec.css.length) {
          await chrome.scripting.insertCSS({ target, files: spec.css });
        }
      } catch (err) {
        // tab may have navigated or be inaccessible; not an error
      }
    }
  }
}

/** Broadcast module state to every open http/https tab. */
async function broadcast(moduleId, enabled) {
  let tabs = [];
  try { tabs = await chrome.tabs.query({ url: HTTP }); } catch (e) { return; }
  const msg = {
    module: '*', type: 'moduleToggled',
    payload: { moduleId, enabled: !!enabled }
  };
  for (const tab of tabs) {
    try { await chrome.tabs.sendMessage(tab.id, msg); } catch (e) {
      // no listener in that tab (module never loaded there) - fine
    }
  }
}

```

```

    }
  }

  async function broadcastAll(msg) {
    let tabs = [];
    try { tabs = await chrome.tabs.query({ url: HTTP }); } catch (e) { return; }
    for (const tab of tabs) {
      try { await chrome.tabs.sendMessage(tab.id, msg); } catch (e) {}
    }
  }

  root.Bootstrap = { reconcile, injectIntoOpenTabs, broadcast, broadcastAll };
})(globalThis);

```

background/module-router.js

The router is one `runtime.onMessage` listener and a handler table keyed `module -> type -> async fn`. It normalizes every response to `{ok, data} / {ok, error}`, which is what `SL.msg.send` on the other end expects. The `status` route is worth reading closely: content scripts report one-line summaries that the router files into session storage keyed by tab id, and `dropTab()` (wired to `tabs.onRemoved` in the service worker) prunes them - that is the entire plumbing behind the popup's live status lines.

```

background/module-router.js
/* background/module-router.js - the single message dispatcher.
 * Contract: {module, type, payload} -> {ok:true, data} | {ok:false, error}.
 * Later parts call SL.Router.register(moduleId, handlers) to add routes
 * without editing this file. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.Router) return;

  const HANDLERS = {};

  function register(moduleId, handlers) {
    HANDLERS[moduleId] = Object.assign(HANDLERS[moduleId] || {}, handlers);
  }

  /* ----- engine routes ----- */
  register('engine', {
    async analyze(payload) {
      const v = await SL.Engine.analyze(payload.url, payload.context || {});
      await SL.Storage.bumpStat('engine', v.verdict);
      if (v.verdict !== 'CLEAR') {
        await SL.Storage.pushEvent({
          module: 'engine', verdict: v.verdict,
          url: payload.url, note: v.reasons[0] || ''
        });
      }
      return v;
    },
    async resolveRedirect(payload) {
      return SL.Engine.Redirect.resolve(payload.url);
    }
  });

  /* ----- generic status ----- */
  register('core', {
    async 'status-sink'(payload, sender) { return null; }
  });

  /* ----- core routes ----- */
  register('core', {
    async getSnapshot(payload) {
      const tabId = payload && payload.tabId;
      const tab = await chrome.tabs.get(tabId);
      let host = '';
      try { host = new URL(tab.url || '').hostname; } catch (e) {}
      const [settings, statusMap, safeTabs, stats] = await Promise.all([
        SL.Settings.effective(host),

```

```

    SL.Storage.sessionGet('sl.status', {}),
    SL.Storage.sessionGet('sl.safemode', []),
    SL.Storage.getStats()
  ]);
  const pageVerdict = tab.url && /^https?:/i.test(tab.url)
    ? await SL.Engine.analyze(tab.url, {})
    : null;
  return {
    settings, host, pageVerdict, stats,
    statusLines: statusMap[tabId] || {},
    safeModeOn: safeTabs.includes(tabId),
    url: tab.url || ''
  };
},
async setModuleEnabled(payload) {
  return SL.Settings.setModuleEnabled(payload.moduleId, payload.enabled);
},
async setModuleOption(payload) {
  return SL.Settings.setModuleOption(
    payload.moduleId, payload.option, payload.value);
},
async setGlobalEnabled(payload) {
  return SL.Settings.setGlobalEnabled(payload.enabled);
},
async setSiteOverride(payload) {
  return SL.Settings.setSiteOverride(
    payload.host, payload.moduleId, payload.enabled);
},
async clearSiteOverride(payload) {
  return payload.moduleId
    ? SL.Settings.clearSiteOverride(payload.host, payload.moduleId)
    : SL.Settings.clearAllSiteOverrides(payload.host);
},
async safeMode(payload) {
  await SL.SafeMode.set(payload.tabId, payload.on);
  return { on: payload.on };
},
async getEvents(payload) {
  return SL.Storage.getEvents((payload && payload.limit) || 50);
},
async getStats() { return SL.Storage.getStats(); },
async resetData() { return SL.Storage.resetData(); },
async exportSettings() {
  const s = await SL.Settings.get();
  return JSON.stringify(s, null, 2);
},
async importSettings(payload) {
  const parsed = JSON.parse(payload.json);
  return SL.Settings.save(parsed);
}
});

/* Status lines from content scripts: file under the sender's tab. */
async function handleStatus(payload, sender) {
  const tabId = sender && sender.tab && sender.tab.id;
  if (typeof tabId !== 'number') return null;
  const map = await SL.Storage.sessionGet('sl.status', {});
  map[tabId] = Object.assign({}, map[tabId]);
  map[tabId][payload.moduleId] = payload.line;
  await SL.Storage.sessionSet('sl.status', map);
  return null;
}

async function dropTab(tabId) {
  const map = await SL.Storage.sessionGet('sl.status', {});
  if (map[tabId]) {
    delete map[tabId];
    await SL.Storage.sessionSet('sl.status', map);
  }
}

function initRouter() {
  chrome.runtime.onMessage.addListener((msg, sender, sendResponse) => {
    if (!msg || typeof msg !== 'object') return false;

```

```

const moduleTable = HANDLERS[msg.module];
(async () => {
  // Every module's 'status' route lands here, generically.
  if (msg.type === 'status') return handleStatus(msg.payload, sender);
  const fn = moduleTable && moduleTable[msg.type];
  if (typeof fn !== 'function') {
    return { ok: false, error: 'unknown-route:' +
      msg.module + '/' + msg.type };
  }
  return { ok: true, data: await fn(msg.payload || {}, sender) };
})();
.then((res) => sendResponse(res || { ok: true, data: null })))
.catch((err) => sendResponse({
  ok: false, error: String((err && err.message) || err)
})));
return true; // async response
});
}

root.Router = { register, initRouter, dropTab, HANDLERS };
})(globalThis);

```

background/context-menu.js

Two right-click entries ship with the core. "Where does this link really go?" resolves the link's redirect chain in the background and hands the result to the page (Redirect Detective's content script renders it as a card; until Part 2 lands the message simply has no listener, which the router tolerates). "Safe Mode for this tab" freezes that tab's network with a DNR session rule. Menu ids are constants so re-install never duplicates them.

```

background/context-menu.js
/* background/context-menu.js - right-click entries. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.CtxMenu) return;

  const ID_TRACE = 'sl-where-does-this-go';
  const ID_SAFE = 'sl-safe-mode';

  function init() {
    chrome.contextMenus.removeAll(() => {
      chrome.contextMenus.create({
        id: ID_TRACE,
        title: 'Where does this link really go?',
        contexts: ['link']
      });
      chrome.contextMenus.create({
        id: ID_SAFE,
        title: 'SecondLook: Safe Mode for this tab',
        contexts: ['page']
      });
    });
  }

  chrome.contextMenus.onClicked.addListener(async (info, tab) => {
    if (!tab || typeof tab.id !== 'number') return;
    if (info.menuItemId === ID_TRACE && info.linkUrl) {
      const result = await SL.Engine.Redirect.resolve(info.linkUrl);
      try {
        await chrome.tabs.sendMessage(tab.id, {
          module: 'redirectDetective',
          type: 'showChain',
          payload: { startUrl: info.linkUrl, chain: result.chain,
            hops: result.hops, error: result.error }
        });
      } catch (e) {
        // Redirect Detective content script not present (module off or
        // Part 2 not built yet). Nothing to do - resolution still ran.
      }
    } else if (info.menuItemId === ID_SAFE) {
      const safeTabs = await SL.Storage.sessionGet('sl.safemode', []);
      const on = !safeTabs.includes(tab.id);
    }
  });
}

```

```

    await SL.SafeMode.set(tab.id, on);
  }
});

root.CtxMenu = { init, ID_TRACE, ID_SAFE };
})(globalThis);

```

background/safe-mode.js

Safe Mode is the only blocking feature in the entire extension, and it is scoped to a single tab by design. A DNR **session** rule (never static, never persisted) blocks every resource type except `main_frame`, so the already-rendered page stays visible but cannot fetch, ping, beacon or reconnect - the right shape for "I think this page is hostile and I want it inert while I decide." The rule id is derived from the tab id, which makes cleanup on toggle-off or tab-close a one-liner.

```

background/safe-mode.js
/* background/safe-mode.js - per-tab network freeze via DNR session rules.
 * This is the ONLY blocking code path in the whole extension. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.SafeMode) return;

  const RULE_BASE = 90000; // rule id = RULE_BASE + tabId (unique per tab)
  const RESOURCE_TYPES = [ // everything except main_frame: the page
    'sub_frame', 'stylesheet', 'script', 'image', 'font', 'object',
    'xmlhttprequest', 'ping', 'media', 'websocket', 'webtransport',
    'webbundle', 'csp_report', 'other'
  ];

  async function set(tabId, on) {
    const list = await SL.Storage.sessionGet('sl.safemode', []);
    const idx = list.indexOf(tabId);
    if (on) {
      if (idx === -1) { list.push(tabId);
        await SL.Storage.sessionSet('sl.safemode', list); }
      await chrome.declarativeNetRequest.updateSessionRules({
        addRules: [{
          id: RULE_BASE + tabId,
          priority: 1,
          condition: { tabIds: [tabId], resourceTypes: RESOURCE_TYPES },
          action: { type: 'BLOCK' }
        }]
      });
    } else {
      if (idx !== -1) { list.splice(idx, 1);
        await SL.Storage.sessionSet('sl.safemode', list); }
      await chrome.declarativeNetRequest.updateSessionRules({
        removeRuleIds: [RULE_BASE + tabId]
      });
    }
    // Toolbar state for this tab only.
    await chrome.action.setBadgeText({ tabId, text: on ? 'SAFE' : '' });
    await chrome.action.setBadgeBackgroundColor({ tabId, color: '#b45309' });
    try {
      await chrome.tabs.sendMessage(tabId,
        { module: '*', type: 'safeModeChanged', payload: { on } });
    } catch (e) { /* no listener in tab - fine */ }
  }

  async function isOn(tabId) {
    const list = await SL.Storage.sessionGet('sl.safemode', []);
    return list.includes(tabId);
  }

  async function clearFor(tabId) {
    try { await set(tabId, false); } catch (e) { /* tab already gone */ }
  }

  root.SafeMode = { set, isOn, clearFor, RULE_BASE, RESOURCE_TYPES };
})(globalThis);

```

TIP - When Safe Mode misbehaves, the two fastest debug windows are the service worker console (chrome://extensions -> SecondLook -> "service worker") for rule errors, and the network tab of the frozen page, which should show every request as (blocked:inspector_extension)-style entries the moment the session rule exists.

0.11 The popup - all ten rows, live

The popup is the product's face: 360px wide, four collapsible pillar groups, ten module rows with working toggles, a per-site verdict card for the current tab, a Safe Mode button, and a global pause switch. It renders from the registry (single source of truth) and subscribes to two storage streams: settings changes flip the toggles, and session `sl.status` updates rewrite the one-line status under each module for `*this*` tab. Every action travels through the router, never directly to storage - the single-writer rule is what keeps the toggle lifecycle deterministic.

ui/popup.html

ui/popup.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>SecondLook</title>
  <link rel="stylesheet" href="/ui/tokens.css">
  <link rel="stylesheet" href="/ui/components.css">
  <link rel="stylesheet" href="/ui/popup.css">
</head>
<body class="sl-page sl-root">
  <main id="slp">
    <header class="slp-header">
      <div class="slp-brand">
        <svg viewBox="0 0 24 24" width="18" height="18" aria-hidden="true">
          <path d="M2 12s3.5-6 10-6 10 6 10 6-3.5 6-10 6s-6-3.5-6-10 6-10 6-3.5"
              fill="none" stroke="currentColor" stroke-width="1.6"/>
          <circle cx="12" cy="12" r="3" fill="currentColor"/>
        </svg>
        <span class="slp-brand__name">SecondLook</span>
      </div>
      <label class="sl-toggle" title="Pause all modules">
        <input id="slp-global" type="checkbox" aria-label="Pause all modules">
        <span class="sl-toggle__track"></span>
      </label>
    </header>

    <div id="slp-paused" class="slp-paused" hidden>
      Paused - everything stays off until you flip this back.
    </div>

    <section id="slp-site" class="slp-site" hidden>
      <div class="slp-site__row">
        <div class="slp-site__host" id="slp-host"></div>
        <span id="slp-verdict"></span>
      </div>
      <button id="slp-safemode" class="sl-btn" type="button">
        Safe Mode this tab
      </button>
    </section>
    <div id="slp-empty" class="sl-empty" hidden>
      Nothing to look at on this page type.
    </div>

    <nav id="slp-pillars" aria-label="Modules"></nav>

    <footer class="slp-footer">
      <button id="slp-dashboard" class="sl-btn" type="button" disabled
        title="Arrives in Part 11">Dashboard</button>
      <button id="slp-options" class="sl-btn" type="button">Settings</button>
      <div class="slp-footer__privacy">Local-only. No telemetry.</div>
    </footer>
```

```

</main>

<script src="/shared/storage.js"></script>
<script src="/shared/settings.js"></script>
<script src="/shared/messaging.js"></script>
<script src="/shared/module-registry.js"></script>
<script src="/ui/toast.js"></script>
<script src="/ui/verdict-card.js"></script>
<script src="/ui/popup.js"></script>
</body>
</html>

```

ui/popup.css

ui/popup.css

```

/* ui/popup.css - popup layout only. Components come from components.css. */
html, body { margin: 0; padding: 0; }
body { width: 340px; }

#slp { display: flex; flex-direction: column; max-height: 580px; }
#slp-pillars { overflow-y: auto; flex: 1 1 auto; }

.slp-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: var(--sl-s3);
  border-bottom: 1px solid var(--sl-line);
}
.slp-brand {
  display: flex;
  align-items: center;
  gap: 8px;
  color: var(--sl-ink);
}
.slp-brand__name { font-weight: 700; font-size: var(--sl-fz-title); }

.slp-paused {
  background: var(--sl-warn-bg);
  color: var(--sl-warn);
  font-size: var(--sl-fz-small);
  padding: var(--sl-s2) var(--sl-s3);
}

.slp-site { padding: var(--sl-s2) var(--sl-s3); border-bottom: 1px solid var(--sl-line); }
.slp-site__row {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 8px;
  margin-bottom: var(--sl-s2);
}
.slp-site__host {
  font-size: var(--sl-fz-small);
  font-weight: 600;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}

.slp-pillar { border-bottom: 1px solid var(--sl-line); }
.slp-pillar__head {
  display: flex;
  width: 100%;
  align-items: center;
  justify-content: space-between;
  padding: var(--sl-s2) var(--sl-s3);
  background: none;
  border: 0;
  font-size: var(--sl-fz-tiny);
  font-weight: 700;
  letter-spacing: 0.05em;
  text-transform: uppercase;
}

```

```

    color: var(--sl-ink-3);
    cursor: pointer;
}
.slp-pillar__chev { transition: transform var(--sl-t-fast); }
.slp-pillar[data-collapsed="true"] .slp-pillar__chev {
  transform: rotate(-90deg);
}
.slp-pillar__rows { padding: var(--sl-s1) 0; }
.slp-pillar[data-collapsed="true"] .slp-pillar__rows { display: none; }

.slp-footer {
  display: flex;
  align-items: center;
  gap: var(--sl-s2);
  padding: var(--sl-s2) var(--sl-s3);
  border-top: 1px solid var(--sl-line);
}
.slp-footer__privacy {
  margin-left: auto;
  font-size: var(--sl-fz-tiny);
  color: var(--sl-ink-3);
}

```

ui/popup.js

ui/popup.js

```

/* ui/popup.js - renders pillars/rows from the registry; every mutation
 * goes through the router so the SW stays the single settings writer. */
(function () {
  'use strict';
  const send = SL.msg.send;
  const $ = (id) => document.getElementById(id);

  const state = { tabId: null, snapshot: null, settings: null };

  /* one 16x16 stroke icon per module - SVG only, never emoji */
  const ICONS = {
    demoGreeting: eyeIcon(), linkSniper: eyeIcon(),
    redirectDetective: chainIcon(), trustBadge: sealIcon(),
    formGuardian: formIcon(), ghostClick: ghostIcon(),
    fakeDownload: dlIcon(), downloadGuard: shieldIcon(),
    tabGuard: tabsIcon(), cookieTamer: cookieIcon(), avIndicator: camIcon()
  };

  function svgWrap(inner) {
    return '<svg viewBox="0 0 24 24" width="16" height="16" ' +
      'aria-hidden="true">' + inner + '</svg>';
  }

  function eyeIcon() {
    return svgWrap('<path d="M2 12s3.5-6 10-6 10 6 6-3.5 6-10 6s2 2 12z"' +
      ' fill="none" stroke="currentColor" stroke-width="1.8"/>' +
      '<circle cx="12" cy="12" r="2.6" fill="currentColor"/>');
  }

  function chainIcon() {
    return svgWrap('<path d="M9 15l6-6M8 12l-2 2a3 3 0 04 4l2-2' +
      'M16 12l2-2a3 3 0 00-4-4l-2 2" fill="none" stroke="currentColor" ' +
      ' stroke-width="1.8"/>');
  }

  function sealIcon() {
    return svgWrap('<path d="M12 3l2.5 2 3 .5.5 3L20 12l-2 2.5-.5 3-3 .5' +
      '-2.5 2-2.5-2-3-.5-.5 3L4 12l2-2.5.5-3 3-.5z" fill="none" stroke=' +
      '"currentColor" stroke-width="1.6"/><path d="M9 12l2 4-4" fill=' +
      '"none" stroke="currentColor" stroke-width="1.8"/>');
  }

  function formIcon() {
    return svgWrap('<rect x="4" y="6" width="16" height="12" rx="2" fill=' +
      '"none" stroke="currentColor" stroke-width="1.8"/><path d="M7 10h10' +
      'M7 14h6" stroke="currentColor" stroke-width="1.8"/>');
  }

  function ghostIcon() {
    return svgWrap('<rect x="4" y="4" width="16" height="16" rx="3" fill=' +
      '"none" stroke="currentColor" stroke-width="1.8" stroke-dasharray=' +
      '"3 2"/>');
  }

```

```

}
function dlIcon() {
  return svgWrap('<path d="M12 4v10m0 0l-4-4m4 4l4-4M5 19h14" fill=' +
    '"none" stroke="currentColor" stroke-width="1.8"/>');
}
function shieldIcon() {
  return svgWrap('<path d="M12 3l7 3v6c0 4-3 7-7 9-4-2-7-5-7-9V6z" fill=' +
    '"none" stroke="currentColor" stroke-width="1.8"/><path d="M12 9v4" ' +
    'stroke="currentColor" stroke-width="1.8"/>');
}
function tabsIcon() {
  return svgWrap('<rect x="3" y="7" width="18" height="13" rx="2" fill=' +
    '"none" stroke="currentColor" stroke-width="1.8"/><path d="M3 7l3-4h8' +
    'l3 4" fill="none" stroke="currentColor" stroke-width="1.8"/>');
}
function cookieIcon() {
  return svgWrap('<circle cx="12" cy="12" r="8" fill="none" stroke=' +
    '"currentColor" stroke-width="1.8"/><circle cx="10" cy="10" r="1.2" ' +
    'fill="currentColor"/><circle cx="14" cy="13" r="1.2" fill=' +
    '"currentColor"/><circle cx="9" cy="15" r="1.2" fill="currentColor"/>');
}
function camIcon() {
  return svgWrap('<rect x="3" y="7" width="12" height="10" rx="2" fill=' +
    '"none" stroke="currentColor" stroke-width="1.8"/><path d="M15 11l6-' +
    '3v8l-6-3z" fill="none" stroke="currentColor" stroke-width="1.8"/>');
}
}

async function init() {
  const [tab] = await chrome.tabs.query({ active: true,
    currentWindow: true });

  state.tabId = tab ? tab.id : null;
  const res = await send('core', 'getSnapshot', { tabId: state.tabId });
  state.snapshot = res.ok ? res.data : null;
  state.settings = state.snapshot && state.snapshot.settings;
  renderSite();
  renderPillars();
  renderGlobal();
  chrome.storage.onChanged.addListener(onStorage);
}

function onStorage(changes, area) {
  if (area === 'local' && changes['sl.settings']) {
    state.settings = changes['sl.settings'].newValue;
    renderPillars();
    renderGlobal();
  }
  if (area === 'session' && changes['sl.status']) {
    state.snapshot.statusLines =
      (changes['sl.status'].newValue || {})[state.tabId] || {};
    renderPillars();
  }
}

function renderGlobal() {
  const on = !(state.settings && state.settings.globalEnabled !== false);
  $('slp-global').checked = on;
  $('slp-paused').hidden = on;
}

function renderSite() {
  const snap = state.snapshot;
  const isHttp = snap && /^https?:/i.test(snap.url || '');
  $('slp-site').hidden = !isHttp;
  $('slp-empty').hidden = !isHttp;
  if (!isHttp) return;
  $('slp-host').textContent = snap.host || '(this page)';
  const holder = $('slp-verdict');
  holder.textContent = '';
  if (snap.pageVerdict) {
    const v = snap.pageVerdict;
    const pill = SLVerdict.pill(v.verdict,
      v.verdict === 'CLEAR' ? 'OK' : 'Take a look');
    if (v.reasons && v.reasons.length) {
      pill.title = v.reasons.join(' ');
    }
  }
}

```

```

    }
    holder.appendChild(pill);
  }
  updateSafeModeButton();
}

function updateSafeModeButton() {
  const btn = $('slp-safemode');
  const on = !(state.snapshot && state.snapshot.safeModeOn);
  btn.textContent = on ? 'Safe Mode is ON - unfreeze' : 'Safe Mode this tab';
  btn.classList.toggle('sl-btn--primary', on);
}

function renderPillars() {
  const nav = $('slp-pillars');
  nav.textContent = '';
  const collapsed = {};
  SL.Registry.PILLARS.forEach((pillar, idx) => {
    const section = document.createElement('section');
    section.className = 'sl-pillar';
    section.dataset.collapsed = idx > 0 ? 'true' : 'false';

    const head = document.createElement('button');
    head.className = 'slp-pillar__head';
    head.type = 'button';
    head.setAttribute('aria-expanded', idx > 0 ? 'false' : 'true');
    head.innerHTML = '<span>' + pillar.name + '</span>' +
      '<svg class="slp-pillar__chev" viewBox="0 0 24 24" width="12" ' +
      'height="12" aria-hidden="true"><path d="M6 9l6 6 6-6" fill="none" ' +
      'stroke="currentColor" stroke-width="2"/></svg>';
    head.addEventListener('click', () => {
      const now = section.dataset.collapsed !== 'true';
      section.dataset.collapsed = String(now);
      head.setAttribute('aria-expanded', String(!now));
      collapsed[pillar.id] = now;
    });
    section.appendChild(head);

    const rows = document.createElement('div');
    rows.className = 'slp-pillar__rows';
    for (const mod of SL.Registry.MODULES) {
      if (mod.pillar !== pillar.id) continue;
      rows.appendChild(makeRow(mod));
    }
    section.appendChild(rows);
    nav.appendChild(section);
  });
}

function makeRow(mod) {
  const row = document.createElement('div');
  row.className = 'sl-row';
  row.dataset.module = mod.id;

  const icon = document.createElement('div');
  icon.className = 'sl-row__icon';
  icon.innerHTML = ICONS[mod.id] || eyeIcon();
  row.appendChild(icon);

  const body = document.createElement('div');
  body.className = 'sl-row__body';
  const name = document.createElement('div');
  name.className = 'sl-row__name';
  name.textContent = mod.name;
  if (mod.devOnly) {
    const tag = document.createElement('span');
    tag.className = 'sl-row__dev-tag';
    tag.textContent = 'DEV';
    tag.style.marginLeft = '6px';
    name.appendChild(tag);
  }
  const status = document.createElement('div');
  status.className = 'sl-row__status';
  status.textContent = statusText(mod.id);
}

```

```

body.appendChild(name);
body.appendChild(status);
row.appendChild(body);

const toggle = document.createElement('label');
toggle.className = 'sl-toggle';
const input = document.createElement('input');
input.type = 'checkbox';
input.setAttribute('aria-label', mod.name + ' on this browser');
input.checked = isOn(mod.id);
input.addEventListener('change', () => onToggle(mod.id, input.checked,
row));

const track = document.createElement('span');
track.className = 'sl-toggle__track';
toggle.appendChild(input);
toggle.appendChild(track);
row.appendChild(toggle);

row.title = mod.blurb;
return row;
}

function isOn(moduleId) {
  return !(state.settings &&
    SL.Settings.isModuleOn(state.settings, moduleId, null));
}

function statusText(moduleId) {
  const lines = state.snapshot && state.snapshot.statusLines;
  const line = lines && lines[moduleId];
  if (line) return line;
  return isOn(moduleId) ? 'Idle' : 'Off';
}

async function onToggle(moduleId, enabled, rowEl) {
  rowEl.classList.add('sl-row--syncing');
  const statusEl = rowEl.querySelector('.sl-row__status');
  if (statusEl) statusEl.textContent = 'syncing';
  await send('core', 'setModuleEnabled', { moduleId, enabled });
  // settings change event clears the syncing state via renderPillars()
}

$('slp-global').addEventListener('change', async (e) => {
  await send('core', 'setGlobalEnabled', { enabled: e.target.checked });
});

$('slp-safemode').addEventListener('click', async () => {
  const on = !(state.snapshot && state.snapshot.safeModeOn);
  await send('core', 'safeMode', { tabId: state.tabId, on });
  state.snapshot.safeModeOn = on;
  renderSite();
});

$('slp-options').addEventListener('click', () => {
  chrome.runtime.openOptionsPage();
});

$('slp-dashboard').addEventListener('click', () => {
  chrome.tabs.create({ url: chrome.runtime.getURL('/ui/dashboard.html') });
});

init();
})();

```

0.12 The demo module - proof the toggles are real

Everything above is machinery; this is the proof. The demo module is fifteen lines of module code riding the exact same registration, settings, broadcast and cleanup pipeline as the ten real modules, and its chip is deliberately placed bottom-left so it never collides with the camera indicator from Part 10. When you flip its toggle in the popup, the chip vanishes from every open tab within a second - no reload, no exception. That behavior is the entire product promise in miniature, which is why Part 13 removes it rather than ships it.

```
modules/demo-greeting/demo-greeting.js
```

```
/* modules/demo-greeting/demo-greeting.js - dev-only toggle proof.
 * Rides the exact pipeline real modules use; removed in Part 13. */
(function () {
  'use strict';
  if (!globalThis.SL || !SL.__bridge) return;

  let chip = null;

  function mount() {
    if (chip) return;
    chip = document.createElement('div');
    chip.className = 'sl-chip sl-demo-chip';
    chip.textContent = 'SecondLook active on this page';
    chip.appendChild(dismissButton());
    (document.body || document.documentElement).appendChild(chip);
    SL.registerCleanup('demoGreeting', unmount);
    SL.status('demoGreeting', 'Demo chip visible - toggles are real');
  }

  function dismissButton() {
    const btn = document.createElement('button');
    btn.className = 'sl-chip__dismiss';
    btn.type = 'button';
    btn.setAttribute('aria-label', 'Dismiss demo chip');
    btn.textContent = '\u00d7';
    btn.addEventListener('click', (e) => {
      e.stopPropagation();
      unmount();
      SL.status('demoGreeting', 'Demo chip dismissed this tab');
    });
    return btn;
  }

  function unmount() {
    if (chip) { chip.remove(); chip = null; }
  }

  SL.boot('demoGreeting', {
    onEnabled() { mount(); },
    onDisabled() { unmount(); }
  });
})();
```

```
modules/demo-greeting/demo-greeting.css
```

```
/* modules/demo-greeting/demo-greeting.css */
.sl-demo-chip {
  left: var(--sl-s2);
  right: auto; /* bottom-LEFT: never collides with P10 chip */
  bottom: var(--sl-s2);
  top: auto;
  color: var(--sl-ink-2);
}
```

0.13 Icons

The toolbar icon is a rounded dark tile carrying a single amber ring with an offset pupil - an eye caught mid-second-glance. The generator below uses Pillow (pip install pillow) to emit the four base sizes plus the three state variants Chrome swaps at runtime. Run it once from the extension root; rerun it any time you tweak the constants.

```
tools/make-icons.py
```

```
#!/usr/bin/env python3
"""tools/make-icons.py - generate SecondLook toolbar icons.

Usage: python3 tools/make-icons.py
Writes: assets/icons/icon-{16,32,48,128}.png
        assets/icons/icon-alert-48.png icon-paused-48.png
        assets/icons/icon-camera-48.png
"""
```

```

import os
from PIL import Image, ImageDraw

OUT = os.path.join(os.path.dirname(__file__), "..", "assets", "icons")
SIZES = [16, 32, 48, 128]

NAVY = (15, 42, 67)      # tile
AMBER = (217, 119, 6)    # ring / accent
SLATE = (203, 210, 217)  # paused icon ring
RED = (185, 28, 28)      # camera-active dot

def rounded(draw, box, radius, fill):
    draw.rounded_rectangle(box, radius=radius, fill=fill)

def draw_icon(size, ring, pupil=NAVY, tile=NAVY, dot=None):
    img = Image.new("RGBA", (size, size), (0, 0, 0, 0))
    d = ImageDraw.Draw(img)
    radius = max(2, size // 5)
    rounded(d, [0, 0, size - 1, size - 1], radius, tile)
    cx, cy = size / 2, size / 2
    ring_r = size * 0.34
    lw = max(1, int(size * 0.075))
    d.ellipse([cx - ring_r, cy - ring_r, cx + ring_r, cy + ring_r],
              outline=ring, width=lw)
    pr = size * 0.14
    px = cx + size * 0.06      # pupil offset = the "second glance"
    d.ellipse([px - pr, cy - pr, px + pr, cy + pr], fill=pupil)
    if dot:
        dr = size * 0.10
        d.ellipse([size - dr * 2.2, dr * 1.1,
                  size - dr * 0.6, dr * 2.7], fill=dot)
    return img

def main():
    os.makedirs(OUT, exist_ok=True)
    for size in SIZES:
        draw_icon(size, AMBER).save(
            os.path.join(OUT, f"icon-{size}.png"))
        draw_icon(48, AMBER).save(os.path.join(OUT, "icon-alert-48.png"))
        draw_icon(48, SLATE).save(os.path.join(OUT, "icon-paused-48.png"))
        draw_icon(48, AMBER, dot=RED).save(
            os.path.join(OUT, "icon-camera-48.png"))
    print("icons written to", os.path.abspath(OUT))

if __name__ == "__main__":
    main()

```

0.14 Load & test Part 0

You now have every file needed for a loadable extension. The sequence below takes about five minutes and ends with the single most important demonstration in this guide - watching a toggle physically remove code from live pages.

- Create the folder tree from section 0.5 and paste every file from Parts 0.7-0.13 exactly as printed. The popup, options and dashboard HTML files for later parts do not exist yet and are not needed - the manifest only references `ui/popup.html`, which you have; `options_page` points at a file Part 11 creates, and until then Chrome shows the menu item but the page 404s. If that bothers you during development, comment out the `options_page` line until Part 11.
- Run `python3 tools/make-icons.py` from the `secondlook/` folder. You need Pillow: `pip install pillow`.
- Open `chrome://extensions`, enable **Developer mode** (top right), click **Load unpacked**, and select the `secondlook/` folder. Chrome should load it with zero errors; any red error text means a path typo - the error message names the exact file.
- Click the service worker link ("SecondLook - service worker") to open its DevTools console. You should see `[sl:bootstrap] install:installed registered sl-cs-demo`.

- Open any normal http(s) website. A small chip appears bottom-left: **SecondLook active on this page**.
- Open the popup (toolbar icon). You should see: the site card with the host and an OK pill, four pillar groups (only the first expanded), all ten module rows plus the DEV-tagged Demo Greeting, and the footer. Every row is live - flip any of them and the row pulses "syncing" then settles.
- **The money shot:** flip *Demo Greeting* off. The chip disappears from every tab you have open, instantly, with no reload. Flip it back on: the chip returns immediately, including on tabs that were already loaded. Watch the service worker console log `unregistered sl-cs-demo / registered sl-cs-demo` as you do it.
- Test the engine from the SW console:
`SL.Engine.analyze('http://paypal-secure-login.com/verify').then(console.log)` - expect verdict: 'SECOND_LOOK' with the typo-squat reason, and a score capped at 39. Then try
`SL.Engine.analyze('https://www.wikipedia.org/')` for a CLEAR.
- Right-click any page and choose **SecondLook: Safe Mode for this tab**. The toolbar badge shows SAFE and the page's DevTools network panel shows requests failing. Right-click again (or use the popup's Safe Mode button) to unfreeze.
- Finally, confirm the manifest contract: in `chrome://extensions` open the "Inspect views" for the extension, run `await chrome.scripting.getRegisteredContentScripts()` in the SW console, and verify that with Demo Greeting off the list is empty, and with it on the list contains exactly `sl-cs-demo`. That is the "OFF must be real OFF" guarantee, testable in one line.

Tuning knobs introduced in Part 0

Knob	File	Default	Effect
EVENTS_MAX	shared/storage.js	200	Local event-log cap (privacy + size)
VERDICT_CACHE_MAX	shared/cs-bridge.js	300	Verdict cache size per page
THRESHOLDS.SECOND_LOOK	engine/verdict-engine.js	20	Amber pill threshold
THRESHOLDS.INTERCEPT	engine/verdict-engine.js	40	Red threshold, module evidence only
THRESHOLDS.URL_CAP	engine/verdict-engine.js	39	Hard clamp for URL-only scores
SIGNAL_WEIGHTS	engine/url-analyzer.js	8-50	Per-signal weights
MAX_HOPS / TIMEOUT_MS	engine/redirect-resolver.js	8 / 6000	Redirect chain budget
MULTI_SUFFIXES	engine/domain-intel.js	~140	eTLD+1 accuracy
KNOWN_BRANDS	engine/known-brands.js	~100	Typo-squat coverage

DEVIATION NOTE - The manifest's `options_page` points at `ui/options.html`, which Part 11 creates. During Parts 0-10 either tolerate the 404 when clicking "Settings", or comment the line out and restore it in Part 11. This is the only forward reference in the whole guide, and it exists so the shipped manifest never needs editing after Part 0.

Part 1 - Link Sniper

1.1 Design analysis

Link Sniper answers one question at the exact moment it matters: **should I click this?** Hover (or keyboard-focus) any link for about a third of a second and a verdict pill appears beside the cursor - green "Nothing unusual" for the boring majority, amber with one plain sentence when something deserves attention. The module never navigates, never blocks, never modifies the link; it is a pure observe-and-annotate layer, which is why it is the right first module: it exercises the entire stack - registration, bridge, message bus, engine, UI kit - with zero interception risk.

The heuristic work is deliberately not in this file: every judgment comes from the engine's `analyze()`, with the module contributing exactly one piece of context the engine cannot know on its own - the link's **visible text**, which feeds the invisible-characters check (`displayText` vs `href`). Shortlinks get a second round: when the engine flags a URL as a known shortener, the module asks the service worker to resolve the chain, then re-analyzes with `resolvedUrl` and upgrades the pill with the real destination. That two-step flow is the template every other module copies when it needs "cheap first pass, expensive second pass."

Heuristic / behavior	Source	Verdict effect
Hover dwell $\geq$ <code>hoverDelayMs</code> (350 default)	module	pill appears; quick passes stay silent
display text vs href invisible chars	engine via <code>displayText</code>	+50 <code>INVISIBLE_CHARS</code>
punycode / IDN host	engine	+45
brand typo on eTLD+1 (Damerau $\leq$ 2)	engine	+40
shortener (bit.ly, t.co, ...)	engine then resolve	re-verdict on <code>resolvedUrl</code>
raw IP, <code>userinfo@</code> , nonstd port, http, TLD, subdomain depth, keywords	engine	+8..+30
CLEAR with <code>showClear</code> off	module	pill suppressed entirely

False-positive pattern	Risk	Mitigation
Legitimate internationalized domains (e.g. a German or Japanese site using IDN)	amber pill on a safe site	The plain sentence says characters <i>*can*</i> imitate sites, never that this one does; one click opens details; per-site off exists
Brand's own country domains (amazon.de, paypal.jp)	typo false match	eTLD+1 comparison plus official-domains list; exact match is exempt
Deep CDN subdomains (cdn.vendor.net.edgekey.net)	depth signal noise	Weight 12 alone stays below <code>SECOND_LOOK</code> threshold 20
Internal http tooling / localhost	http signal noise	Weight 8; and localhost rarely hovered with module focus
Shortener on a trusted service (lnkd.in from LinkedIn)	resolution latency only	Resolution runs in background; pill updates, never blocks

Chrome APIs used: none directly beyond what the bridge provides - `chrome.runtime.sendMessage` (via `SL.msg`), `chrome.storage` (via bridge settings cache), and the scripting registration from Part 0. This is a feature, not an omission: content modules that touch no privileged APIs are trivial to reason about and impossible to abuse. **Popup row:** "Link Sniper" with a live status line like "42 links scanned, 3 flagged". **Options entries** (wired in Part 11): hover delay slider (150-1000 ms), "show green pills" toggle, "resolve shortened links" toggle. **Toggle-off cleanup:** pill removed from the page, all listeners detached, counters dropped from the popup status line - and the script id `sl-cs-link-sniper` unregistered, so future pages never load it.

1.2 Files

```
modules/link-sniper/link-sniper.js
/* modules/link-sniper/link-sniper.js - hover verdict pills for links.
 * Observe-only: never blocks, never rewrites the link. */
(function () {
  'use strict';
  if (!globalThis.SL || !SL.__bridge) return;

  const PILL_OFFSET = 14;          // px from cursor
  const DISPLAY_TEXT_MAX = 60;     // chars of anchor text fed to engine
  const STATUS_EVERY = 8;         // scan count between status updates

  let cfg = { hoverDelayMs: 350, showClear: true };
  let hoverTimer = 0;
  let current = null;              // { link, url, pillEl }
  let scanned = 0;
  let flagged = 0;
  let scansSinceStatus = 0;

  function onConfig(next) { cfg = Object.assign(cfg, next || {}); }

  /* ---- pill rendering ----- */
  function showPill(link, url, verdict) {
    hidePill();
    if (verdict.verdict === 'CLEAR' && !cfg.showClear) return;
    const pill = SLVerdict.pill(verdict.verdict,
      verdict.verdict === 'CLEAR' ? 'Nothing unusual'
      : (verdict.reasons && verdict.reasons[0]) || 'Second look');
    pill.classList.add('sl-ls-pill');
    pill.addEventListener('click', () => openDetail(link, url, verdict));
    document.body.appendChild(pill);
    positionPill(pill, link);
    current = { link, url, pillEl: pill };
    if (verdict.verdict !== 'CLEAR') {
      flagged++;
      SL.status('linkSniper', scanned + ' links scanned, ' +
        flagged + ' flagged');
    }
  }

  function positionPill(pill, link) {
    const r = link.getBoundingClientRect();
    const x = Math.min(window.scrollX + r.right + PILL_OFFSET,
      window.scrollX + window.innerWidth - 240);
    const y = Math.min(window.scrollY + r.bottom + 4,
      window.scrollY + window.innerHeight - 34);
    pill.style.left = x + 'px';
    pill.style.top = y + 'px';
  }

  function hidePill() {
    if (current && current.pillEl) current.pillEl.remove();
    current = null;
  }

  function openDetail(link, url, verdict) {
    SLVerdict.card({
      verdict: verdict.verdict,
      title: 'Link details',
      reasons: verdict.reasons,
      rows: [
        { label: 'Text', value: textOf(link) || '(no text)' },
        { label: 'Goes to', value: shortHost(url) }
      ],
      anchorEl: link
    });
  }

  function textOf(link) {
    return (link.textContent || '').trim().slice(0, DISPLAY_TEXT_MAX);
  }

  function shortHost(url) {
```

```

    try { return new URL(url).hostname; } catch (e) { return url; }
}

/* ---- hover / focus flow ----- */
function inspect(link) {
    const url = link.getAttribute('href') || '';
    if (!/^https?:/i.test(url)) return; // ignore anchors, mailto, js:
    scanned++;
    scansSinceStatus++;
    if (scansSinceStatus >= STATUS_EVERY && !flagged) {
        scansSinceStatus = 0;
        SL.status('linkSniper', scanned + ' links scanned, 0 flagged');
    }
    const context = { displayText: textOf(link) };
    SL.verdict(url, context).then((verdict) => {
        if (current && current.link !== link && hoverTimer) return;
        showPill(link, url, verdict);
        // Shorteners get a second, resolved pass.
        if (verdict.meta && verdict.meta.isShortener) {
            SL.send('engine', 'resolveRedirect', { url }).then((res) => {
                if (!res.ok || !res.data || !res.data.finalUrl) return;
                SL.verdict(url, Object.assign({}, context, {
                    resolvedUrl: res.data.finalUrl
                })).then((v2) => showPill(link, url, v2));
            });
        }
    });
}

function onOver(e) {
    const link = e.target && e.target.closest &&
        e.target.closest('a[href]');
    if (!link || link === (current && current.link)) return;
    clearTimeout(hoverTimer);
    hoverTimer = setTimeout(() => inspect(link), cfg.hoverDelayMs);
}

function onOut(e) {
    const link = e.target && e.target.closest && e.target.closest('a[href]');
    if (link && current && current.link === link) hidePill();
    clearTimeout(hoverTimer);
    hoverTimer = 0;
}

function onFocusIn(e) {
    const link = e.target;
    if (link && link.tagName === 'A' && link.getAttribute('href')) {
        clearTimeout(hoverTimer);
        inspect(link); // keyboard users get the pill immediately
    }
}

function onKey(e) {
    if (e.key === 'Escape') hidePill();
}

function onScroll() { hidePill(); } // pill is cursor-anchored

/* ---- lifecycle ----- */
function attach() {
    document.addEventListener('mouseover', onOver, true);
    document.addEventListener('mouseout', onOut, true);
    document.addEventListener('focusin', onFocusIn, true);
    document.addEventListener('keydown', onKey, true);
    window.addEventListener('scroll', onScroll, true);
    SL.status('linkSniper', '0 links scanned, 0 flagged');
}

function detach() {
    document.removeEventListener('mouseover', onOver, true);
    document.removeEventListener('mouseout', onOut, true);
    document.removeEventListener('focusin', onFocusIn, true);
    document.removeEventListener('keydown', onKey, true);
    window.removeEventListener('scroll', onScroll, true);
}

```

```

clearTimeout(hoverTimer);
hidePill();
scanned = 0; flagged = 0;
}

SL.boot('linkSniper', {
  onEnabled(config) { cfg = Object.assign(cfg, config); attach(); },
  onConfig,
  onDisabled() { detach(); }
});
SL.registerCleanup('linkSniper', detach);
})();

```

```

modules/link-sniper/link-sniper.css
/* modules/link-sniper/link-sniper.css */
.sl-ls-pill {
  position: absolute;
  z-index: var(--sl-z-chip);
  cursor: pointer;
  max-width: 280px;
  overflow: hidden;
  text-overflow: ellipsis;
}

```

1.3 Registry entry for Part 1

Open `shared/module-registry.js` and replace the Link Sniper placeholder entry (the one whose `scripts` is null) with the entry below. From this moment the popup row controls a real script registration, and reloading the extension registers `sl-cs-link-sniper` at install time.

```

shared/module-registry.js - Link Sniper entry (replace placeholder)
{ id: 'linkSniper', name: 'Link Sniper', pillar: 'linkIntel',
  blurb: 'Hover any link to see where it really goes.',
  defaultEnabled: true,
  scripts: [{
    id: 'sl-cs-link-sniper', matches: ['http://*', 'https://*'],
    runAt: 'document_idle', world: 'ISOLATED', allFrames: false,
    js: ['shared/messaging.js', 'shared/cs-bridge.js',
      'ui/toast.js', 'ui/verdict-card.js',
      'modules/link-sniper/link-sniper.js'],
    css: ['ui/tokens.css', 'ui/components.css',
      'modules/link-sniper/link-sniper.css']
  } ] },

```

1.4 Load & test

- Reload the extension in `chrome://extensions`. The service worker console should log `install:... registered sl-cs-link-sniper, sl-cs-demo`.
- Open any content-rich site (a news site or Wikipedia works well) and hover links: green "Nothing unusual" pills should appear after ~350 ms and vanish on mouse-out.
- Focus a link with the keyboard (Tab) - the pill appears immediately without the dwell delay.
- Inject a suspicious link on any page via its DevTools console, then hover it: `var a=document.createElement('a');a.href='http://paypal-secure-login.com/verify';a.textContent='PayPal - verify account';document.body.prepend(a);` - expect an amber pill carrying the typo-squat sentence.
- Same trick with an invisible-character link: `var b=document.createElement('a');b.href='https://example.com';b.textContent='Safe link\u200b to somewhere else';document.body.prepend(b);` - the pill flags disguised text.
- Click an amber pill: the detail card opens with the reasons and the destination host; Escape closes it.

- Flip Link Sniper off in the popup while an amber pill is showing: the pill disappears immediately and hovering does nothing on any open tab. Flip it back on: hover works again with no reload.
- Verify the popup status line counts up as you hover around ("24 links scanned, 1 flagged").

1.5 Tuning knobs

Knob	Where	Default	Effect
hoverDelayMs	settings.modules.linkSniper	350	Dwell before analysis fires
showClear	settings.modules.linkSniper	true	Show green pills at all
PILL_OFFSET	link-sniper.js	14 px	Pill gap from the link box
DISPLAY_TEXT_MAX	link-sniper.js	60	Anchor text fed to the engine
STATUS_EVERY	link-sniper.js	8	Status-line update cadence
VERDICT_CACHE_MAX	shared/cs-bridge.js	300	Shared verdict cache size

Part 2 - Redirect Detective

2.1 Design analysis

A shortlink is a promise with the destination redacted. Redirect Detective makes the full journey visible: right-click any link and choose **Where does this link really go?**, and the service worker follows the chain (using the Part 0 resolver) while a card on the page lists every hop in order - host by host, with the ones that deserve attention marked. The card is the module's entire UI; there is also a hover integration point, but the context menu is the primary surface because trace-on-demand is an explicit, intentional act by the user.

The engineering interest of this part is almost entirely in the resolution pipeline you already own: the opaque-redirect trap is dodged by the `webRequest.onBeforeRedirect` + following-fetch design in `engine/redirect-resolver.js`, and the router's `engine/resolveRedirect` route exposes it to any content script. What this part adds is the **presentation** contract: hops render as rows (host + status code), the final host is emphasized, and any hop whose host trips the engine gets its own one-line reason. Chains that pass through known SSO hops (`login.microsoftonline.com`, `accounts.google.com`) are labeled "sign-in handoff" instead of being treated as suspicious, which kills the single largest source of false alarms in this class of feature.

Chain pattern	What the card shows	Why it is safe
bit.ly -> newspaper article	1 hop, green final host	The 99% case; instant, quiet
bit.ly -> IP-literal host	amber hop row + engine reason	Raw IP mid-chain is a strong signal
shortener -> login.microsoftonline.com -> target	hop labeled 'sign-in handoff'	SSO flows are legitimate redirect machinery
shortener -> punycode host	amber hop row	IDN in a chain user cannot see
hop loop / timeout	partial chain + 'chain did not finish'	MAX_HOPS and TIMEOUT_MS bounds

Chrome APIs used: `contextMenus` (created in Part 0's `context-menu.js`), `webRequest` observer events, service-worker `fetch` with host permissions, and `tabs.sendMessage` to deliver the result to the page. **Popup row:** "Redirect Detective - standing by" / "3 traces this session" (per-tab status line updated on each trace). **Options entries:** max hops (3-12), "auto-trace shortlinks on hover" (off by default - it is the same resolution path triggered proactively by Link Sniper). **Toggle-off cleanup:** no page footprint beyond the card, which closes; the context-menu entry hides by updating the menu

title visibility (menu remains but routes to a module whose listener is gone, so Part 0's router tolerates it silently) - and the script id `sl-cs-redirect-detective` unregisters.

2.2 Files

modules/redirect-detective/redirect-detective.js

```
/* modules/redirect-detective/redirect-detective.js - renders redirect
 * chains as a card. Resolution itself lives in the SW (Part 0). */
(function () {
  'use strict';
  if (!globalThis.SL || !SL.__bridge) return;

  /* SSO handoffs: legitimate redirect machinery, not suspicion. */
  const SSO_SAFE_HOSTS = [
    'login.microsoftonline.com', 'login.live.com',
    'accounts.google.com', 'accounts.youtube.com',
    'login.yahoo.com', 'auth0.com', 'okta.com', 'appleid.apple.com'
  ];

  const MAX_CARD_ROWS = 10;
  let traces = 0;

  function hostOf(url) {
    try { return new URL(url).hostname; } catch (e) { return url; }
  }

  function classifyHop(url) {
    const host = hostOf(url);
    if (SSO_SAFE_HOSTS.some((h) => host === h || host.endsWith('.' + h))) {
      return { kind: 'ok', note: 'sign-in handoff' };
    }
    // Ask the engine for this hop's verdict - reasons only, no state.
    return { kind: 'pending', note: host };
  }

  function renderChain(payload) {
    const chain = payload.chain || [];
    if (!chain.length) return;
    traces++;
    SL.status('redirectDetective', traces + ' traces this session');

    const rows = [];
    const shown = chain.slice(0, MAX_CARD_ROWS);
    shown.forEach((url, idx) => {
      const host = hostOf(url);
      const label = idx === 0 ? 'Start' :
        (idx === shown.length - 1 ? 'Final' : 'Hop ' + idx);
      let note = host;
      if (SSO_SAFE_HOSTS.some((h) => host === h ||
        host.endsWith('.' + h))) {
        note += ' (sign-in handoff)';
      }
      rows.push({ label, value: note });
    });
    if (chain.length > MAX_CARD_ROWS) {
      rows.push({ label: '...', value: (chain.length - MAX_CARD_ROWS) +
        ' more hops' });
    }

    // Final-host verdict shapes the card's own verdict level.
    SL.verdict(chain[chain.length - 1], {}).then((verdict) => {
      const reasons = verdict.reasons.slice();
      if (payload.error) {
        reasons.push('The chain did not finish (' + payload.error + ').');
      }
      if (payload.hops === 0) {
        reasons.push('This link does not redirect - it goes straight ' +
          'to ' + hostOf(chain[0]) + '.');
      }
      SLVerdict.card({
        verdict: verdict.verdict,

```

```

        title: 'Redirect trace (' + payload.hops + ' hops)',
        reasons,
        rows
    });
});
}

function listenForTraces() {
    SL.msg.listen('redirectDetective', (msg) => {
        if (msg.type === 'showChain') renderChain(msg.payload);
        return null;
    });
}

SL.boot('redirectDetective', {
    onEnabled() {
        listenForTraces();
        SL.status('redirectDetective', 'standing by');
    },
    onDisabled() {
        SLVerdict.closeCard(); // remove any open trace card
        SL.status('redirectDetective', 'off');
    }
});
SL.registerCleanup('redirectDetective', () => SLVerdict.closeCard());
})();

```

```
modules/redirect-detective/redirect-detective.css
```

```

/* modules/redirect-detective/redirect-detective.css
 * No extra styles: the shared .sl-card handles the trace card. */

```

2.3 Registry entry for Part 2

```
shared/module-registry.js - Redirect Detective entry (replace placeholder)
```

```

{ id: 'redirectDetective', name: 'Redirect Detective', pillar: 'linkIntel',
  blurb: 'Follows redirect chains hop by hop.',
  defaultEnabled: true,
  scripts: [{
    id: 'sl-cs-redirect-detective', matches: ['http://*', 'https://*'],
    runAt: 'document_idle', world: 'ISOLATED', allFrames: false,
    js: ['shared/messaging.js', 'shared/cs-bridge.js',
        'ui/toast.js', 'ui/verdict-card.js',
        'modules/redirect-detective/redirect-detective.js'],
    css: ['ui/tokens.css', 'ui/components.css',
        'modules/redirect-detective/redirect-detective.css']
  } ] },

```

2.4 Load & test

- Reload the extension. The SW console logs the new registration.
- On any page, right-click a shortlink (a bit.ly, t.co or tinyurl.com link from any tweet or article) and choose **Where does this link really go?** A card should appear listing each hop's host, ending with the final host and its verdict pill.
- Right-click a plain direct link: the card opens with "This link does not redirect" and the straight destination.
- Watch the timing: the card appears after the chain resolves (usually under a second); for slow hosts you will see the card only when the answer is ready - deliberately no spinner, because a trace with no answer yet shows nothing rather than anxiety.
- Trace a login flow: right-click a "Sign in with Microsoft" style link - hops through login.microsoftonline.com should render with the "(sign-in handoff)" annotation instead of alarm language.
- Flip Redirect Detective off in the popup, then right-click a link and trace: no card appears (no listener in the tab). The context-menu item stays - Part 13 documents hiding it when the module is off. Flip it back on and re-trace.

2.5 Tuning knobs

Knob	Where	Default	Effect
maxHops	settings.modules.redirectDetective	8	Chain length bound (engine + card)
MAX_HOPS	engine/redirect-resolver.js	8	Hard resolution bound
TIMEOUT_MS	engine/redirect-resolver.js	6000	Per-trace time budget
SSO_SAFE_HOSTS	redirect-detective.js	8 hosts	Handoff allowlist
MAX_CARD_ROWS	redirect-detective.js	10	Rows before 'more hops' elision

DEVIATION NOTE - The context-menu entry cannot be removed per-tab or per-module-toggle without rebuilding all menus on every settings write (contextMenus has no per-tab visibility). The chosen behavior: the entry always exists, and when the module is off the trace resolves in the SW but no card renders - no user-visible breakage, no dead click either (the resolution simply has nowhere to land).

Part 3 - Trust Badge

3.1 Design analysis

Trust seals are the internet's self-service security theater: a "Norton Secured" or "BBB Accredited" image pasted into a footer confers exactly the credibility of any other image - anyone can save, upload and display one. Trust Badge's job is to convert that theater into information at the moment it is consumed. When the scanner finds a seal-looking image, it answers two questions: does the image link anywhere at all, and if it does, does it lead to the seal issuer's real verification domain? Seals that are inert pictures, or that link somewhere unrelated to their claimed issuer, get a small amber annotation the user can expand into a card explaining precisely why the badge earns no trust.

Detection is a two-stage funnel that keeps the scan cheap. Stage one selects candidates: small images (both dimensions under 260 px) whose alt text, aria-label, file name, class names or surrounding caption text match a seal vocabulary ("norton", "mcafee", "secured", "ssl", "verified", "accredited", "trust", "hacker safe", "godaddy verified"). Stage two judges each candidate against the issuer table: a mapping from vocabulary hits to the official verification domains for that seal (norton -> norton.com / symantec.com, mcafee -> mcafee.com, bbb -> bbb.org, trustpilot -> trustpilot.com, and so on). A seal with no enclosing link, or a link whose host is not in the issuer's domain set, is annotated. Everything else stays silent - the module's noise budget is spent only where a claim is being visually made.

Seal situation	Annotation	Card says
Seal image, no link at all	amber outline + note	'This badge is just an image - anyone can paste one. It links nowhere.'
Seal links to issuer's verification domain	none	(genuine seal; leave it alone)
Seal links to a different domain	amber outline + note	'This badge claims {issuer} but points to {host} - not the issuer.'
Seal links via javascript: or #	amber outline + note	'The badge cannot be verified: its link goes nowhere.'

Chrome APIs used: none beyond the bridge - the module is a pure DOM scanner plus engine consultation for the link host. That keeps it in the same safety class as Link Sniper: no privileged surface, no interception, annotation only. **Popup row:** "Trust Badge - 1 unverifiable seal on this page". **Options entries:** "annotate only seals in footers/near payment forms" scope toggle (reduces scans on image-heavy pages), seal vocabulary is a code constant, not user-editable, to keep the table curated. **Toggle-off cleanup:** all annotations and outline classes removed, scanner observer disconnected, status line cleared, script id `sl-cs-trust-badge` unregistered.

3.2 Files

modules/trust-badge/trust-badge.js

```
/* modules/trust-badge/trust-badge.js - trust-seal authenticity notes.
 * Stage 1: find seal-looking images. Stage 2: check the claim. */
(function () {
  'use strict';
  if (!globalThis.SL || !SL.__bridge) return;

  const MAX_IMG_EDGE = 260;      // px; seals are small, heroes are not
  const SCAN_DELAY_MS = 800;    // debounce after DOM settles
  const ISSUER_TABLE = [
    { issuer: 'Norton / Symantec', words: ['norton', 'symantec',
      'hacker safe', 'norton secured'],
      domains: ['norton.com', 'symantec.com', 'safeweb.norton.com'] },
    { issuer: 'McAfee', words: ['mcafee', 'mcafee secure'],
      domains: ['mcafee.com', 'mcafeesecure.com'] },
    { issuer: 'BBB', words: ['bbb', 'bbb accredited', 'better business'],
      domains: ['bbb.org'] },
    { issuer: 'Trustpilot', words: ['trustpilot', 'trust pilot'],
      domains: ['trustpilot.com'] },
    { issuer: 'TRUSTe', words: ['truste', 'trustarc', 'privacy trusted'],
      domains: ['truste.com', 'trustarc.com'] },
    { issuer: 'GoDaddy', words: ['godaddy verified', 'godaddy'],
      domains: ['godaddy.com', 'verify.godaddy.com'] },
    { issuer: 'SSL / padlock art', words: ['ssl secure', 'ssl certified',
      'ssl verified', 'comodo', 'sectigo', 'digicert', 'globalsign'],
      domains: ['comodo.com', 'sectigo.com', 'digicert.com',
        'globalsign.com', 'ssls.com'] },
    { issuer: 'Yotpo / reviews', words: ['yotpo', 'verified reviews',
      'certified reviews'],
      domains: ['yotpo.com'] }
  ];

  let observer = null;
  let scanTimer = 0;
  const annotated = [];        // elements to clean up on toggle-off

  function textAround(img) {
    const bits = [img.getAttribute('alt') || '',
      img.getAttribute('aria-label') || '',
      img.getAttribute('title') || '',
      img.getAttribute('src') || '',
      img.className && String(img.className) || ''];
    // caption-ish text directly around the seal
    const parent = img.closest('figure, a, div, span');
    if (parent && parent.textContent &&
      parent.textContent.length < 140) {
      bits.push(parent.textContent);
    }
    return bits.join(' ').toLowerCase();
  }

  function matchIssuer(haystack) {
    for (const entry of ISSUER_TABLE) {
      for (const w of entry.words) {
        if (haystack.includes(w)) return entry;
      }
    }
    return null;
  }

  function verdictForSeal(img, issuer) {
    const link = img.closest('a');
    const href = link ? link.getAttribute('href') : '';
    if (!link || !href || href === '#' ||
      /^javascript:/.test(href)) {
      return {
        kind: 'warn',
        note: 'This badge is just an image - anyone can paste one. ' +
          'It links nowhere.'
      };
    }
  }
}
```

```

let host = '';
try { host = new URL(href, location.href).hostname; } catch (e) {}
const legit = issuer.domains.some((d) => host === d ||
  host.endsWith('.' + d));
if (legit) return { kind: 'ok', note: '' };
return {
  kind: 'warn',
  note: 'This badge claims ' + issuer.issuer + ' but points to ' +
    (host || 'an unknown host') + ' - not the issuer.'
};
}

function annotate(img, issuer, verdict) {
  img.classList.add('sl-outline-warn');
  img.classList.add('sl-tb-marked');
  img.setAttribute('data-sl-note', verdict.note);
  img.addEventListener('click', (e) => {
    // Only when the seal itself is clicked without following a link
    if (verdict.kind === 'warn' && !img.closest('a[href]')) {
      e.preventDefault();
    }
    SLVerdict.card({
      verdict: 'SECOND_LOOK',
      title: issuer.issuer + ' seal',
      reasons: [verdict.note],
      rows: [{ label: 'Badge', value: issuer.issuer }],
      anchorEl: img
    });
  });
  annotated.push(img);
}

function scan() {
  if (document.readyState !== 'complete' &&
    document.readyState !== 'interactive') return;
  let flagged = 0;
  for (const img of document.querySelectorAll('img')) {
    if (img.classList.contains('sl-tb-marked')) continue;
    const w = img.naturalWidth || img.width || 0;
    const h = img.naturalHeight || img.height || 0;
    if (!w || !h) continue;
    if (w > MAX_IMG_EDGE || h > MAX_IMG_EDGE) continue;
    const issuer = matchIssuer(textAround(img));
    if (!issuer) continue;
    const verdict = verdictForSeal(img, issuer);
    if (verdict.kind === 'warn') {
      annotate(img, issuer, verdict);
      flagged++;
    }
  }
  SL.status('trustBadge', flagged
    ? flagged + ' unverifiable seal' + (flagged > 1 ? 's' : '') +
      ' on this page'
    : 'No suspicious seals');
}

function scheduleScan() {
  clearTimeout(scanTimer);
  scanTimer = setTimeout(scan, SCAN_DELAY_MS);
}

function attach() {
  scan();
  observer = new MutationObserver(scheduleScan);
  observer.observe(document.documentElement || document.body, {
    childList: true, subtree: true
  });
}

function detach() {
  clearTimeout(scanTimer);
  if (observer) { observer.disconnect(); observer = null; }
  for (const el of annotated) {
    el.classList.remove('sl-outline-warn', 'sl-tb-marked');
  }
}

```

```

    el.removeAttribute('data-sl-note');
  }
  annotated.length = 0;
  SLVerdict.closeCard();
  SL.status('trustBadge', 'off');
}

SL.boot('trustBadge', {
  onEnabled() { attach(); },
  onDisabled() { detach(); }
});
SL.registerCleanup('trustBadge', detach);
})();

```

```
modules/trust-badge/trust-badge.css
```

```

/* modules/trust-badge/trust-badge.css */
.sl-tb-marked {
  cursor: pointer;
}

```

3.3 Registry entry for Part 3

```
shared/module-registry.js - Trust Badge entry (replace placeholder)
```

```

{ id: 'trustBadge', name: 'Trust Badge', pillar: 'linkIntel',
  blurb: 'Flags trust seals that are just images.',
  defaultEnabled: true,
  scripts: [{
    id: 'sl-cs-trust-badge', matches: ['http://*', 'https://*'],
    runAt: 'document_idle', world: 'ISOLATED', allFrames: false,
    js: ['shared/messaging.js', 'shared/cs-bridge.js',
        'ui/toast.js', 'ui/verdict-card.js',
        'modules/trust-badge/trust-badge.js'],
    css: ['ui/tokens.css', 'ui/components.css',
        'modules/trust-badge/trust-badge.css']
  } ] },

```

3.4 Load & test

- Reload the extension; the SW console logs `sl-cs-trust-badge` registered.
- Open any small online shop. Most will show either no annotation (genuine or no seals) or, on sketchier ones, amber-dashed outlines around footer seals. Click an annotated seal: the card explains the exact mismatch.
- Inject a fake seal on any page from its DevTools console: `var i=document.createElement('img');i.src='https://via.placeholder.com/120x48.png?text=Norton+Secured';i.alt='Norton Secured';document.body.appendChild(i);` (any small placeholder image with that alt text works) - after ~800 ms the image gets the amber dashed outline, and clicking it opens the "just an image" card.
- Now wrap that image in a link to the real issuer: `var a=document.createElement('a');a.href='https://www.norton.com';a.appendChild(i);document.body.appendChild(a);` - after a rescan it is no longer flagged (reload the page first, since scanned images are not re-judged).
- Flip Trust Badge off: outlines vanish from every open tab instantly. Flip back on: outlines reappear after the rescan.

3.5 Tuning knobs

Knob	Where	Default	Effect
MAX_IMG_EDGE	trust-badge.js	260 px	Seal candidate size ceiling
SCAN_DELAY_MS	trust-badge.js	800	Debounce for dynamic content
ISSUER_TABLE	trust-badge.js	8 issuers	Vocabulary + verification domains

TIP - The 260 px ceiling is what keeps this scanner quiet on real sites. Raising it pulls hero logos and article images into stage one and multiplies false candidates; if you extend `ISSUER_TABLE`, prefer tightening the vocabulary words over loosening the size bound.

Part 4 - Form Guardian

4.1 Design analysis

Form Guardian is the first module allowed to say **INTERCEPTED**, and it earns it with hard evidence: a form on the page you are looking at is about to send your password or card number to a **different origin** than the one in your address bar. That single pattern - visible origin A, credential destination B - is the mechanics of most phishing pages ever built, and it is checkable deterministically at submit time, which is why this module intercepts instead of annotating. Everything else about the design follows from the discipline that interception must never be wrong **AND** never be permanent: the layer always offers Stop and Continue, the form's contents are untouched, and Continue re-fires the real submit event with the module stepping aside.

Field detection leans on the platform before heuristics: `input[type=password]` is definitive for credentials; card fields are identified by `autocomplete` hints (`cc-*`) or the well-known name/id vocabulary plus numeric input modes. Action resolution leans on the platform too - the `form.action` IDL property returns an absolutely-resolved URL even when the attribute is empty or relative (the trap that catches naive `getAttribute('action')` implementations, which read "" and mis-blame the page's own origin). Approval memory is salted and one-way: the service worker stores SHA-256(salt + destination origin), never the origin text, so the "sites I already trust with this" list is useless if exfiltrated.

Submit pattern	Behavior	Rationale
Password field, action origin == page origin	silent	The normal case: 99.9% of logins
Password field, action origin != page origin, not approved	full-page INTERCEPTED card	The phishing mechanic, caught exactly
Card hints, cross-origin action	full-page INTERCEPTED card	Card numbers deserve the same bar
Cross-origin POST to SSO allowlist (login.microsoftonline.com, ...)	silent	Legitimate handoff flows
Previously approved destination (salted hash match)	silent	User already decided; no nagging
Same-origin action but page is itself suspicious	nothing here	Link/Site modules' job, not Form Guardian's

Chrome APIs used: none in the content world beyond the bridge; the approval-hash route runs in the service worker using `crypto.subtle` and `chrome.storage.local`. **Popup row:** "Form Guardian - watching 2 forms" / "1 password form on this page". **Options entries:** "remember approved destinations" toggle (on by default), an "approved destinations" count with a Clear button (Part 11). **Toggle-off cleanup:** the intercept layer is removed if present, submit listeners detach, and any form whose native submission was prevented gets its submit event re-dispatched with the `bypass` flag set - the page is left exactly as found, mid-submit and unharmed.

4.2 Files

```

modules/form-guardian/form-guardian.js
/* modules/form-guardian/form-guardian.js - credential exfiltration
 * stop. The only module allowed to escalate to INTERCEPTED alongside
 * Download Guard. */
(function () {
  'use strict';
  if (!globalThis.SL || !SL.__bridge) return;

  /* Legitimate cross-origin SSO handoffs - never warn on these. */
  const SSO_ACTION_ALLOWLIST = [
    'login.microsoftonline.com', 'login.live.com',
    'accounts.google.com', 'appleid.apple.com',
    'auth0.com', 'okta.com', 'secure.login.gov'
  ];

  const CARD_AUTOCOMPLETE = /^cc-(name|number|csc|exp|type|cvc)/i;
  const CARD_NAME_RE = /(card|cc|cvc|cvv|csc|expir|pan)[_\-]?/i;
  let cfg = { rememberApproved: true };
  let bypass = false;          // set while re-dispatching after Continue
  let layer = null;            // the intercept DOM
  let heldForm = null;         // form awaiting the user's decision
  let watching = 0;

  function originOf(url) {
    try { return new URL(url, location.href).origin; } catch (e) {
      return location.origin;
    }
  }

  function isSensitive(form) {
    if (form.querySelector('input[type=password]')) return 'password';
    for (const el of form.querySelectorAll('input, select')) {
      const ac = el.getAttribute('autocomplete') || '';
      if (CARD_AUTOCOMPLETE.test(ac)) return 'card';
      const name = (el.name || '') + ' ' + (el.id || '');
      const numeric = (el.getAttribute('inputmode') === 'numeric' ||
        /\d[\d\s\-\]{11,23}$/.test(el.value || ''));
      if (CARD_NAME_RE.test(name) && numeric) return 'card';
    }
    return null;
  }

  function allowlisted(actionOrigin) {
    const host = new URL(actionOrigin).hostname;
    return SSO_ACTION_ALLOWLIST.some((h) =>
      host === h || host.endsWith('.' + h));
  }

  /* The IDL property form.action is ALWAYS an absolute URL - this is
   * the correct way to resolve empty/relative action attributes. */
  function actionUrl(form) {
    try { return new URL(form.action || location.href, location.href); }
    catch (e) { return new URL(location.href); }
  }

  function onSubmit(e) {
    if (bypass) return;          // our own re-dispatch
    const form = e.target;
    if (!form || form.tagName !== 'FORM') return;
    const kind = isSensitive(form);
    if (!kind) return;
    const action = actionUrl(form);
    const destOrigin = action.origin;
    if (destOrigin === location.origin) return;    // normal case
    if (allowlisted(destOrigin)) return;          // SSO handoff

    // Hard evidence + credentials in flight -> INTERCEPTED.
    e.preventDefault();
    e.stopImmediatePropagation();
    heldForm = form;
    SL.verdict(action.href, { credentials: true, escalate: true,
      displayText: kind })
      .then((v) => showLayer(form, action, kind, v));
  }

```

```

}

function showLayer(form, action, kind, verdict) {
  removeLayer();
  layer = document.createElement('div');
  layer.className = 'sl-intercept sl-root';

  const card = document.createElement('div');
  card.className = 'sl-intercept__card';
  const title = document.createElement('div');
  title.className = 'sl-intercept__title';
  title.textContent = 'Your ' +
    (kind === 'password' ? 'password' : 'card number') +
    ' is about to leave this site';
  const body = document.createElement('div');
  body.className = 'sl-intercept__body';
  const destHost = action.hostname;
  body.textContent = 'You are on ' + location.hostname +
    ', but this form sends what you type to ' + destHost +
    '. If that surprises you, Stop and look again.';

  const actions = document.createElement('div');
  actions.className = 'sl-intercept__actions';

  const stop = document.createElement('button');
  stop.className = 'sl-btn sl-btn--danger';
  stop.type = 'button';
  stop.textContent = 'Stop';
  stop.addEventListener('click', () => {
    removeLayer();
    heldForm = null;
    SL.status('formGuardian', 'blocked a ' + kind + ' to ' + destHost);
  });

  const go = document.createElement('button');
  go.className = 'sl-btn';
  go.type = 'button';
  go.textContent = 'Continue';
  go.addEventListener('click', () => {
    proceed(cfg.rememberApproved !== false);
  });

  actions.appendChild(stop);
  actions.appendChild(go);
  card.appendChild(title);
  card.appendChild(body);
  if (verdict.reasons && verdict.reasons.length) {
    const why = document.createElement('div');
    why.className = 'sl-card__reason';
    why.textContent = verdict.reasons[0];
    card.appendChild(why);
  }
  card.appendChild(actions);
  layer.appendChild(card);
  (document.body || document.documentElement).appendChild(layer);
  stop.focus();
}

function proceed(remember) {
  const form = heldForm;
  removeLayer();
  heldForm = null;
  if (!form) return;
  if (remember) {
    const action = actionUrl(form);
    SL.send('formGuardian', 'rememberOrigin', { origin: action.origin });
  }
  // Re-fire the submit event with the module standing aside:
  // requestSubmit() runs the page's own handlers and validation,
  // then performs the native submission - the faithful path.
  bypass = true;
  try {
    if (typeof form.requestSubmit === 'function') form.requestSubmit();
    else form.submit(); // legacy fallback
  }
}

```

```

    } finally {
      bypass = false;
    }
  }

function removeLayer() {
  if (layer) { layer.remove(); layer = null; }
}

function countForms() {
  const forms = document.querySelectorAll('form');
  let sensitive = 0;
  for (const f of forms) if (isSensitive(f)) sensitive++;
  watching = forms.length;
  SL.status('formGuardian', sensitive
    ? sensitive + ' sensitive form' + (sensitive > 1 ? 's' : '') +
      ' watched on this page'
    : 'watching ' + watching + ' forms');
}

let formObserver = null;

function attach() {
  document.addEventListener('submit', onSubmit, true);
  countForms();
  formObserver = new MutationObserver(() => countForms());
  formObserver.observe(document.documentElement || document.body, {
    childList: true, subtree: true
  });
}

function detach() {
  document.removeEventListener('submit', onSubmit, true);
  if (formObserver) { formObserver.disconnect(); formObserver = null; }
  removeLayer();
  heldForm = null;
  SL.status('formGuardian', 'off');
}

SL.boot('formGuardian', {
  onEnabled(config) { cfg = Object.assign(cfg, config); attach(); },
  onConfig(next) { cfg = Object.assign(cfg, next || {}); },
  onDisabled() { detach(); }
});
SL.registerCleanup('formGuardian', detach);
})();

```

```
modules/form-guardian/form-guardian.css
```

```

/* modules/form-guardian/form-guardian.css
 * The intercept layer is built entirely from components.css classes
 * (.sl-intercept, .sl-intercept__card, __title, __body, __actions). */

```

4.3 Background route + registry entry

Two small edits land with this part. First, the router route that stores salted approval hashes - paste this block anywhere after the `register('engine', ...)` block in `background/module-router.js`. Second, the registry entry.

```
background/module-router.js - add after the engine routes
```

```

/* Approved credential destinations, stored as salted hashes. */
register('formGuardian', {
  async rememberOrigin(payload) {
    const salt = await SL.Storage.ensureSalt();
    const data = new TextEncoder().encode(salt + payload.origin);
    const digest = await crypto.subtle.digest('SHA-256', data);
    const hash = Array.from(new Uint8Array(digest), (b) =>
      b.toString(16).padStart(2, '0')).join('');
    const list = await SL.Storage.localGet('sl.approved', []);
    if (!list.includes(hash)) {
      list.push(hash);
      await SL.Storage.localSet('sl.approved', list.slice(-300));
    }
  }
});

```

```

    }
    return { stored: true };
  },
  async originApproved(payload) {
    const salt = await SL.Storage.ensureSalt();
    const data = new TextEncoder().encode(salt + payload.origin);
    const digest = await crypto.subtle.digest('SHA-256', data);
    const hash = Array.from(new Uint8Array(digest), (b) =>
      b.toString(16).padStart(2, '0')).join('');
    const list = await SL.Storage.localGet('sl.approved', []);
    return { approved: list.includes(hash) };
  },
  async clearApproved() {
    await SL.Storage.localSet('sl.approved', []);
    return { cleared: true };
  }
});

```

shared/module-registry.js - Form Guardian entry (replace placeholder)

```

{ id: 'formGuardian', name: 'Form Guardian', pillar: 'clickSafe',
  blurb: 'Stops passwords leaving for the wrong domain.',
  defaultEnabled: true,
  scripts: [{
    id: 'sl-cs-form-guardian', matches: ['http://*/*', 'https://*/*'],
    runAt: 'document_idle', world: 'ISOLATED', allFrames: false,
    js: ['shared/messaging.js', 'shared/cs-bridge.js',
      'ui/toast.js', 'ui/verdict-card.js',
      'modules/form-guardian/form-guardian.js'],
    css: ['ui/tokens.css', 'ui/components.css',
      'modules/form-guardian/form-guardian.css']
  } ] },

```

4.4 Load & test

- Reload the extension; confirm `sl-cs-form-guardian` in the SW console registration log.
- Open any site with a real login form (your webmail, a social network) and submit it: nothing should appear - same-origin submits are the silent majority.
- Inject a hostile form from DevTools and submit it: `var f=document.createElement('form');f.method='POST';f.action='https://httpbin.org/post';f.innerHTML='<input name=p type=password><button>Sign in</button>';document.body.appendChild(f);` then type anything into the password box and click Sign in. The full-page red card should stop the submit dead, naming the destination host.
- Click **Continue**: the submit proceeds (httpbin will echo your field back), and a second submit of the same form is silent - the destination is now remembered as a salted hash.
- Click **Stop** on a fresh hostile form: the layer dismisses, the page is untouched, and the status line reads "blocked a password to ...".
- Flip Form Guardian off in the popup while the red card is showing: the card vanishes and the hostile form submits normally with no module present.

4.5 Tuning knobs

Knob	Where	Default	Effect
SSO_ACTION_ALLOWLIST	form-guardian.js	7 hosts	Legit handoff origins
CARD_AUTOCOMPLETE / CARD_NAME_RE	form-guardian.js	-	Card field recognition
sl.approved cap	background/module-router.js	300	Memory list bound
rememberApproved	settings.modules.formGuardian	true	Remembering on/off

DEVIATION NOTE - Form Guardian intercepts native form submissions only. Credential exfiltration performed by page JavaScript through `fetch/XHR` is invisible to it - covering that would require request-body inspection, which MV3 makes impractical for store-listed extensions and which the design philosophy rejects anyway (we stop the classic phishing mechanic, the one that accounts for the overwhelming majority of real victims, and stay honest about the rest).

Part 5 - Ghost Click Blocker

5.1 Design analysis

The invisible overlay is the oldest trick in the annoyance playbook: a full-viewport div with opacity 0 (or 0.01) sits on top of the content, so the ad network's click handler fires on **your** click and the button you aimed at never sees it. Ghost Click Blocker's answer is deliberately surgical: when an element is provably a click-capturing ghost, the module sets `pointer-events: none` on it - and that is **all** it does. It never removes the element, never touches the page's JavaScript, never fakes clicks. The overlay still exists; it just cannot see your pointer anymore. This is also why the cleanup story is perfect: deleting one inline style attribute restores the page bit-for-bit.

The ghost test is a conjunction of five conditions, all of which must hold, re-verified after a persistence window: computed opacity below 0.15; actually laid out with nonzero size; covering at least 50% of the viewport; pointer-interactive (`pointer-events` not already none) and on top of the main content; and **contentless** - no text, at most two child elements, no form controls, no media. The contentless condition is what keeps video-player overlay divs (which are also large and transparent-ish) out of the blast radius: player wrappers have real children and often real function. Candidates must persist in this state for 600 ms before being neutralized, which filters load-time scaffolding that pages tear down themselves.

Page element	Ghost verdict	Why
Full-viewport div, opacity 0.01, empty, stays 600 ms	neutralized	The textbook click thief
Video player transparent control layer	left alone	Has interactive children; contentless test fails
Cookie overlay backdrop (dimmed, opacity 0.6)	left alone	Opacity 0.6 fails the < 0.15 bar
Sticky header that is transparent but has text	left alone	Contentless test fails
Ad landing interstitial with an iframe	left alone	iframes count as content
Skeleton shimmer overlay during load (< 600 ms)	left alone	Persistence window not met

Chrome APIs used: none beyond the bridge - pure DOM and computed styles. **Popup row:** "Ghost Click Blocker - 2 ghosts neutralized on this page", with a **Show me** action in Part 11's polished popup that flashes the neutralized elements (the module listens for a `flashOverlays` message; the flash outline is temporary and removes itself). **Options entries:** annotate toggle (default off - silent operation), viewport ratio slider (30-90%), persistence window (300-2000 ms). **Toggle-off cleanup:** every neutralized element gets its inline `pointer-events` value restored and its `data-sl-ghost` attribute removed; observers disconnect; the flash style rule if present is dropped; the script id unregisters.

5.2 Files

```
modules/ghost-click/ghost-click.js
/* modules/ghost-click/ghost-click.js - neutralize invisible
 * click-capturing overlays with pointer-events:none. NEVER removes
 * elements, NEVER touches page JavaScript. */
(function () {
  'use strict';
  if (!globalThis.SL || !SL.__bridge) return;

  /* All five conditions must hold simultaneously. */
```

```

const OPACITY_MAX = 0.15;
const VIEWPORT_RATIO_MIN = 0.5;
const PERSIST_MS = 600;
const MAX_CHILDREN = 2;
const RESCAN_MS = 900;

const neutralized = new Map(); // element -> saved inline pointer-events

function computed(el) {
  return window.getComputedStyle(el);
}

function coversViewport(el, rect) {
  const vw = window.innerWidth, vh = window.innerHeight;
  if (!vw || !vh || !rect.width || !rect.height) return false;
  const covered = Math.min(rect.width, vw) * Math.min(rect.height, vh);
  return covered / (vw * vh) >= VIEWPORT_RATIO_MIN;
}

function isContentless(el) {
  if ((el.textContent || '').trim().length > 0) return false;
  if (el.childElementCount > MAX_CHILDREN) return false;
  // anything functional or media means this is page machinery
  return !el.querySelector(
    'input, select, textarea, button, a, video, audio, canvas, ' +
    'iframe, svg, img, label, form, object, embed');
}

function isGhost(el) {
  const style = computed(el);
  if (style.visibility === 'hidden' || style.display === 'none') {
    return false; // not participating in layout
  }
  if (parseFloat(style.opacity) > OPACITY_MAX) return false;
  if (style.pointerEvents === 'none') return false; // already inert
  const rect = el.getBoundingClientRect();
  if (!rect.width || !rect.height) return false;
  if (!coversViewport(el, rect)) return false;
  if (parseInt(style.zIndex || '0', 10) < 0) return false;
  return isContentless(el);
}

function neutralize(el) {
  neutralized.set(el, el.style.pointerEvents);
  el.style.setProperty('pointer-events', 'none', 'important');
  el.setAttribute('data-sl-ghost', '1');
  if (cfg.annotate) el.classList.add('sl-ghost-flash');
  count = neutralized.size;
  SL.status('ghostClick', count + ' ghost' + (count === 1 ? '' : 's') +
    ' neutralized on this page');
}

function restore(el) {
  const saved = neutralized.get(el);
  if (typeof saved === 'string' && saved !== '') {
    el.style.pointerEvents = saved;
  } else {
    el.style.removeProperty('pointer-events');
  }
  el.removeAttribute('data-sl-ghost');
  el.classList.remove('sl-ghost-flash');
  neutralized.delete(el);
}

function restoreAll() {
  for (const el of Array.from(neutralized.keys())) {
    if (el.isConnected) restore(el);
    else neutralized.delete(el);
  }
}

/* Candidates must PERSIST as ghosts before we act. */
let candidates = new Map(); // element -> first-seen timestamp
let cfg = { annotate: false };

```

```

let count = 0;
let scanTimer = 0;

function scan() {
  const now = Date.now();
  const seen = new Set();
  for (const el of document.body ? document.body.querySelectorAll('*') : []) {
    const style = computed(el);
    if (style.position !== 'fixed' && style.position !== 'absolute') {
      continue;
    }
    if (!isGhost(el)) continue;
    seen.add(el);
    if (!candidates.has(el)) candidates.set(el, now);
    else if (now - candidates.get(el) >= PERSIST_MS &&
      !neutralized.has(el)) {
      neutralize(el);
    }
  }
  // forget candidates that stopped qualifying (page tore them down)
  for (const el of Array.from(candidates.keys())) {
    if (!seen.has(el) && !neutralized.has(el)) candidates.delete(el);
    if (!el.isConnected) { candidates.delete(el); neutralized.delete(el); }
  }
}

function schedule() {
  clearTimeout(scanTimer);
  scanTimer = setTimeout(() => { scan(); schedule(); }, RESCAN_MS);
}

/* Popup 'Show me': flash outlines for 2.5 seconds, then fade out. */
SL.msg.listen('ghostClick', (msg) => {
  if (msg.type === 'flashOverlays') {
    for (const el of neutralized.keys()) {
      el.classList.add('sl-ghost-flash');
      setTimeout(() => el.classList.remove('sl-ghost-flash'), 2500);
    }
  }
  return null;
});

function attach() {
  scan();
  schedule();
  SL.status('ghostClick', '0 ghosts neutralized on this page');
}

function detach() {
  clearTimeout(scanTimer);
  restoreAll();
  candidates = new Map();
  count = 0;
  SL.status('ghostClick', 'off');
}

SL.boot('ghostClick', {
  onEnabled(config) { cfg = Object.assign(cfg, config); attach(); },
  onConfig(next) { cfg = Object.assign(cfg, next || {}); },
  onDisabled() { detach(); }
});
SL.registerCleanup('ghostClick', detach);
})();

```

```
modules/ghost-click/ghost-click.css
```

```

/* modules/ghost-click/ghost-click.css
* .sl-ghost-flash lives in components.css; nothing else needed. */

```

5.3 Registry entry for Part 5

```
shared/module-registry.js - Ghost Click Blocker entry (replace placeholder)
```

```
{ id: 'ghostClick', name: 'Ghost Click Blocker', pillar: 'clickSafe',
  blurb: 'Neutralizes invisible click-stealing overlays.',
  defaultEnabled: true,
  scripts: [{
    id: 'sl-cs-ghost-click', matches: ['http://*', 'https://*'],
    runAt: 'document_idle', world: 'ISOLATED', allFrames: false,
    js: ['shared/messaging.js', 'shared/cs-bridge.js',
        'ui/toast.js', 'ui/verdict-card.js',
        'modules/ghost-click/ghost-click.js'],
    css: ['ui/tokens.css', 'ui/components.css',
        'modules/ghost-click/ghost-click.css']
  }] },
```

5.4 Load & test

- Reload the extension and confirm the registration log line.
- Inject a textbook ghost into any page from DevTools, then try to click anything underneath it: `var g=document.createElement('div');g.style.cssText='position:fixed;inset:0;background:#000;opacity:0.01;z-index:2147483647;';g.addEventListener('click',function(){console.log('GHOST CAPTURED YOUR CLICK')});document.body.appendChild(g);` - within about a second (persistence + rescan cadence) the console stops printing captures and clicks reach the real page again.
- Prove it is the module: flip Ghost Click Blocker off in the popup - the ghost instantly captures clicks again (its style is restored). Flip it back on and wait one rescan cycle.
- Negative test - a *legitimate* transparent layer with content: `var ok=document.createElement('div');ok.style.cssText='position:fixed;inset:0;opacity:0.01;z-index:2147483646;';ok.innerHTML='<button>Real overlay button</button>';document.body.appendChild(ok);` - it is never neutralized (contentless test fails), and its button remains clickable.
- Check the popup status line ("1 ghost neutralized on this page"); in Part 11 the Show me button will flash these elements.

5.5 Tuning knobs

Knob	Where	Default	Effect
OPACITY_MAX	ghost-click.js	0.15	Opacity bar for ghost candidacy
VIEWPORT_RATIO_MIN	ghost-click.js	0.5	Minimum viewport coverage
PERSIST_MS	ghost-click.js	600	Time a ghost must persist
MAX_CHILDREN	ghost-click.js	2	Contentless ceiling
RESCAN_MS	ghost-click.js	900	Scan cadence
annotate	settings.modules.ghostClick	false	Persistent outline on ghosts

TIP - The `pointer-events: none` approach is also the honest cleanup story: because the module's entire footprint is one inline style per ghost plus a data attribute, "OFF is real OFF" restores the page by deleting exactly what it added. If you are ever tempted to also delete ghost elements, resist - deletion is irreversible page surgery and breaks the guarantee.

Part 6 - Fake Download Button

6.1 Design analysis

On a download site, the real button is the one that starts the file; the fake ones are decoys whose clicks pay someone else. Humans distinguish them by gestalt - real buttons do not come in threes, do not float in corners, and do not live next to the word "Advertisement". Fake Download Button encodes exactly that gestalt. It collects all download-worded interactive elements on the page, then flags the ones that look like decoys: duplicates competing for the same verb, elements positioned in ad zones (fixed/absolute corners, aside regions, within labelled advertisement containers), or links that do not actually point at a file and lead cross-origin. Every flag is an annotation - amber dashed outline plus a hover pill with one plain sentence - never a block; the module's contract is "mark the suspects," and Download Guard (Part 7) is the layer that inspects what actually downloads.

The heuristic funnel keeps precision high on real pages. A candidate must be an `a` or `button` whose trimmed text matches the download vocabulary (under 40 characters, so "Download the complete 90-page user manual" does not count). The page's *primary* download is identified and protected: the candidate whose href most resembles a direct file (same-origin, or ending in a known file extension) and that lives inside main content - it gets a green check instead of suspicion. Only when at least two decoy-pattern candidates remain does the module annotate; a single ambiguous button is left alone, because the cost of a wrong amber outline on a legit button is higher than the value of one caught decoy.

Pattern	Annotation	Reasoning
4+ 'Download' buttons scattered in corners/sidebars	amber on the decoys, green check on the primary	Competing buttons is itself the tell
'Free Download' button inside a container labelled Advertisement/Sponsored	amber	Explicit ad placement
Download button with javascript: or # href	amber	Real downloads have destinations
Direct same-origin link ending in .zip/.pdf/.exe/... in main content	green check (primary)	Looks like the actual file
Single download-ish button, ambiguous placement	none	One ambiguous button is not a signal
Browsers' own download prompts, extension popups	none (not page DOM)	Out of scope by construction

Chrome APIs used: none beyond the bridge. **Popup row:** "Fake Download Button - 2 decoy buttons marked". **Options entries:** "mark the primary download too" (green check, on by default), decoy vocabulary is code-curated. **Toggle-off cleanup:** outlines, attributes and hover pills removed from every marked element; scanner disconnected; script id `sl-cs-fake-download` unregisters.

6.2 Files

```
modules/fake-download/fake-download.js
/* modules/fake-download/fake-download.js - mark decoy download
 * buttons. Annotation only: never blocks, never hides. */
(function () {
  'use strict';
  if (!globalThis.SL || !SL.__bridge) return;

  const TEXT_MAX = 40;
  const CLUSTER_MIN = 2;           // decoys needed before annotating
  const SCAN_DELAY_MS = 800;
  const DOWNLOAD_WORDS = [
```

```

    'download', 'start download', 'free download', 'download now',
    'get it now', 'start now', 'direct download', 'click to download'
];
const FILE_EXT_RE = /\.(zip|rar|7z|tar|gz|exe|msi|dmg|pkg|apk|iso|pdf|mp3|mp4|avi|mkv)(\?|$)/i;
const AD_ZONE_RE = /(advertisement|sponsored|ads?|promo|banner)/i;

let cfg = { markPrimary: true };
let scanTimer = 0;
const marked = [];

function isCandidateText(text) {
    const t = (text || '').trim().toLowerCase();
    if (!t || t.length > TEXT_MAX) return false;
    return DOWNLOAD_WORDS.some((w) => t === w || t.startsWith(w + ' ') ||
        t.indexOf(w) !== -1);
}

function candidates() {
    const out = [];
    for (const el of document.querySelectorAll('a, button')) {
        const text = el.textContent || el.getAttribute('value') || '';
        if (!isCandidateText(text)) continue;
        out.push({ el, text: text.trim() });
    }
    return out;
}

function inAdZone(el) {
    let node = el;
    for (let depth = 0; depth < 6 && node; depth++) {
        if (node.nodeType !== 1) { node = node.parentElement; continue; }
        const clue = (node.className && String(node.className)) + ' ' +
            (node.id || '') + ' ' +
            (node.getAttribute('aria-label') || '');
        if (AD_ZONE_RE.test(clue)) return true;
        if (node.tagName === 'ASIDE') return true;
        node = node.parentElement;
    }
    const style = window.getComputedStyle(el);
    return style.position === 'fixed';
}

function hrefOf(el) {
    if (el.tagName === 'A') return el.getAttribute('href') || '';
    return '';
}

function looksLikeRealFile(el) {
    const href = hrefOf(el);
    if (!href || /^javascript:|^#/i.test(href)) return false;
    try {
        const u = new URL(href, location.href);
        if (u.origin !== location.origin) return false;
        if (FILE_EXT_RE.test(u.pathname)) return true;
        return u.pathname.split('/').pop().indexOf('.') === -1 &&
            el.closest('main, article, #content, .content') !== null;
    } catch (e) { return false; }
}

function annotate(el, note) {
    if (el.hasAttribute('data-sl-fake')) return;
    el.classList.add('sl-outline-warn');
    el.setAttribute('data-sl-fake', note);
    el.setAttribute('title', note); // native tooltip as fallback
    el.addEventListener('mouseenter', () => pillFor(el, note));
    el.addEventListener('mouseleave', hidePill);
    marked.push(el);
}

let pillEl = null;
function pillFor(el, note) {
    hidePill();
    pillEl = SLVerdict.pill('SECOND_LOOK', note);
    pillEl.classList.add('sl-ls-pill');
}

```

```

document.body.appendChild(pillEl);
const r = el.getBoundingClientRect();
pillEl.style.left = Math.min(window.scrollX + r.left,
    window.scrollX + window.innerWidth - 260) + 'px';
pillEl.style.top = (window.scrollY + r.bottom + 4) + 'px';
}
function hidePill() {
    if (pillEl) { pillEl.remove(); pillEl = null; }
}

function markPrimary(el) {
    if (!cfg.markPrimary || el.hasAttribute('data-sl-fake')) return;
    const check = SLVerdict.pill('CLEAR', 'looks like the real file');
    check.classList.add('sl-fd-primary');
    check.style.cssText = 'position:absolute;z-index:8000;';
    if (el.positionStatic) el.style.position = 'relative';
    try { el.appendChild(check); marked.push(check); } catch (e) {}
}

function scan() {
    if (!document.body) return;
    const cands = candidates();
    const decoys = [];
    let primary = null;
    for (const c of cands) {
        if (looksLikeRealFile(c.el) && !primary) { primary = c.el; continue; }
        if (inAdZone(c.el)) { decoys.push(c); continue; }
        const href = hrefOf(c.el);
        if (!href || /^javascript:|^#/i.test(href)) decoys.push(c);
    }
    // Competing verbs: everything except the primary is a suspect.
    if (cands.length >= CLUSTER_MIN + 1) {
        for (const c of cands) {
            if (c.el !== primary) decoys.push(c);
        }
    }
    if (primary) markPrimary(primary);
    const seen = new Set();
    for (const d of decoys) {
        if (seen.has(d.el)) continue;
        seen.add(d.el);
        const href = hrefOf(d.el);
        const note = cands.length >= CLUSTER_MIN + 1
            ? 'Several buttons compete for this download - real ones rarely do.'
            : 'This download button sits in an ad zone.';
        annotate(d.el, href ? note : note + ' Its link goes nowhere.');
```

```

    }
    SL.status('fakeDownload', decoys.length
        ? decoys.length + ' decoy button' +
            (decoys.length === 1 ? '' : 's') + ' marked'
        : 'No decoy download buttons');
}

function schedule() {
    clearTimeout(scanTimer);
    scanTimer = setTimeout(() => { scan(); schedule(); }, 2000);
}

function detach() {
    clearTimeout(scanTimer);
    hidePill();
    for (const el of marked) {
        if (el.classList) el.classList.remove('sl-outline-warn');
        if (el.removeAttribute) {
            el.removeAttribute('data-sl-fake');
            el.removeAttribute('title');
        }
        if (el.remove && el.classList &&
            el.classList.contains('sl-fd-primary')) el.remove();
    }
    marked.length = 0;
    SL.status('fakeDownload', 'off');
```

```

}
SL.boot('fakeDownload', {
```

```

    onEnabled(config) { cfg = Object.assign(cfg, config); scan(); schedule(); },
    onConfig(next) { cfg = Object.assign(cfg, next || {}); },
    onDisabled() { detach(); }
  });
  SL.registerCleanup('fakeDownload', detach);
})();

```

```
modules/fake-download/fake-download.css
```

```

/* modules/fake-download/fake-download.css
 * Outlines come from components.css (.sl-outline-warn). The hover pill
 * reuses .sl-ls-pill positioning from link-sniper.css, so both files
 * are injected together via the registry css array. */

```

6.3 Registry entry for Part 6

```
shared/module-registry.js - Fake Download entry (replace placeholder)
```

```

{ id: 'fakeDownload', name: 'Fake Download Button', pillar: 'clickSafe',
  blurb: 'Marks competing download buttons.',
  defaultEnabled: true,
  scripts: [{
    id: 'sl-cs-fake-download', matches: ['http://*', 'https://*'],
    runAt: 'document_idle', world: 'ISOLATED', allFrames: false,
    js: ['shared/messaging.js', 'shared/cs-bridge.js',
        'ui/toast.js', 'ui/verdict-card.js',
        'modules/fake-download/fake-download.js'],
    css: ['ui/tokens.css', 'ui/components.css',
        'modules/link-sniper/link-sniper.css',
        'modules/fake-download/fake-download.css']
  } ] },

```

6.4 Load & test

- Reload the extension and confirm the new registration log.
- The honest first test is a real download page - any software vendor's download page should produce zero amber outlines, and if it has an obvious primary file link, a small green "looks like the real file" pill attaches to it.
- Build the decoy scenario in DevTools on any page:


```
var
ad=document.createElement('div');ad.className='advertisement';var
a=document.createElement('a');a.href='https://elsewhere.example/x';a.textContent='Free
Download';ad.appendChild(a);document.body.prepend(ad);var
b=document.createElement('button');b.textContent='Download Now';b.style.position='fixed';b.styl
e.right='10px';b.style.bottom='10px';document.body.appendChild(b);
```

 - within two seconds both decoys get amber dashed outlines, and hovering them shows the plain-language note.
- Clicking a decoy still navigates (annotation, not blocking) - that is by design; if the decoy actually starts a download, Part 7 is the layer that catches the file itself.
- Flip the module off: outlines and pills vanish from every open tab; flip it on: they return after the next scan tick.

6.5 Tuning knobs

Knob	Where	Default	Effect
CLUSTER_MIN	fake-download.js	2	Decoys required before annotating
TEXT_MAX	fake-download.js	40	Candidate text length ceiling
DOWNLOAD_WORDS	fake-download.js	8 phrases	Verb vocabulary
AD_ZONE_RE	fake-download.js	see code	Ad-container classification
FILE_EXT_RE	fake-download.js	12 exts	Primary-file recognition

Knob	Where	Default	Effect
markPrimary	settings.modules.fakeDownload	true	Green check on the real one

Part 7 - Download Guard

7.1 Design analysis

Download Guard is the file-safety pillar's single module, and it earns its interception rights with bytes rather than words: when a download begins, the service worker fetches the first 16 bytes of the file with a `Range` header (host permissions make this cross-origin legal) and compares them to a magic-number table. A file named `invoice.pdf` whose bytes begin with `MZ` is a Windows executable wearing a PDF costume - that is a high-confidence contradiction, and it is the second and last situation in which SecondLook will stop something from happening. Every stop ships with an undo: "Continue anyway" re-issues the exact download through `chrome.downloads.download`.

The pipeline runs three checks in priority order, cheapest first. **Filename forensics**: double-extension disguises (`photo.jpg.exe`, `report.pdf.scr`) and outright dangerous extensions (`.scr`, `.pif`, `.hta`, `.bat`, `.cmd` from the open web) are flagged from the name alone, no network needed. **Magic-byte sniffing**: the mismatch test above, skipped for `blob:` and `data:` URLs, which the service worker cannot fetch back (the honest fallback is filename analysis plus a status note, never a guess). **Everything else passes silently** - ordinary files whose bytes match their extension category never produce so much as a log entry, because a file checker that chatters about normal files trains users to ignore it.

Situation	Verdict	Action
Name ends <code>.exe</code> but bytes are <code>%PDF</code>	SECOND LOOK note only	Contradiction is odd but harmless direction
Name ends <code>.pdf/.doc/.jpg/...</code> but bytes are <code>MZ</code> or <code>ELF</code>	INTERCEPTED	Download cancelled + undoable toast
Double extension (<code>.pdf.exe</code> , <code>.jpg.scr</code> , ...)	INTERCEPTED	Cancelled before any bytes move
<code>.blob:/data:</code> download with clean name	silent	Cannot sniff; honest fallback
Archives (<code>.zip</code>) containing archives	silent (this version)	See deviation note
Extension and bytes agree	silent	The boring majority, kept boring

Chrome APIs used: `downloads` (`onCreated`, `cancel`, `erase`, `download`), service-worker `fetch` with `Range`, `scripting.executeScript/insertCSS` for the in-page undo toast, and `tabs.query` to locate the originating tab. **Popup row**: "Download Guard - 1 file stopped this session" (per-tab status written directly by the SW). **Options entries**: `block-on-mismatch` toggle (on), "always ask before executable downloads" is native Chrome behavior and deliberately not duplicated. **Toggle-off cleanup**: no page footprint exists by default (the toast scripts are injected on demand, and the SW listener early-exits when the module is off), so there is nothing to undo - which is the cleanest possible cleanup story.

DEVIATION NOTE - `chrome.downloads.DownloadItem` carries no tab id, so the undo toast targets the tab whose URL matches the download's referrer, falling back to a toolbar badge alert if the referrer tab is gone or ambiguous. MV3 provides no better signal; this is documented rather than papered over.

7.2 Files

```

/* background/download-guard.js - magic-byte mismatch interception.
 * SW-only module: no standing content script. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.DownloadGuard) return;

  const SNIFF_BYTES = 16; // Range: bytes=0-15
  const DANGEROUS_EXT = /\.(scr|pif|hta|vbs|cpl|jar)$/i;
  const DISGUISE_RE = /\.(pdf|doc|docx|xls|xlsx|ppt|pptx|jpe?g|png|gif|txt|mp[34])\.(\.(exe|scr|bat|
  ↪ cmd|com|pif|vbs|js|jar|msi|dmg|apk))+$/i;
  const EXECUTABLE_MAGIC = [
    { bytes: [0x4d, 0x5a], label: 'Windows program (MZ)' },
    { bytes: [0x7f, 0x45, 0x4c, 0x46], label: 'Linux program (ELF)' }
  ];
  const DOCUMENT_MAGIC = [
    { bytes: [0x25, 0x50, 0x44, 0x46], label: 'PDF' },
    { bytes: [0x50, 0x4b, 0x03, 0x04], label: 'ZIP/Office' },
    { bytes: [0xd0, 0xcf, 0x11, 0xe0], label: 'legacy Office' },
    { bytes: [0xff, 0xd8, 0xff], label: 'JPEG' },
    { bytes: [0x89, 0x50, 0x4e, 0x47], label: 'PNG' }
  ];
  const DOC_EXT = /\.(pdf|docx?|xlsx?|pptx?|txt|jpe?g|png|gif|csv)$/i;

  let stoppedThisSession = 0;

  async function moduleOn(host) {
    const s = await SL.Settings.get();
    return SL.Settings.isModuleOn(s, 'downloadGuard', host || null);
  }

  function nameOf(item) {
    return (item.filename || item.url || '').split(/[\\\/]/).pop() || '';
  }

  async function sniff(url) {
    // blob:/data: cannot be re-fetched by the SW - honest fallback.
    if (/^(blob|data):/i.test(url)) return { skipped: true };
    try {
      const res = await fetch(url, {
        headers: { Range: 'bytes=0-' + (SNIFF_BYTES - 1) },
        credentials: 'omit', cache: 'no-store'
      });
      const buf = new Uint8Array(await res.arrayBuffer());
      return { bytes: buf.slice(0, SNIFF_BYTES) };
    } catch (err) {
      return { skipped: true, error: String(err) };
    }
  }

  function startsWith(bytes, prefix) {
    if (!bytes) return false;
    for (let i = 0; i < prefix.length; i++) {
      if (bytes[i] !== prefix[i]) return false;
    }
    return true;
  }

  async function findOriginTab(item) {
    const ref = item.referrer || item.finalUrl || item.url;
    try {
      const tabs = await chrome.tabs.query({ url: ref });
      if (tabs.length) return tabs[0];
      const host = new URL(ref).hostname;
      const fallback = await chrome.tabs.query({ url:
        'https://' + host + '/*' });
      return fallback[0] || null;
    } catch (e) { return null; }
  }

  async function notifyUser(item, reason, name) {
    const tab = await findOriginTab(item);
  }

```

```

if (!tab) {
  await chrome.action.setBadgeText({ text: '!' });
  await chrome.action.setIcon({ path:
    '/assets/icons/icon-alert-48.png' });
  setTimeout(async () => {
    await chrome.action.setBadgeText({ text: '' });
    await chrome.action.setIcon({ path:
      '/assets/icons/icon-48.png' });
  }, 6000);
  return;
}
const target = { tabId: tab.id };
try {
  await chrome.scripting.insertCSS({ target, files: [
    '/ui/tokens.css', '/ui/components.css'
  ] });
  await chrome.scripting.executeScript({ target, files: [
    '/shared/messaging.js',
    '/modules/download-guard/notify.js'
  ] });
  await chrome.tabs.sendMessage(tab.id, {
    module: 'downloadGuard',
    type: 'showBlocked',
    payload: { reason, name, url: item.finalUrl || item.url }
  });
} catch (e) { /* tab not injectable (chrome:// etc.) - badge already
  not an option; the download is still stopped */ }
}

async function stop(item, reason, name) {
  await chrome.downloads.cancel(item.id);
  await chrome.downloads.erase({ id: item.id });
  stoppedThisSession++;
  await SL.Storage.bumpStat('downloadGuard', 'INTERCEPTED');
  await SL.Storage.pushEvent({
    module: 'downloadGuard', verdict: 'INTERCEPTED',
    url: item.finalUrl || item.url, note: reason
  });
  const tabs = await chrome.tabs.query({ active: true,
    currentWindow: true });
  if (tabs[0]) {
    const map = await SL.Storage.sessionGet('sl.status', {});
    map[tabs[0].id] = Object.assign({}, map[tabs[0].id]);
    map[tabs[0].id].downloadGuard = stoppedThisSession +
      ' file' + (stoppedThisSession === 1 ? '' : 's') +
      ' stopped this session';
    await SL.Storage.sessionSet('sl.status', map);
  }
  await notifyUser(item, reason, name);
}

async function onCreated(item) {
  try {
    const host = item.referrer ? new URL(item.referrer).hostname : null;
    if (!(await moduleOn(host))) return; // settings gate: real OFF
    const name = nameOf(item);
    const settings = await SL.Settings.effective(host);
    const blockOnMismatch =
      settings.modules.downloadGuard.blockOnMismatch !== false;

    // 1. filename forensics - cheapest, no network
    if (DISGUISE_RE.test(name)) {
      await stop(item, 'The name hides a second extension - ' +
        'programs love to dress as documents.', name);
      return;
    }
    if (DANGEROUS_EXT.test(name)) {
      await stop(item, 'This file type is almost never legitimate ' +
        'when offered for download.', name);
      return;
    }

    // 2. magic-byte contradiction (documents claiming to be programs)
    if (blockOnMismatch && DOC_EXT.test(name)) {

```

```

const sniffed = await sniff(item.finalUrl || item.url);
if (sniffed.skipped) return;
for (const magic of EXECUTABLE_MAGIC) {
  if (startsWith(sniffed.bytes, magic.bytes)) {
    await stop(item, 'Named like a document, but its first ' +
      'bytes are a ' + magic.label + '.', name);
    return;
  }
}
}
// Bytes match category or nothing suspicious: silent pass.
} catch (err) {
  console.error('[sl:download-guard]', err);
}
}

chrome.downloads.onCreated.addListener(onCreated);

root.DownloadGuard = {
  onCreated, sniff, EXECUTABLE_MAGIC, DOCUMENT_MAGIC, DISGUISE_RE,
  DANGEROUS_EXT, DOC_EXT, SNIFF_BYTES
};
})(globalThis);

```

modules/download-guard/notify.js

```

/* modules/download-guard/notify.js - injected ON DEMAND into the tab
 * that started a blocked download. Shows the undoable toast. */
(function () {
  'use strict';
  if (!globalThis.SL || globalThis.SL.__dgNotify) return;
  globalThis.SL.__dgNotify = true;

  SL.msg.listen('downloadGuard', (msg) => {
    if (msg.type !== 'showBlocked') return null;
    const p = msg.payload || {};
    SLToast.show({
      kind: 'danger',
      title: 'Download stopped: ' + (p.name || 'file'),
      body: p.reason || 'The file contradicted its own name.',
      timeoutMs: 0,
      actions: [
        { label: 'Undo - download it anyway', primary: true,
          onClick: () => SL.send('downloadGuard', 'downloadAgain',
            { url: p.url, filename: p.name }) },
        { label: 'Keep it stopped', onClick: () => {} }
      ]
    });
    return null;
  });
})();
})();

```

7.3 Router route + wiring

Two edits: the router route that re-issues a stopped download, and the service worker's `importScripts` list gains the new file.

background/module-router.js - add after the formGuardian routes

```

register('downloadGuard', {
  /* Undo path for a stopped download: re-issue it verbatim. */
  async downloadAgain(payload) {
    const id = await chrome.downloads.download({
      url: payload.url,
      filename: payload.filename || undefined,
      saveAs: false
    });
    await SL.Storage.pushEvent({
      module: 'downloadGuard', verdict: null,
      url: payload.url, note: 'user continued stopped download'
    });
    return { id };
  }
});

```

```
background/service-worker.js - importScripts list gains one line
'/background/safe-mode.js',
'/background/download-guard.js'      // <- Part 7
);
```

7.4 Load & test

- Reload the extension. Trigger an ordinary download (any file from any site) and confirm it completes with zero interference and zero console output.
- The mismatch test needs a lying file. Easiest honest source: <https://httpbin.org/bytes/64> named doc.pdf via a link you create in DevTools: `var a=document.createElement('a');a.href='https://httpbin.org/bytes/64?x=1';a.setAttribute('download','report.pdf');a.textContent='Get report.pdf';document.body.prepend(a);` then click it. Random bytes are not MZ, so this passes silently - correct behavior. To see the stop, serve a real MZ file: run the Part 12 test server (`python3 test-pages/serve-tests.py`) and click its "invoice.pdf (actually a program)" link - the download is cancelled instantly and the red toast appears in the page with the Undo button.
- Double-extension test without any server: `var b=document.createElement('a');b.href='https://httpbin.org/bytes/8';b.setAttribute('download','photo.jpg.exe');b.textContent='photo.jpg.exe';document.body.prepend(b);` - the name alone triggers the stop and toast.
- Click **Undo - download it anyway** on any stopped download: the file downloads normally, and the event log (Part 11 dashboard) records that you overrode the stop.
- Flip Download Guard off and retry the double-extension link: the download proceeds untouched; flip it back on to restore the guard.

7.5 Tuning knobs

Knob	Where	Default	Effect
SNIFF_BYTES	background/download-guard.js	16	Range length fetched
DISGUISE_RE	background/download-guard.js	see code	Double-extension matcher
DANGEROUS_EXT	background/download-guard.js	6 exts	Name-only stop list
EXECUTABLE_MAGIC	background/download-guard.js	MZ, ELF	Byte prefixes that contradict doc names
blockOnMismatch	settings.modules.downloadGuard	true	Sniffing on/off

DEVIATION NOTE - Archive content inspection (a .zip whose contents are an .exe) is out of scope: MV3 service workers can fetch the archive but decompression-inspection of every download is a performance and false-positive minefield. The disguise patterns plus byte contradictions catch the overwhelming share of real-world attacks; the store listing says exactly that and no more.

Part 8 - Tab Guard

8.1 Design analysis

Popup storms are the denial-of-service attack against your attention: a page or its ad stack starts opening tab after tab, and by the time you reach for the close button you have six new tabs, three of which have already loaded autoplaying media. Tab Guard contains the storm in one move: it counts new tabs **by opener** inside a sliding window, and once one opener exceeds the threshold within the window, every additional tab it spawns is diverted into a collapsed tab group named "Held for review" - visible, countable, and silent, but not loading into your face. The group has a one-click Release that ungroups everything, and the first two tabs of any burst are always allowed through so that legitimate multi-tab flows (a site legitimately opening a login popup plus a callback tab) never get quarantined.

Containment, not blocking, is the deliberate design. Closing tabs that a page opened is destructive (one of them might be a payment handoff you actually wanted), and Chrome's own popup blocker already handles the fully-automated case; Tab Guard's value is the gray zone of **user-gesture-adjacent** storms that slip past it. A collapsed group neutralizes the storm's cost - attention and noise - while keeping every tab recoverable, which is the same reversibility contract every other module signs.

Pattern	Behavior	Why
A page opens its 1st and 2nd tabs in 10 s	allowed	Legitimate flows live here
3rd+ tabs from the same opener within the window	moved to 'Held for review' group	The storm signature
Storm tabs already in the group	stay grouped	No repeated moves, no flicker
Login popup + callback tab (2 tabs)	allowed	Under threshold
Group released by the user	all tabs ungrouped, window state reset	Explicit user intent
Module toggled off mid-storm	held tabs ungrouped automatically	OFF is real OFF

Chrome APIs used: `tabs.onCreated` (with `openerTabId`), `tabGroups` (create/update the isolation group), `tabs.group/tabs.ungroup`, `storage.session` for the window state. **Popup row:** "Tab Guard - 3 tabs held in the storm pen". **Options entries:** threshold (2-10, default 3), window length (5-60 s, default 10). **Toggle-off cleanup:** the isolation group, if this module created one, is ungrouped - held tabs return to normal - and the session window state is deleted; the listener (which lives in the SW) early-exits on every event thereafter.

8.2 Files

```
background/tab-guard.js
/* background/tab-guard.js - popup-storm containment via tab groups.
 * SW-only module: no content script. */
(function (global) {
  'use strict';
  const root = global.SL = global.SL || {};
  if (root.TabGuard) return;

  const STATE_KEY = 'sl.tabguard'; // session: {openerId -> timestamps[]}
  const GROUP_TITLE = 'Held for review';
  let cfg = { threshold: 3, windowMs: 10000 };
  let groupId = null; // our tab group id (chrome.tabGroups)

  async function moduleOn() {
    const s = await SL.Settings.get();
    return SL.Settings.isModuleOn(s, 'tabGuard', null);
  }
  async function loadState() {
```

```

    return SL.Storage.sessionGet(STATE_KEY, {});
}

async function saveState(state) {
    await SL.Storage.sessionSet(STATE_KEY, state);
}

function pruneTimestamps(stamps, now) {
    return (stamps || []).filter((t) => now - t < cfg.windowMs);
}

async function ensureGroup(windowId) {
    if (groupId !== null) return groupId;
    const created = await chrome.tabs.group({ createProperties: {
        windowId: windowId, titleState: 'EXPLICIT' }});
    groupId = created;
    try {
        await chrome.tabGroups.update(groupId, {
            title: GROUP_TITLE, collapsed: true, color: 'grey'
        });
    } catch (e) { /* tabGroups.update optional polish */ }
    return groupId;
}

async function releaseGroup() {
    if (groupId === null) return;
    const state = await loadState();
    try {
        const tabs = await chrome.tabs.query({ groupId });
        const ids = tabs.map((t) => t.id);
        if (ids.length) await chrome.tabs.ungroup(ids);
    } catch (e) { /* group already gone */ }
    groupId = null;
    await saveState({});
    SL.Storage.sessionSet(STATE_KEY, {});
}

async function onCreated(tab) {
    try {
        if (!(await moduleOn())) return; // real OFF: early exit
        const openerId = tab.openerTabId;
        if (typeof openerId !== 'number') return; // user-opened tab
        let opener;
        try { opener = await chrome.tabs.get(openerId); }
        catch (e) { return; } // opener already closed
        if (!opener.url || !/^https?:/i.test(opener.url)) return;

        const now = Date.now();
        const state = await loadState();
        state[openerId] = pruneTimestamps(state[openerId], now);
        state[openerId].push(now);
        await saveState(state);

        const count = state[openerId].length;
        if (count < cfg.threshold) return; // legit flow so far

        // Storm confirmed: divert this and every further tab.
        const g = await ensureGroup(tab.windowId);
        try {
            await chrome.tabs.group({ tabIds: [tab.id], groupId: g });
        } catch (e) { /* tab navigated away already - acceptable */ }
        await writeStatus(openerId, count);
    } catch (err) {
        console.error('[sl:tab-guard]', err);
    }
}

async function writeStatus(openerId, count) {
    let host = '';
    try { host = (await chrome.tabs.get(openerId)).url &&
        new URL((await chrome.tabs.get(openerId)).url).hostname; }
    catch (e) {}
    const map = await SL.Storage.sessionGet('sl.status', {});
    // Attribute the status line to the opener tab - that is the storm.

```

```

    map[openerId] = Object.assign({}, map[openerId]);
    map[openerId].tabGuard = count + ' tabs held from ' +
      (host || 'this page');
    await SL.Storage.sessionSet('sl.status', map);
  }

  async function onConfig(next) {
    cfg = Object.assign(cfg, next || {});
  }

  async function onDisabled() {
    await releaseGroup();
  }

  /* Popup / options control surface. */
  root.Router.register('tabGuard', {
    async releaseGroup() {
      await releaseGroup();
      return { released: true };
    },
    async heldCount() {
      if (groupId === null) return { count: 0 };
      const tabs = await chrome.tabs.query({ groupId });
      return { count: tabs.length, groupId };
    }
  });

  chrome.tabs.onCreated.addListener(onCreated);
  SL.Settings.subscribe(async (next) => {
    const on = SL.Settings.isModuleOn(next, 'tabGuard', null);
    const effective = (await SL.Settings.effective(null)).modules.tabGuard;
    Object.assign(cfg, effective);
    if (!on) await onDisabled();
  });

  root.TabGuard = { onCreated, releaseGroup, GROUP_TITLE };
})(globalThis);

```

background/service-worker.js - importScripts list gains one line

```

'/background/download-guard.js',    // <- Part 7
'/background/tab-guard.js'          // <- Part 8
);

```

8.3 Registry entry for Part 8

Tab Guard is SW-only, so its registry entry declares no scripts - the popup row and the settings lifecycle still work from the catalog data, and the toggle's effect is the SW listener's early-exit plus group release.

shared/module-registry.js - Tab Guard entry (replace placeholder)

```

{ id: 'tabGuard', name: 'Tab Guard', pillar: 'annoyance',
  blurb: 'Contains popup storms in an isolation group.',
  defaultEnabled: true, scripts: null },

```

8.4 Load & test

- Reload the extension; the importScripts list now loads tab-guard.js (check the SW console for syntax errors on load - a blank console is success).
- Open any page and run a controlled storm from its DevTools console: `for (var i = 0; i < 6; i++) window.open(location.href, '_blank');` - the first two tabs open normally; from the third onward they land in a collapsed grey group titled "Held for review" at the top of your tab strip.
- Click the group to expand it: all held tabs are there, intact and recoverable.
- From the popup, Part 11 wires a Release button; for now release from the SW console:
`SL.Router.HANDLERS.tabGuard.releaseGroup[0]()` is awkward to type, so simply drag tabs out of the group

manually or flip the module off - which ungroups everything automatically. That off-switch release is itself the cleanup test: flip Tab Guard off and watch the group dissolve.

- Confirm the two-tab allowance: `window.open(location.href, '_blank');`
`window.open(location.href, '_blank');` - both open ungrouped, no matter how quickly.

8.5 Tuning knobs

Knob	Where	Default	Effect
threshold	settings.modules.tabGuard	3	Tabs per opener before containment
windowMs	settings.modules.tabGuard	10000	Sliding window length
GROUP_TITLE	background/tab-guard.js	'Held for review'	Isolation group name
STATE_KEY	background/tab-guard.js	'sl.tabguard'	Session state key

WARNING - The sliding-window state lives in `chrome.storage.session`, which MV3 clears when the browser exits - exactly the lifecycle a popup-storm counter should have. Do not move it to `storage.local`: a stale count from yesterday's session quarantining today's first tab is a bug no user will forgive.

Part 9 - Cookie Tamer

9.1 Design analysis

Consent banners are a negotiation you did not ask to hold, usually designed so that "Accept All" is a big blue button and "Reject" is a grey text link two dialogs deep. Cookie Tamer's policy is the one that makes it a privacy tool rather than an auto-clicker: **it never clicks Accept, ever**. When it finds a consent banner it does one of two things. If the banner exposes an honest rejection control - a button whose own text says reject, refuse, decline, or "necessary cookies only" - Cookie Tamer clicks exactly that button, once, and verifies the banner actually left the DOM. If there is no honest rejection control, or the button does not dismiss the banner, the module **parks** the banner: it is hidden (reversibly) and replaced by a slim bar of our own saying the banner is parked, with a "Decide now" button that restores the original, scrolls to it, and steps aside so you can make the choice yourself.

The button classifier is the safety-critical piece, so it is deliberately paranoid. A button qualifies as a rejection only when its accessible text (plus aria-label and title) matches the rejection vocabulary and does **not** contain any accept-flavored word - "Accept necessary" passes the first test but contains "accept", so it is rejected by the second and the banner falls back to parking. Short affirmative labels like "OK" and "Got it" are classified as neither reject nor accept-safe: they are exactly the buttons consent dark patterns want automated clickers to press, so they never receive a click from us. Parking, not clicking, is always the safe default.

Banner situation	Action	Rationale
OneTrust banner with a visible 'Reject All' control	one click on Reject, verify removal	The good case: honest control
Banner whose only controls are 'Accept All' and 'OK'	park + slim bar	No honest reject exists
'Reject' button that fails to dismiss the banner	park + slim bar	Never click twice, never guess
Dark pattern: reject hidden behind 'Manage preferences' sub-dialog	park + slim bar	We do not walk multi-step mazes
No banner detected	silent	Nothing to negotiate

Banner situation	Action	Rationale
Login flow on the same site later	untouched	We only act on consent banners

The second capability is the **site sweep**, a deliberate, user-invoked action (never automatic, never scheduled): from the popup or dashboard you can sweep the current site's cookies down to the functional set. Login and session cookies are preserved by name heuristics (SID, SAPISID, __Secure-*, session, auth, csrf, ...) and by the structural rule that any HttpOnly + Secure cookie is almost certainly an authentication token and survives by default; analytics and ad cookies do not. **Chrome APIs used:** content-side none beyond the bridge; sweep-side **cookies** in the service worker. **Popup row:** "Cookie Tamer - 1 banner parked" / "standing by". **Options entries:** strategy (prefer-reject / park-only), sweep is exposed as a button in Part 11's popup. **Toggle-off cleanup:** parked banners are restored to exactly their prior inline styles, the slim bar is removed, observers detach, script id `sl-cs-cookie-tamer` unregisters.

9.2 Files

```
modules/cookie-tamer/cookie-tamer.js
/* modules/cookie-tamer/cookie-tamer.js - consent banner tamer.
 * POLICY: never clicks Accept. Clicks only an explicit reject control;
 * otherwise parks the banner behind a slim, reversible bar. */
(function () {
  'use strict';
  if (!globalThis.SL || !SL.__bridge) return;

  /* CMP fingerprints: one selector per provider family. */
  const CMP_SELECTORS = [
    '#onetrust-consent-sdk',          // OneTrust
    '#CybotCookieDialog',           // CookieBot
    'div[id^="didomi"], span[id^="didomi"]', // Didomi
    '#cmpbox', '#cmpwrapper',        // consentmanager
    '#cookie-law-info-bar',          // CookieYes / GDPR-era WP plugins
    '[data-testid="cookie-banner"]', // React apps
    '[class*="cookie-consent"]',
    '[id*="cookieBanner"]'
  ];

  const GENERIC_RE = /cookie|consent/i;

  const REJECT_RE = /reject all|reject|refuse|decline|decline all|necessary (cookies
  ↳ )?only|essential (cookies )?only|deny|opt[- ]?out/i;
  const ACCEPT_DANGER_RE = /accept all|accept|agree|allow all|consent|allow|enable/i;
  const AMBIGUOUS_OK_RE = /^(ok|okay|got it|gotcha|continue|close|dismiss|x|\u2715)$/i;

  const PARK_BAR_TEXT = 'Cookie banner parked - it is hidden, not answered.';

  let cfg = { strategy: 'prefer-reject' };
  let parked = []; // {banner, savedStyle, barEl}
  let scanTimer = 0;
  let observer = null;

  function findBanner() {
    for (const sel of CMP_SELECTORS) {
      try {
        const el = document.querySelector(sel);
        if (el && el.offsetHeight > 0) return el;
      } catch (e) { /* bad selector - skip */ }
    }
    // Generic fallback: fixed/absolute overlay mentioning cookies.
    for (const el of document.querySelectorAll(
      'div, section, aside, dialog')) {
      const style = window.getComputedStyle(el);
      if (style.position !== 'fixed' && style.position !== 'absolute') {
        continue;
      }
      const text = (el.id + ' ' + (el.className && String(el.className)) +
        ' ' + el.textContent.slice(0, 300)).trim();
      if (GENERIC_RE.test(text) && el.offsetHeight > 40 &&
        el.querySelectorAll('button, a[role="button"], input').length) {

```

```

    return el;
  }
}
return null;
}

function buttonLabels(banner) {
  const out = [];
  for (const b of banner.querySelectorAll(
    'button, a[role="button"], input[type="button"], input[type="submit"]')) {
    const label = ((b.getAttribute('aria-label') || '') + ' ' +
      (b.getAttribute('title') || '') + ' ' +
      (b.value || '') + ' ' + (b.textContent || '')).trim();
    if (label) out.push({ el: b, label: label.slice(0, 120) });
  }
  return out;
}

function findRejectButton(banner) {
  for (const { el, label } of buttonLabels(banner)) {
    if (AMBIGUOUS_OK_RE.test(label.trim())) continue; // never 'OK'
    if (ACCEPT_DANGER_RE.test(label)) continue; // never accept
    if (REJECT_RE.test(label)) return el;
  }
  return null;
}

function gone(el) {
  return !el || !el.isConnected || el.offsetHeight === 0;
}

async function tryReject(banner) {
  if (cfg.strategy !== 'prefer-reject') return false;
  const btn = findRejectButton(banner);
  if (!btn) return false;
  btn.click(); // exactly one click, ever
  await new Promise((r) => setTimeout(r, 450));
  if (gone(banner)) {
    SL.status('cookieTamer', 'rejected via the honest button');
    await SL.Storage.pushEvent({
      module: 'cookieTamer', verdict: null, url: location.href,
      note: 'clicked explicit reject control'
    });
    return true;
  }
  return false; // did not dismiss: park instead
}

function park(banner) {
  if (parked.some((p) => p.banner === banner)) return;
  const saved = banner.getAttribute('style');
  banner.style.setProperty('display', 'none', 'important');
  const bar = document.createElement('div');
  bar.className = 'sl-banner sl-root';
  const text = document.createElement('span');
  text.className = 'sl-banner__text';
  text.textContent = PARK_BAR_TEXT;
  const decide = document.createElement('button');
  decide.className = 'sl-btn';
  decide.type = 'button';
  decide.textContent = 'Decide now';
  decide.addEventListener('click', () => {
    unpark(banner);
    banner.scrollIntoView({ block: 'center',
      behavior: (window.matchMedia('(prefers-reduced-motion: reduce)')
        .matches ? 'auto' : 'smooth') });
  });
  const keep = document.createElement('button');
  keep.className = 'sl-btn';
  keep.type = 'button';
  keep.textContent = 'Keep hidden';
  keep.addEventListener('click', () => bar.remove());
  bar.appendChild(text);
  bar.appendChild(decide);

```

```

    bar.appendChild(keep);
    (document.body || document.documentElement).appendChild(bar);
    parked.push({ banner, saved, barEl: bar });
    SL.status('cookieTamer', '1 banner parked (Accept was never clicked)');
  }

  function unpark(banner) {
    const idx = parked.findIndex((p) => p.banner === banner);
    if (idx === -1) return;
    const { banner: el, saved, barEl } = parked[idx];
    parked.splice(idx, 1);
    if (saved === null) el.removeAttribute('style');
    else el.setAttribute('style', saved);
    if (barEl) barEl.remove();
  }

  function unparkAll() {
    for (const { banner } of parked.slice()) unpark(banner);
  }

  async function scan() {
    if (!document.body) return;
    const banner = findBanner();
    if (!banner) return;
    const ok = await tryReject(banner);
    if (!ok && !gone(banner)) park(banner);
  }

  function schedule() {
    clearTimeout(scanTimer);
    scanTimer = setTimeout(() => { scan(); schedule(); }, 2500);
  }

  function attach() {
    scan();
    schedule();
    observer = new MutationObserver(() => { clearTimeout(scanTimer);
      scanTimer = setTimeout(scan, 350); });
    observer.observe(document.documentElement || document.body, {
      childList: true, subtree: true
    });
    SL.status('cookieTamer', 'standing by');
  }

  function detach() {
    clearTimeout(scanTimer);
    if (observer) { observer.disconnect(); observer = null; }
    unparkAll();
    SL.status('cookieTamer', 'off');
  }

  SL.boot('cookieTamer', {
    onEnabled(cfg) { cfg = Object.assign(cfg, config); attach(); },
    onConfig(next) { cfg = Object.assign(cfg, next || {}); },
    onDisabled() { detach(); }
  });
  SL.registerCleanup('cookieTamer', detach);
})();

```

```
modules/cookie-tamer/cookie-tamer.css
```

```

/* modules/cookie-tamer/cookie-tamer.css
 * The parked bar uses .sl-banner from components.css. Nothing extra. */

```

9.3 Sweep route + registry entry

The cookie sweep runs in the service worker, where `chrome.cookies` lives. Paste the route block into `background/module-router.js` after the `downloadGuard` routes. Preservation is doubly conservative: cookies whose names match the login vocabulary survive, and **any** cookie that is both `HttpOnly` and `Secure` survives regardless of name, because that combination is the signature of an authentication token set by the server.

background/module-router.js - add after downloadGuard routes

```
/* Cookie Tamer: user-invoked sweep of the current site's cookies.
 * Login/session cookies survive by name AND by structure. */
const LOGIN_COOKIE_RE = /^(sid|hsid|ssid|apisid|apsid|lsid|__secure-.*|.*session.*|.*login.*|
↪ *auth.*|.*csrf.*|.*xsrif.*|.*remember.*)$/i;
register('cookieTamer', {
  async sweepCurrentSite(payload) {
    const tab = await chrome.tabs.get(payload.tabId);
    const host = new URL(tab.url).hostname;
    const all = await chrome.cookies.getAll({ domain: host });
    let kept = 0, removed = 0;
    for (const c of all) {
      const keep = LOGIN_COOKIE_RE.test(c.name) ||
        (c.httpOnly && c.secure);
      if (keep) { kept++; continue; }
      const url = (c.secure ? 'https://' : 'http://') +
        (c.domain.replace(/^\.\/, '')) + c.path;
      try {
        await chrome.cookies.remove({ url, name: c.name });
        removed++;
      } catch (e) { /* cookie already gone */ }
    }
    await SL.Storage.pushEvent({
      module: 'cookieTamer', verdict: null, url: tab.url,
      note: 'sweep: kept ' + kept + ', removed ' + removed
    });
    return { kept, removed };
  }
});
```

shared/module-registry.js - Cookie Tamer entry (replace placeholder)

```
{ id: 'cookieTamer', name: 'Cookie Tamer', pillar: 'annoyance',
  blurb: 'Parks consent banners; never clicks Accept.',
  defaultEnabled: true,
  scripts: [{
    id: 'sl-cs-cookie-tamer', matches: ['http://*', 'https://*'],
    runAt: 'document_idle', world: 'ISOLATED', allFrames: false,
    js: ['shared/messaging.js', 'shared/cs-bridge.js',
      'ui/toast.js', 'ui/verdict-card.js',
      'modules/cookie-tamer/cookie-tamer.js'],
    css: ['ui/tokens.css', 'ui/components.css',
      'modules/cookie-tamer/cookie-tamer.css']
  }] },
```

9.4 Load & test

- Reload the extension; confirm `sl-cs-cookie-tamer` registered.
- Visit two or three mainstream news sites. On those with OneTrust or CookieBot banners carrying visible "Reject All" controls, the banner should dismiss itself within a couple of seconds (one click, the honest one). On banners with only accept-flavored controls, the slim grey bar appears at the bottom instead.
- Click **Decide now** on the parked bar: the original banner reappears and scrolls into view, exactly as it was before parking.
- Test the sweep from the SW console against the active tab: `const t = (await chrome.tabs.query({active:true,currentWindow:true}))[0]; SL.Router.HANDLERS.cookieTamer.sweepCurrentSite[0]({tabId: t.id})` - expect {kept: N, removed: M} with your login cookies among the kept.
- Flip Cookie Tamer off while a banner is parked: the original banner reappears on every open tab (styles restored verbatim). Flip it on: the next scan parks it again.

9.5 Tuning knobs

Knob	Where	Default	Effect
CMP_SELECTORS	cookie-tamer.js	9 selectors	Provider fingerprints
REJECT_RE	cookie-tamer.js	see code	Honest-reject vocabulary
ACCEPT_DANGER_RE	cookie-tamer.js	see code	Veto vocabulary (never click)
AMBIGUOUS_OK_RE	cookie-tamer.js	7 labels	'OK'-class buttons never clicked
strategy	settings.modules.cookieTamer	'prefer-reject'	Click rejects or park only
LOGIN_COOKIE_RE	module-router.js	see code	Sweep preservation names

WARNING - The classifier's ordering is load-bearing: the accept-veto runs BEFORE the reject-match, so a button labeled "Accept necessary cookies only" is never clicked even though it contains "necessary only". If you extend either vocabulary, re-test that ordering with a banner containing such hybrid labels - this is the single line of defense between a privacy tool and an auto-consent machine.

Part 10 - Webcam/Mic Indicator

10.1 Design analysis

When a page uses your camera or microphone, Chrome already tells you - a tiny camera glyph on the tab itself. The problem is that the tab strip is the last place a focused user is looking. The indicator module makes the fact unmissable: from the moment a page's `getUserMedia` or `getDisplayMedia` succeeds until the last track ends, a chip sits top-right of the page - red pulsing dot, "Camera active" - and the toolbar icon for that tab switches to the camera variant with a REC badge. When the last track ends, the chip leaves without a trace.

The engineering is the interesting part, because page JavaScript cannot be observed from the isolated world: `navigator.mediaDevices` in the page's own world is a different object from the one content scripts see. The module therefore ships its only MAIN-world script, injected at `document_start` before any page code runs, which wraps `getUserMedia` and `getDisplayMedia` and reports activity across the world boundary via `window.postMessage` (`postMessage` events are visible to both worlds; the payload carries a source tag the isolated bridge verifies). The bridge never trusts the message beyond "which kind started or stopped" - no page data flows through it, only the words camera, mic and screen.

Event	Chip	Toolbar badge
Page calls <code>getUserMedia({video:true})</code> and it resolves	'Camera active' + red pulse	REC + camera icon
<code>getUserMedia({audio:true, video:true})</code>	'Camera + microphone active'	REC
<code>getDisplayMedia</code> (screen share)	'Screen sharing active'	REC
Last track ends (page stops or user revokes)	chip removed	cleared
Toggle module off mid-session	chip removed, future pages get no hook	cleared
Page already had permission granted before this module loaded	nothing detectable (see note)	-

Chrome APIs used: content-side none beyond the bridge; the MAIN-world file uses only native page APIs; the badge uses `chrome.action.setBadgeText/setIcon` per tab via a router route. **Popup row:** "Webcam/Mic Indicator - watching" / "CAMERA ACTIVE" when live. **Options entries:** chip position (top-right default), "also show while loading" (off - only granted streams). **Toggle-off cleanup:** the chip and badge clear immediately, and future pages never receive the hook; pages loaded while the module was on keep their (now unobserved) wrapper until reload - an honest limitation spelled out below.

DEVIATION NOTE - Two limitations are inherent and stated up front. First, camera/microphone use that began **before** this module loaded cannot be detected: the hook only sees calls made after injection (document_start of each navigation). Second, toggling the module off cannot un-hook the wrapper in already-loaded pages - the chip and badge go away, the page's `getUserMedia` wrapper remains inert until the page reloads. Both are consequences of how MAIN-world injection works in MV3, and both are documented in the README rather than hidden.

10.2 Files

modules/av-indicator/main-world.js

```
/* modules/av-indicator/main-world.js - runs in the page's MAIN world
 * at document_start, BEFORE any page script. Wraps getUserMedia and
 * getDisplayMedia, reports starts/stops via window.postMessage.
 * SECURITY: no page data crosses the boundary - only the words
 * camera / mic / screen and start / stop. */
(function () {
  'use strict';
  if (globalThis.__slAVHooked) return;
  globalThis.__slAVHooked = true;

  const TAG = 'secondlook-av';

  function post(event, media) {
    try {
      window.postMessage({ source: TAG, event: event, media: media },
        location.origin);
    } catch (e) { /* cross-origin about:blank edge - ignore */ }
  }

  function kindsFromConstraints(constraints) {
    const kinds = [];
    const c = constraints || {};
    if (c.video !== undefined && c.video !== false) kinds.push('camera');
    if (c.audio !== undefined && c.audio !== false) kinds.push('mic');
    return kinds;
  }

  function watchTracks(stream, kinds) {
    for (const track of stream.getTracks()) {
      const kind = track.kind === 'video'
        ? (kinds.includes('screen') ? 'screen' : 'camera') : 'mic';
      track.addEventListener('ended', () => post('stop', [kind]));
      // 'inactive' on the MediaStream itself means all tracks ended
    }
    stream.addEventListener('inactive', () => {
      for (const k of kinds) post('stop', [k]);
    });
  }

  const devices = navigator.mediaDevices;
  if (devices && devices.getUserMedia) {
    const origGUM = devices.getUserMedia.bind(devices);
    devices.getUserMedia = async function (constraints) {
      const stream = await origGUM(constraints);
      try {
        const kinds = kindsFromConstraints(constraints);
        post('start', kinds);
        watchTracks(stream, kinds);
      } catch (e) { /* never break the page on our accounting */ }
      return stream;
    };
  }

  if (devices && devices.getDisplayMedia) {
    const origGDM = devices.getDisplayMedia.bind(devices);
    devices.getDisplayMedia = async function (constraints) {
      const stream = await origGDM(constraints);
      try {
        post('start', ['screen']);
        watchTracks(stream, ['screen']);
      }
    };
  }
}
```

```

    } catch (e) { /* ignore */ }
    return stream;
  };
}
})();

```

modules/av-indicator/indicator.js

```

/* modules/av-indicator/indicator.js - ISOLATED world bridge.
 * Listens for the MAIN world's postMessage reports, renders the chip,
 * relays state to the service worker for the toolbar badge. */
(function () {
  'use strict';
  if (!globalThis.SL || !SL.__bridge) return;
  const TAG = 'secondlook-av';

  const active = new Set();    // 'camera' | 'mic' | 'screen'
  let chip = null;

  const LABELS = {
    camera: 'Camera', mic: 'microphone', screen: 'screen sharing'
  };

  function labelFor() {
    const parts = Array.from(active).map((k) => LABELS[k]);
    if (!parts.length) return '';
    return parts.length === 1 ? parts[0] + ' active'
      : parts.slice(0, -1).join(', ') + ' and ' +
        parts[parts.length - 1] + ' active';
  }

  function render() {
    if (!active.size) { removeChip(); return; }
    if (!chip) {
      chip = document.createElement('button');
      chip.type = 'button';
      chip.className = 'sl-chip sl-av-chip';
      const dot = document.createElement('span');
      dot.className = 'sl-chip__dot';
      chip.appendChild(dot);
      const text = document.createElement('span');
      text.className = 'sl-av-chip__text';
      chip.appendChild(text);
      chip.addEventListener('click', openCard);
      const mount = () => (document.body || document.documentElement)
        .appendChild(chip);
      if (document.body) mount();
      else document.addEventListener('DOMContentLoaded', mount, { once: true });
    }
    chip.querySelector('.sl-av-chip__text').textContent = labelFor();
    SL.status('avIndicator', labelFor().toUpperCase());
  }

  function removeChip() {
    if (chip) { chip.remove(); chip = null; }
    SL.status('avIndicator', 'watching');
  }

  function openCard() {
    SLVerdict.card({
      verdict: 'SECOND_LOOK',
      title: 'This page is using your hardware',
      reasons: [labelFor() + '. You can stop it from the camera icon ' +
        'in Chrome\'s address bar.'],
      rows: [
        { label: 'Page', value: location.hostname },
        { label: 'In use', value: labelFor() }
      ]
    });
  }

  /* Only tagged messages from our MAIN-world wrapper are accepted. */
  window.addEventListener('message', (e) => {
    if (e.source !== window) return;

```

```

const data = e.data;
if (!data || data.source !== TAG) return;
if (data.event === 'start') {
  for (const k of data.media || []) active.add(k);
} else if (data.event === 'stop') {
  for (const k of data.media || []) active.delete(k);
}
render();
SL.send('avIndicator', 'activeChange',
  { active: active.size > 0, kinds: Array.from(active) });
});

SL.boot('avIndicator', {
  onEnabled() {
    SL.status('avIndicator', 'watching');
  },
  onDisabled() { removeChip(); }
});
SL.registerCleanup('avIndicator', removeChip);
})();

```

modules/av-indicator/av-indicator.css

```

/* modules/av-indicator/av-indicator.css */
.sl-av-chip {
  border: 0;
  cursor: pointer;
  color: var(--sl-ink);
}
.sl-av-chip__text { font-weight: 600; }

```

10.3 Badge route + registry entry

The badge route lives in the router (paste after the cookieTamer block). Per-tab badges keep state naturally: Chrome scopes `setBadgeText({tabId})` to one tab, and the session map prunes itself on stops.

background/module-router.js - add after cookieTamer routes

```

register('avIndicator', {
  async activeChange(payload, sender) {
    const tabId = sender && sender.tab && sender.tab.id;
    if (typeof tabId !== 'number') return { ok: true };
    const map = await SL.Storage.sessionGet('sl.avtabs', {});
    if (payload.active) map[tabId] = true;
    else delete map[tabId];
    await SL.Storage.sessionSet('sl.avtabs', map);
    await chrome.action.setBadgeText({
      tabId, text: payload.active ? 'REC' : ''
    });
    if (payload.active) {
      await chrome.action.setIcon({
        tabId, path: '/assets/icons/icon-camera-48.png'
      });
    } else {
      await chrome.action.setIcon({
        tabId, path: '/assets/icons/icon-48.png'
      });
    }
    return { tabId, active: !!payload.active };
  }
});

```

shared/module-registry.js - AV Indicator entry (replace placeholder)

```

{ id: 'avIndicator', name: 'Webcam/Mic Indicator', pillar: 'annoyance',
  blurb: 'A chip whenever camera or mic is in use.',
  defaultEnabled: true,
  scripts: [
    { // 1) MAIN world hook, before any page script runs
      id: 'sl-cs-av-main', matches: ['http://*/*', 'https://*/*'],
      runAt: 'document_start', world: 'MAIN', allFrames: false,
      js: ['modules/av-indicator/main-world.js'], css: []
    },
  ],

```

```

{ // 2) ISOLATED world bridge + chip UI
  id: 'sl-cs-av-bridge', matches: ['http://*', 'https://*'],
  runAt: 'document_start', world: 'ISOLATED', allFrames: false,
  js: ['shared/messaging.js', 'shared/cs-bridge.js',
        'ui/toast.js', 'ui/verdict-card.js',
        'modules/av-indicator/indicator.js'],
  css: ['ui/tokens.css', 'ui/components.css',
        'modules/av-indicator/av-indicator.css']
}
] },

```

10.4 Load & test

- Reload the extension; the registration log should now list both `sl-cs-av-main` and `sl-cs-av-bridge`.
- Open any page that uses camera/mic demos (a "WebRTC sample" search finds plenty; <https://webrtc.github.io/samples/> is the canonical one) and click its "Start" button. Grant permission: the red-pulse chip appears top-right with "Camera active", and this extension's toolbar icon for that tab shows the camera variant with a REC badge.
- Click the page's "Stop" button: the chip disappears and the badge clears. If the page merely pauses the track rather than ending it, the chip correctly persists - only a real ended event clears it.
- Screen-share test with `getDisplayMedia`: the chip reads "screen sharing active".
- Click the chip: the card names the hardware in use and points to Chrome's own camera icon as the kill switch.
- Flip the module off while the chip is showing: the chip and badge vanish immediately. Reload the page and use the camera again: no chip (the hook never injected), which is the real-OFF guarantee for future loads.

10.5 Tuning knobs

Knob	Where	Default	Effect
TAG	main-world.js / indicator.js	'secondlook-av'	Cross-world message tag
LABELS	indicator.js	3 entries	Chip wording per kind
postMessage origin	main-world.js	location.origin	Boundary strictness

WARNING - The MAIN-world wrapper must never alter call semantics: it returns the original promise and never rejects on its own, and all of its own accounting is wrapped in try/catch so a failure inside the wrapper can never break the page's media call. If you ever extend this file, that invariant is the review gate.

Part 11 - Options, Dashboard & Popup Polish

11.1 Design analysis

Part 11 turns the settings surface from "the popup's ten toggles" into a real control room, and it exists to keep a promise made in every module part: that the knobs described there would be wired somewhere a user can reach. The options page is a two-pane layout - navigation on the left, one pane per concern on the right - with a single hard rule inherited from Part 0: **pages read settings directly but never write them**. Every control routes its change through the router (`core/setModuleOption`, `core/setSiteOverride`, ...), which keeps the service worker the single writer and the toggle lifecycle intact even when the edit comes from a full page instead of the popup.

The dashboard is the read-side counterpart: stat tiles for the three verdict counters, a module activity grid, the capped event log rendered as a table (hostnames only - the Part 0 privacy boundary), and the site-override manager where per-site

exceptions can be reviewed and lifted. The popup gets its final polish here: the Dashboard button wakes up, the site card gains "Show ghosts" (flashes neutralized overlays via a new router route) and "Sweep cookies" (the Part 9 sweep, user-invoked), and the chrome:// empty state stops pretending there is something to do.

11.2 Options page

ui/options.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>SecondLook Settings</title>
  <link rel="stylesheet" href="/ui/tokens.css">
  <link rel="stylesheet" href="/ui/components.css">
  <link rel="stylesheet" href="/ui/options.css">
</head>
<body class="sl-page sl-root">
  <div class="slo">
    <nav class="slo-nav" aria-label="Settings sections">
      <div class="slo-nav__brand">SecondLook</div>
      <button class="slo-nav__item" data-pane="general" type="button">
        General</button>
      <button class="slo-nav__item" data-pane="linkSniper" type="button">
        Link Sniper</button>
      <button class="slo-nav__item" data-pane="redirectDetective" type="button">
        Redirect Detective</button>
      <button class="slo-nav__item" data-pane="trustBadge" type="button">
        Trust Badge</button>
      <button class="slo-nav__item" data-pane="formGuardian" type="button">
        Form Guardian</button>
      <button class="slo-nav__item" data-pane="ghostClick" type="button">
        Ghost Click Blocker</button>
      <button class="slo-nav__item" data-pane="fakeDownload" type="button">
        Fake Download Button</button>
      <button class="slo-nav__item" data-pane="downloadGuard" type="button">
        Download Guard</button>
      <button class="slo-nav__item" data-pane="tabGuard" type="button">
        Tab Guard</button>
      <button class="slo-nav__item" data-pane="cookieTamer" type="button">
        Cookie Tamer</button>
      <button class="slo-nav__item" data-pane="avIndicator" type="button">
        Webcam / Mic Indicator</button>
      <button class="slo-nav__item" data-pane="sites" type="button">
        Site exceptions</button>
      <button class="slo-nav__item" data-pane="data" type="button">
        Your data</button>
    </nav>

    <main class="slo-main" id="slo-main"></main>
  </div>

  <script src="/shared/storage.js"></script>
  <script src="/shared/settings.js"></script>
  <script src="/shared/messaging.js"></script>
  <script src="/shared/module-registry.js"></script>
  <script src="/ui/toast.js"></script>
  <script src="/ui/verdict-card.js"></script>
  <script src="/ui/options.js"></script>
</body>
</html>
```

ui/options.css

```
/* ui/options.css - two-pane options layout. */
html, body { margin: 0; padding: 0; }
body { background: var(--sl-surface-2); min-height: 100vh; }

.slo { display: flex; min-height: 100vh; }
.slo-nav {
  width: 240px; flex: 0 0 auto;
  background: var(--sl-surface);
```

```

border-right: 1px solid var(--sl-line);
padding: var(--sl-s3) var(--sl-s2);
}
.slo-nav__brand {
  font-weight: 700; font-size: var(--sl-fz-title);
  padding: var(--sl-s2) var(--sl-s3) var(--sl-s4);
}
.slo-nav__item {
  display: block; width: 100%;
  text-align: left; border: 0; background: none;
  padding: var(--sl-s2) var(--sl-s3);
  border-radius: var(--sl-r-md);
  font-size: var(--sl-fz-base); color: var(--sl-ink-2);
  cursor: pointer;
}
.slo-nav__item:hover { background: var(--sl-surface-2); }
.slo-nav__item[aria-current="true"] {
  background: var(--sl-surface-2);
  color: var(--sl-ink); font-weight: 600;
}

.slo-main {
  flex: 1 1 auto; padding: var(--sl-s5);
  max-width: 720px;
}
.slo-pane h2 {
  font-size: var(--sl-fz-title); margin: 0 0 var(--sl-s2);
}
.slo-pane p {
  color: var(--sl-ink-2); font-size: var(--sl-fz-small);
  margin: 0 0 var(--sl-s3);
}
.slo-field {
  display: flex; align-items: center; gap: var(--sl-s3);
  padding: var(--sl-s3) 0;
  border-bottom: 1px solid var(--sl-line);
}
.slo-field__label { flex: 1 1 auto; }
.slo-field__name { font-weight: 600; }
.slo-field__hint {
  font-size: var(--sl-fz-tiny); color: var(--sl-ink-3);
  margin-top: 2px;
}
.slo-field input[type="range"] { width: 180px; }
.slo-field output {
  font-variant-numeric: tabular-nums;
  width: 64px; text-align: right;
}
select, textarea {
  font: inherit; border: 1px solid var(--sl-line);
  border-radius: var(--sl-r-md); padding: 4px 8px;
}
textarea { width: 100%; min-height: 140px; font-family: monospace;
  font-size: 12px; }
.slo-events { width: 100%; border-collapse: collapse; }
.slo-events th, .slo-events td {
  text-align: left; padding: 6px 8px;
  border-bottom: 1px solid var(--sl-line);
  font-size: var(--sl-fz-small);
}

```

ui/options.js

```

/* ui/options.js - declarative settings UI. Reads storage directly,
 * WRITES ONLY THROUGH THE ROUTER (single-writer rule). */
(function () {
  'use strict';
  const send = SL.msg.send;
  const main = document.getElementById('slo-main');
  let settings = null;

  async function refresh() { settings = await SL.Settings.get(); }

  function el(tag, cls, text) {

```

```

    const n = document.createElement(tag);
    if (cls) n.className = cls;
    if (text !== undefined) n.textContent = text;
    return n;
}

/* ---- field builders (all write via router) ----- */
function toggleField(name, hint, module, get, set) {
    const row = el('div', 'slo-field');
    const label = el('div', 'slo-field__label');
    label.appendChild(el('div', 'slo-field__name', name));
    if (hint) label.appendChild(el('div', 'slo-field__hint', hint));
    const wrap = el('label', 'sl-toggle');
    const input = document.createElement('input');
    input.type = 'checkbox';
    input.checked = get();
    input.addEventListener('change', () => set(input.checked));
    const track = el('span', 'sl-toggle__track');
    wrap.appendChild(input); wrap.appendChild(track);
    row.appendChild(label); row.appendChild(wrap);
    return row;
}

function rangeField(name, hint, module, option, min, max, step, value) {
    const row = el('div', 'slo-field');
    const label = el('div', 'slo-field__label');
    label.appendChild(el('div', 'slo-field__name', name));
    if (hint) label.appendChild(el('div', 'slo-field__hint', hint));
    const input = document.createElement('input');
    input.type = 'range';
    input.min = String(min); input.max = String(max);
    input.step = String(step || 1); input.value = String(value);
    const out = el('output', null, String(value));
    input.addEventListener('input', () => { out.textContent = input.value; });
    input.addEventListener('change', () => {
        send('core', 'setModuleOption', {
            moduleId: module, option: option,
            value: Number(input.value)
        });
    });
    row.appendChild(label); row.appendChild(input); row.appendChild(out);
    return row;
}

function selectField(name, hint, module, option, choices, value) {
    const row = el('div', 'slo-field');
    const label = el('div', 'slo-field__label');
    label.appendChild(el('div', 'slo-field__name', name));
    if (hint) label.appendChild(el('div', 'slo-field__hint', hint));
    const sel = document.createElement('select');
    for (const [val, txt] of choices) {
        const opt = el('option', null, txt);
        opt.value = val;
        sel.appendChild(opt);
    }
    sel.value = value;
    sel.addEventListener('change', () => {
        send('core', 'setModuleOption', {
            moduleId: module, option: option, value: sel.value
        });
    });
    row.appendChild(label); row.appendChild(sel);
    return row;
}

function actionField(name, hint, btnLabel, onClick, danger) {
    const row = el('div', 'slo-field');
    const label = el('div', 'slo-field__label');
    label.appendChild(el('div', 'slo-field__name', name));
    if (hint) label.appendChild(el('div', 'slo-field__hint', hint));
    const btn = el('button', 'sl-btn' + (danger ? ' sl-btn--danger' : ''),
        btnLabel);
    btn.type = 'button';
    btn.addEventListener('click', onClick);

```

```

    row.appendChild(label); row.appendChild(btn);
    return row;
}

function mod(id) { return settings.modules[id] || {}; }

/* ---- panes ----- */
function paneGeneral() {
    const pane = el('div', 'slo-pane');
    pane.appendChild(el('h2', null, 'General'));
    pane.appendChild(el('p', null,
        'The master switch physically unregisters every module script ' +
        'from every page - pausing SecondLook is a real off, not a mute.'));
    pane.appendChild(toggleField(
        'All modules enabled', 'Unregister everything when off',
        'core',
        () => settings.globalEnabled !== false,
        (v) => send('core', 'setGlobalEnabled', { enabled: v })));
    return pane;
}

function paneModule(id, blurb, fields) {
    const pane = el('div', 'slo-pane');
    const info = SL.Registry.byId(id);
    pane.appendChild(el('h2', null, (info && info.name) || id));
    pane.appendChild(el('p', null, blurb));
    pane.appendChild(toggleField(
        'Enabled', null, id,
        () => mod(id).enabled !== false,
        (v) => send('core', 'setModuleEnabled', { moduleId: id, enabled: v })));
    for (const f of fields) pane.appendChild(f);
    return pane;
}

function paneSites() {
    const pane = el('div', 'slo-pane');
    pane.appendChild(el('h2', null, 'Site exceptions'));
    pane.appendChild(el('p', null,
        'Per-site disables keep a module dormant on that host: no ' +
        'observers, no DOM footprint. Removing an exception restores ' +
        'default behavior.'));
    const list = el('div');
    const overrides = settings.siteOverrides || {};
    const hosts = Object.keys(overrides);
    if (!hosts.length) {
        list.appendChild(el('div', 'sl-empty', 'No site exceptions yet.'));
    }
    for (const host of hosts) {
        const site = overrides[host].modules || {};
        for (const [moduleId, enabled] of Object.entries(site)) {
            const info = SL.Registry.byId(moduleId);
            const row = el('div', 'slo-field');
            const label = el('div', 'slo-field__label');
            label.appendChild(el('div', 'slo-field__name',
                host + ' - ' + ((info && info.name) || moduleId)));
            label.appendChild(el('div', 'slo-field__hint',
                enabled ? 'forced ON for this site' :
                'dormant on this site (global settings untouched)'));
            const btn = el('button', 'sl-btn', 'Remove');
            btn.type = 'button';
            btn.addEventListener('click', () => {
                send('core', 'clearSiteOverride', { host, moduleId });
                setTimeout(render, 200);
            });
            row.appendChild(label); row.appendChild(btn);
            list.appendChild(row);
        }
    }
    pane.appendChild(list);
    return pane;
}

async function paneData() {
    const pane = el('div', 'slo-pane');

```

```

pane.appendChild(el('h2', null, 'Your data'));
pane.appendChild(el('p', null,
  'Everything SecondLook keeps lives in this browser. Event logs ' +
  'store hostnames only - never full URLs, never page content.'));
const stats = await send('core', 'getStats');
const s = (stats.ok && stats.data) || { byVerdict: {}, byModule: {} };
pane.appendChild(el('p', null,
  'Verdicts so far: ' + (s.byVerdict.CLEAR || 0) + ' clear, ' +
  (s.byVerdict.SECOND_LOOK || 0) + ' second looks, ' +
  (s.byVerdict.INTERCEPTED || 0) + ' interceptions.'));
pane.appendChild(actionField(
  'Reset counters and event log', null, 'Reset', async () => {
    await send('core', 'resetData');
    SLToast.show({ kind: 'ok', title: 'Local data cleared' });
    render();
  }, true));
pane.appendChild(actionField(
  'Export settings as JSON', null, 'Export', async () => {
    const res = await send('core', 'exportSettings');
    const ta = pane.querySelector('textarea');
    if (ta && res.ok) ta.value = res.data;
  }));
pane.appendChild(actionField(
  'Import settings from JSON below', null, 'Import', async () => {
    const ta = pane.querySelector('textarea');
    if (!ta || !ta.value.trim()) return;
    await send('core', 'importSettings', { json: ta.value });
    await refresh();
    render();
    SLToast.show({ kind: 'ok', title: 'Settings imported' });
  }));
const ta = document.createElement('textarea');
ta.setAttribute('aria-label', 'Settings JSON');
pane.appendChild(ta);
return pane;
}

const PANES = {
  general: () => paneGeneral(),
  linkSniper: () => paneModule('linkSniper',
    'Hover verdict pills on links. Judgments come from the shared ' +
    'engine; this page only tunes the experience.',
    [rangeField('Hover delay', 'Dwell before a pill appears (ms)',
      'linkSniper', 'hoverDelayMs', 150, 1000, 50,
      mod('linkSniper').hoverDelayMs || 350),
      toggleField('Show green pills', 'Display CLEAR verdicts too',
        'linkSniper', () => mod('linkSniper').showClear !== false,
        (v) => send('core', 'setModuleOption', {
          moduleId: 'linkSniper', option: 'showClear', value: v }))] ),
  redirectDetective: () => paneModule('redirectDetective',
    'Follows redirect chains and lists every hop. Resolution runs ' +
    'in the service worker with a strict time budget.',
    [rangeField('Maximum hops', 'Stop following after N hops',
      'redirectDetective', 'maxHops', 3, 12, 1,
      mod('redirectDetective').maxHops || 8)] ),
  trustBadge: () => paneModule('trustBadge',
    'Flags trust seals that are just images or that link ' +
    'somewhere other than their claimed issuer. Vocabulary and ' +
    'issuer domains are code-curated for precision.', []),
  formGuardian: () => paneModule('formGuardian',
    'Intercepts password and card submits whose destination origin ' +
    'differs from the page. Approved destinations are stored as ' +
    'salted hashes, never in the clear.',
    [toggleField('Remember approved destinations',
      'Silently allow destinations you already continued to',
      'formGuardian',
      () => mod('formGuardian').rememberApproved !== false,
      (v) => send('core', 'setModuleOption', {
        moduleId: 'formGuardian', option: 'rememberApproved', value: v })),
      actionField('Forget all approved destinations',
        'Clears the salted-hash memory list', 'Forget', async () => {
          await send('formGuardian', 'clearApproved');
          SLToast.show({ kind: 'ok', title: 'Approval memory cleared' });
        }, true)] ),

```

```

ghostClick: () => paneModule('ghostClick',
  'Sets pointer-events:none on provably invisible full-viewport ' +
  'overlays. Never removes elements; undo is exact.',
  [toggleField('Annotate neutralized ghosts',
    'Keep a dashed outline on neutralized overlays',
    'ghostClick', () => mod('ghostClick').annotate === true,
    (v) => send('core', 'setModuleOption', {
      moduleId: 'ghostClick', option: 'annotate', value: v }))),
fakeDownload: () => paneModule('fakeDownload',
  'Marks decoy download buttons: clusters, ad-zone placement, ' +
  'dead links. Annotation only - Download Guard inspects the file.',
  [toggleField('Mark the primary button too',
    'Attach a green check to the button that looks real',
    'fakeDownload', () => mod('fakeDownload').markPrimary !== false,
    (v) => send('core', 'setModuleOption', {
      moduleId: 'fakeDownload', option: 'markPrimary', value: v }))),
downloadGuard: () => paneModule('downloadGuard',
  'Fetches the first 16 bytes of each download and contradicts ' +
  'the name when the bytes disagree. Every stop is undoable.',
  [toggleField('Block on byte mismatch',
    'Stop documents whose bytes are actually programs',
    'downloadGuard',
    () => mod('downloadGuard').blockOnMismatch !== false,
    (v) => send('core', 'setModuleOption', {
      moduleId: 'downloadGuard', option: 'blockOnMismatch',
      value: v }))),
tabGuard: () => paneModule('tabGuard',
  'Diverts popup storms into a collapsed "Held for review" tab ' +
  'group. Containment, never deletion.',
  [rangeField('Tabs per opener', 'Containment threshold',
    'tabGuard', 'threshold', 2, 10, 1,
    mod('tabGuard').threshold || 3),
  rangeField('Window (ms)', 'Sliding window length',
    'tabGuard', 'windowMs', 5000, 60000, 5000,
    mod('tabGuard').windowMs || 10000)],
cookieTamer: () => paneModule('cookieTamer',
  'Never clicks Accept. Clicks only explicit reject controls; ' +
  'otherwise parks the banner behind a reversible bar.',
  [selectField('Strategy', null, 'cookieTamer', 'strategy',
    [['prefer-reject', 'Click honest reject buttons, else park'],
    ['park-only', 'Always park the banner']],
    mod('cookieTamer').strategy || 'prefer-reject'))],
avIndicator: () => paneModule('avIndicator',
  'A chip whenever a page uses camera, microphone or screen ' +
  'sharing. Only starts and stops cross the world boundary - ' +
  'never page data.', []),
sites: () => paneSites(),
data: () => paneData()
};

async function render(paneId) {
  await refresh();
  const id = paneId || (location.hash.slice(1) || 'general');
  const builder = PANES[id] || PANES.general;
  main.textContent = '';
  main.appendChild(await builder());
  for (const btn of document.querySelectorAll('.slo-nav__item')) {
    btn.setAttribute('aria-current',
      String(btn.dataset.pane === id));
    btn.onclick = () => {
      location.hash = btn.dataset.pane;
      render(btn.dataset.pane);
    };
  }
}

SL.Settings.subscribe(() => render());
render();
})();

```

11.3 Dashboard

ui/dashboard.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>SecondLook Dashboard</title>
  <link rel="stylesheet" href="/ui/tokens.css">
  <link rel="stylesheet" href="/ui/components.css">
  <link rel="stylesheet" href="/ui/dashboard.css">
</head>
<body class="sl-page sl-root">
  <div class="sld">
    <header class="sld-header">
      <h1>SecondLook</h1>
      <span class="sld-sub">Everything on this page lives in this
        browser only.</span>
    </header>

    <section class="sld-tiles" id="sld-tiles" aria-label="Verdict counts">
    </section>

    <section class="sld-modules">
      <h2>Module activity</h2>
      <div id="sld-modules" class="sld-module-grid"></div>
    </section>

    <section class="sld-events">
      <h2>Recent events <span class="sld-note">(hostnames only - never
        full URLs)</span></h2>
      <table class="sld-table" id="sld-events"></table>
    </section>

    <section class="sld-actions">
      <button class="sl-btn" id="sld-release" type="button">
        Release held tabs</button>
      <button class="sl-btn sl-btn--danger" id="sld-reset" type="button">
        Reset stats and events</button>
      <button class="sl-btn" id="sld-options" type="button">
        Open settings</button>
    </section>
  </div>

  <script src="/shared/storage.js"></script>
  <script src="/shared/settings.js"></script>
  <script src="/shared/messaging.js"></script>
  <script src="/shared/module-registry.js"></script>
  <script src="/ui/toast.js"></script>
  <script src="/ui/verdict-card.js"></script>
  <script src="/ui/dashboard.js"></script>
</body>
</html>
```

ui/dashboard.css

```
/* ui/dashboard.css - dashboard layout. */
html, body { margin: 0; padding: 0; }
body { background: var(--sl-surface-2); }

.sld { max-width: 860px; margin: 0 auto; padding: var(--sl-s5); }
.sld-header h1 { margin: 0; font-size: 20px; }
.sld-sub { color: var(--sl-ink-3); font-size: var(--sl-fz-small); }
.sld h2 { font-size: var(--sl-fz-title); margin: var(--sl-s5) 0
  var(--sl-s2); }
.sld-note { color: var(--sl-ink-3); font-weight: 400;
  font-size: var(--sl-fz-tiny); }

.sld-tiles { display: flex; gap: var(--sl-s3); margin-top: var(--sl-s4); }
.sld-module-grid {
  display: flex; flex-wrap: wrap; gap: var(--sl-s3);
}
.sld-tile { flex: 1 1 120px; }
```

```

.sld-tile--wide { flex: 1 1 220px; }
.sld-tile__module { font-size: var(--sl-fz-small); font-weight: 600; }
.sld-tile__count { font-size: 20px; font-weight: 700; }

.sld-table { width: 100%; border-collapse: collapse; background:
  var(--sl-surface); border-radius: var(--sl-r-md); }
.sld-table th, .sld-table td {
  padding: 7px 10px; font-size: var(--sl-fz-small);
  border-bottom: 1px solid var(--sl-line); text-align: left;
}
.sld-table th { color: var(--sl-ink-3); font-weight: 600; }

.sld-actions { display: flex; gap: var(--sl-s2); margin-top: var(--sl-s5); }

```

ui/dashboard.js

```

/* ui/dashboard.js - read-only control room. Reads storage directly;
 * the few mutations go through the router. */
(function () {
  'use strict';
  const send = SL.msg.send;

  function fmtTime(ts) {
    const d = new Date(ts);
    return d.toLocaleDateString() + ' ' + d.toLocaleTimeString();
  }

  function tile(num, label, extraClass) {
    const t = document.createElement('div');
    t.className = 'sl-stat-tile sld-tile ' + (extraClass || '');
    const n = document.createElement('div');
    n.className = 'sl-stat-tile__num';
    n.textContent = String(num);
    const l = document.createElement('div');
    l.className = 'sl-stat-tile__label';
    l.textContent = label;
    t.appendChild(n); t.appendChild(l);
    return t;
  }

  async function renderTiles() {
    const holder = document.getElementById('sld-tiles');
    holder.textContent = '';
    const res = await send('core', 'getStats');
    const s = (res.ok && res.data) || { byVerdict: {}, byModule: {} };
    holder.appendChild(tile(s.byVerdict.CLEAR || 0, 'clear verdicts'));
    holder.appendChild(tile(s.byVerdict.SECOND_LOOK || 0,
      'second looks'));
    holder.appendChild(tile(s.byVerdict.INTERCEPTED || 0,
      'interceptions'));
  }

  function renderModules(stats) {
    const holder = document.getElementById('sld-modules');
    holder.textContent = '';
    for (const mod of SL.Registry.MODULES) {
      if (mod.devOnly) continue;
      const t = document.createElement('div');
      t.className = 'sl-stat-tile sld-tile sld-tile--wide';
      const m = document.createElement('div');
      m.className = 'sld-tile__module';
      m.textContent = mod.name;
      const c = document.createElement('div');
      c.className = 'sld-tile__count';
      c.textContent = String((stats.byModule &&
        stats.byModule[mod.id]) || 0);
      const l = document.createElement('div');
      l.className = 'sl-stat-tile__label';
      l.textContent = 'engine events';
      t.appendChild(m); t.appendChild(c); t.appendChild(l);
      holder.appendChild(t);
    }
  }

  async function renderEvents() {

```

```

const tbl = document.getElementById('sld-events');
tbl.textContent = '';
const res = await send('core', 'getEvents', { limit: 60 });
const events = (res.ok && res.data) || [];
const thead = document.createElement('thead');
const hrow = document.createElement('tr');
for (const h of ['When', 'Module', 'Verdict', 'Host', 'Note']) {
  const th = document.createElement('th');
  th.textContent = h;
  hrow.appendChild(th);
}
thead.appendChild(hrow);
tbl.appendChild(thead);
const tbody = document.createElement('tbody');
if (!events.length) {
  const row = document.createElement('tr');
  const td = document.createElement('td');
  td.colSpan = 5;
  td.textContent = 'Nothing logged yet - browse a little.';
  row.appendChild(td);
  tbody.appendChild(row);
}
for (const e of events) {
  const row = document.createElement('tr');
  for (const v of [fmtTime(e.ts), e.module,
    e.verdict || '-', e.host || '-', e.note || '']) {
    const td = document.createElement('td');
    td.textContent = v;
    row.appendChild(td);
  }
  tbody.appendChild(row);
}
tbl.appendChild(tbody);
return (await send('core', 'getStats')).data;
}

document.getElementById('sld-release').onclick = async () => {
  const r = await send('tabGuard', 'releaseGroup');
  SLToast.show({
    kind: 'ok', title: 'Held tabs released',
    body: r.ok ? 'The storm pen is empty.' : 'Nothing was held.'
  });
};
document.getElementById('sld-reset').onclick = async () => {
  await send('core', 'resetData');
  await renderAll();
};
document.getElementById('sld-options').onclick = () => {
  chrome.runtime.openOptionsPage();
};

async function renderAll() {
  await renderTiles();
  const stats = await renderEvents();
  renderModules(stats || {});
}

renderAll();
setInterval(renderAll, 15000); // gentle live refresh
})();

```

11.4 Popup polish + two new router routes

The popup's final form enables the Dashboard button and adds two per-site actions to the site card. **Show ghosts** asks the service worker to message the tab, which flashes every neutralized overlay for 2.5 seconds (Part 5's listener does the flashing). **Sweep cookies** invokes Part 9's sweep for the current tab and toasts the kept/removed counts. Both are one route away - paste the block into the router first, then replace `ui/popup.js` wholesale with the version below.

background/module-router.js - add after avIndicator routes

```
register('core', {
  /* Popup: flash neutralized ghost overlays in one tab. */
  async revealGhosts(payload) {
    await chrome.tabs.sendMessage(payload.tabId, {
      module: 'ghostClick', type: 'flashOverlays'
    });
    return { sent: true };
  },
  /* Popup: user-invoked cookie sweep for the current site. */
  async sweepCookies(payload) {
    return (await HANDLERS.cookieTamer.sweepCurrentSite[0](payload));
  }
});
```

ui/popup.js - final version (replace Part 0's file)

```
/* ui/popup.js - final popup: pillars, live rows, site actions. */
(function () {
  'use strict';
  const send = SL.msg.send;
  const $ = (id) => document.getElementById(id);

  const state = { tabId: null, snapshot: null, settings: null };

  function eyeIcon() {
    return '<svg viewBox="0 0 24 24" width="16" height="16" ' +
      'aria-hidden="true"><path d="M2 12s3.5-6 10-6 10 6-3.5 ' +
      '6-10 6s2 12 2 12z" fill="none" stroke="currentColor" ' +
      'stroke-width="1.8"/><circle cx="12" cy="12" r="2.6" ' +
      'fill="currentColor"/></svg>';
  }

  function iconFor(id) {
    const paths = {
      demoGreeting: '', linkSniper: '',
      redirectDetective: 'M9 15l6-6M8 12l-2 2a3 3 0 004 4l2-2' +
        'M16 12l2-2a3 3 0 00-4-4l-2 2',
      trustBadge: 'M12 3l2.5 2 3 .5.5 3L20 12l-2 2.5-.5 3-3 .5' +
        '-2.5 2-2.5-2-3-.5-.5-3L4 12l2-2.5.5-3 3-.5zM9 12l2 2 4-4',
      formGuardian: 'M4 6h16v12H4zM7 10h10M7 14h6',
      ghostClick: 'M4 4h16v16H4z',
      fakeDownload: 'M12 4v10m0 0l-4-4m4 4l4-4M5 19h14',
      downloadGuard: 'M12 3l7 3v6c0 4-3 7-7 9-4-2-7-5-7-9V6z' +
        'M12 9v4',
      tabGuard: 'M3 7h18v13H3zM3 7l3-4h8l3 4',
      cookieTamer: 'M12 4a8 8 0 100 16 8 8 0 000-16z' +
        'M10 10h.01M14 13h.01M9 15h.01',
      avIndicator: 'M3 7h12v10H3zM15 11l6-3v8l-6-3z'
    };
    return '<svg viewBox="0 0 24 24" width="16" height="16" ' +
      'fill="none" stroke="currentColor" stroke-width="1.8" ' +
      'aria-hidden="true">' +
      (paths[id] === '' ? eyeIcon().replace(/<\/?svg[>]*>/g, '')
      : '<path d="' + (paths[id] || '') + '" />') +
      (paths[id] === '' ? '' : '') + '</svg>';
  }

  async function init() {
    const [tab] = await chrome.tabs.query({ active: true,
      currentWindow: true });
    state.tabId = tab ? tab.id : null;
    const res = await send('core', 'getSnapshot', { tabId: state.tabId });
    state.snapshot = res.ok ? res.data : null;
    state.settings = state.snapshot && state.snapshot.settings;
    renderSite();
    renderPillars();
    renderGlobal();
    chrome.storage.onChanged.addListener(onStorage);
  }

  function onStorage(changes, area) {
    if (area === 'local' && changes['sl.settings']) {
      state.settings = changes['sl.settings'].newValue;
      renderPillars();
    }
  }
});
```

```

    renderGlobal();
  }
  if (area === 'session' && changes['sl.status']) {
    if (state.snapshot) {
      state.snapshot.statusLines =
        (changes['sl.status'].newValue || {})[state.tabId] || {};
    }
    renderPillars();
  }
}

function renderGlobal() {
  const on = !(state.settings && state.settings.globalEnabled !== false);
  $('slp-global').checked = on;
  $('slp-paused').hidden = on;
}

function renderSite() {
  const snap = state.snapshot;
  const isHttp = snap && /^https?:/i.test(snap.url || '');
  $('slp-site').hidden = !isHttp;
  $('slp-empty').hidden = !!isHttp;
  if (!isHttp) return;
  $('slp-host').textContent = snap.host || '(this page)';
  const holder = $('slp-verdict');
  holder.textContent = '';
  if (snap.pageVerdict) {
    const v = snap.pageVerdict;
    const pill = SLVerdict.pill(v.verdict,
      v.verdict === 'CLEAR' ? 'OK' : 'Take a look');
    if (v.reasons && v.reasons.length) {
      pill.title = v.reasons.join(' ');
    }
    holder.appendChild(pill);
  }
  updateSafeModeButton();
}

function updateSafeModeButton() {
  const btn = $('slp-safemode');
  const on = !(state.snapshot && state.snapshot.safeModeOn);
  btn.textContent = on ? 'Safe Mode is ON - unfreeze' : 'Safe Mode this tab';
  btn.classList.toggle('sl-btn--primary', on);
}

function renderPillars() {
  const nav = $('slp-pillars');
  nav.textContent = '';
  SL.Registry.PILLARS.forEach((pillar, idx) => {
    const section = document.createElement('section');
    section.className = 'sl-pillar';
    section.dataset.collapsed = idx > 0 ? 'true' : 'false';
    const head = document.createElement('button');
    head.className = 'slp-pillar__head';
    head.type = 'button';
    head.setAttribute('aria-expanded', idx > 0 ? 'false' : 'true');
    head.innerHTML = '<span>' + pillar.name + '</span>' +
      '<svg class="slp-pillar__chev" viewBox="0 0 24 24" width="12" ' +
      'height="12" aria-hidden="true"><path d="M6 9l6 6 6-6" ' +
      'fill="none" stroke="currentColor" stroke-width="2"/></svg>';
    head.addEventListener('click', () => {
      const now = section.dataset.collapsed !== 'true';
      section.dataset.collapsed = String(now);
      head.setAttribute('aria-expanded', String(!now));
    });
    section.appendChild(head);
    const rows = document.createElement('div');
    rows.className = 'slp-pillar__rows';
    for (const mod of SL.Registry.MODULES) {
      if (mod.pillar !== pillar.id) continue;
      rows.appendChild(makeRow(mod));
    }
    section.appendChild(rows);
    nav.appendChild(section);
  });
}

```

```

    });
}

function makeRow(mod) {
    const row = document.createElement('div');
    row.className = 'sl-row';
    row.dataset.module = mod.id;
    row.title = mod.blurb;

    const icon = document.createElement('div');
    icon.className = 'sl-row__icon';
    icon.innerHTML = iconFor(mod.id);
    row.appendChild(icon);

    const body = document.createElement('div');
    body.className = 'sl-row__body';
    const name = document.createElement('div');
    name.className = 'sl-row__name';
    name.textContent = mod.name;
    if (mod.devOnly) {
        const tag = document.createElement('span');
        tag.className = 'sl-row__dev-tag';
        tag.textContent = 'DEV';
        tag.style.marginLeft = '6px';
        name.appendChild(tag);
    }
    body.appendChild(name);
    const status = document.createElement('div');
    status.className = 'sl-row__status';
    status.textContent = statusText(mod.id);
    body.appendChild(status);
    row.appendChild(body);

    const toggle = document.createElement('label');
    toggle.className = 'sl-toggle';
    const input = document.createElement('input');
    input.type = 'checkbox';
    input.setAttribute('aria-label', mod.name + ' on this browser');
    input.checked = isOn(mod.id);
    input.addEventListener('change', () =>
        onToggle(mod.id, input.checked, row));
    const track = document.createElement('span');
    track.className = 'sl-toggle__track';
    toggle.appendChild(input);
    toggle.appendChild(track);
    row.appendChild(toggle);
    return row;
}

function isOn(moduleId) {
    return !(state.settings &&
        SL.Settings.isModuleOn(state.settings, moduleId, null));
}

function statusText(moduleId) {
    const lines = state.snapshot && state.snapshot.statusLines;
    const line = lines && lines[moduleId];
    if (line) return line;
    return isOn(moduleId) ? 'Idle' : 'Off';
}

async function onToggle(moduleId, enabled, rowEl) {
    rowEl.classList.add('sl-row--syncing');
    const statusEl = rowEl.querySelector('.sl-row__status');
    if (statusEl) statusEl.textContent = 'syncing';
    await send('core', 'setModuleEnabled', { moduleId, enabled });
}

$('slp-global').addEventListener('change', async (e) => {
    await send('core', 'setGlobalEnabled', { enabled: e.target.checked });
});

$('slp-safemode').addEventListener('click', async () => {
    const on = !(state.snapshot && state.snapshot.safeModeOn);

```

```

    await send('core', 'safeMode', { tabId: state.tabId, on });
    state.snapshot.safeModeOn = on;
    renderSite();
  });

$('slp-ghosts').addEventListener('click', async () => {
  const res = await send('core', 'revealGhosts', { tabId: state.tabId });
  SLToast.show({
    kind: res.ok ? 'ok' : 'warn',
    title: res.ok ? 'Neutralized overlays flashed in the page'
      : 'No page to flash',
    body: res.ok ? 'Look for the amber outlines.' :
      'Open a normal http(s) page first.',
    timeoutMs: 4000
  });
});

$('slp-sweep').addEventListener('click', async () => {
  const res = await send('core', 'sweepCookies', { tabId: state.tabId });
  if (res.ok) {
    SLToast.show({
      kind: 'ok', title: 'Cookie sweep complete',
      body: 'Kept ' + res.data.kept + ' (login/session), removed ' +
        res.data.removed + '.'
    });
  } else {
    SLToast.show({ kind: 'warn', title: 'Sweep unavailable',
      body: 'Open a normal http(s) site first.' });
  }
});

$('slp-options').addEventListener('click', () => {
  chrome.runtime.openOptionsPage();
});
$('slp-dashboard').addEventListener('click', () => {
  chrome.tabs.create({ url: chrome.runtime.getURL('/ui/dashboard.html') });
});

init();
})();

```

ui/popup.html - two buttons added to the site section (replace it)

```

<section id="slp-site" class="slp-site" hidden>
  <div class="slp-site__row">
    <div class="slp-site__host" id="slp-host"></div>
    <span id="slp-verdict"></span>
  </div>
  <div class="slp-site__actions">
    <button id="slp-safemode" class="sl-btn" type="button">
      Safe Mode this tab</button>
    <button id="slp-ghosts" class="sl-btn" type="button">
      Show ghosts</button>
    <button id="slp-sweep" class="sl-btn" type="button">
      Sweep cookies</button>
  </div>
</section>

```

ui/popup.css - add the actions row (append)

```

.slp-site__actions {
  display: flex;
  gap: var(--sl-s2);
  flex-wrap: wrap;
}

```

11.5 Load & test

- Reload the extension. Right-click the toolbar icon and open Options: every pane should render current values; flip Link Sniper's hover delay and confirm the popup status cadence changes on your next page hover.

- Add a site exception from the console of any site: run `SL.msg.send('core', 'setSiteOverride', {host:location.hostname,moduleId:'ghostClick',enabled:false})` in the page console - then check the Site exceptions pane lists it, remove it, and confirm the module wakes up again.
- Open the Dashboard from the popup. Stat tiles show the counts accumulated during Parts 1-10 testing; the events table lists hostnames with notes; Release held tabs empties any leftover storm pen.
- On a page where Part 5 neutralized ghosts, click **Show ghosts** in the popup: amber outlines flash in the page for 2.5 seconds.
- On any logged-in site, click **Sweep cookies**: the toast reports kept vs removed counts, and reloading the page keeps you logged in (that is the whole point of the preservation rules).

11.6 Tuning knobs introduced in Part 11

Knob	Where	Default	Effect
dashboard refresh	ui/dashboard.js	15 s	Live refresh cadence
getEvents limit	router core route	50-60	Event page size

Part 12 - Test Pages & Local Test Server

12.1 Design analysis

Every module in this guide was verified against pages that deliberately exhibit the hostile pattern - and now you get the same bench. The kit is one small Python server plus seven static pages. The server exists because two behaviors cannot be faked with static files: real HTTP 302 redirect chains (Redirect Detective needs genuine `Location` hops) and files whose bytes contradict their names (Download Guard needs a MZ-byte file named `.pdf`). `serve-tests.py` provides both, plus a POST sink for Form Guardian, and binds on `0.0.0.0:8000` so `localhost:8000` and `127.0.0.1:8000` are **different origins** - which is exactly what the fake-login page needs to demo cross-origin credential submission without a second server.

Run it from the extension root with `python3 test-pages/serve-tests.py`, then browse `http://localhost:8000/` for the index. Content scripts match `http://*/*`, so localhost is fully covered by every module with zero configuration.

12.2 The test server

```
test-pages/serve-tests.py
#!/usr/bin/env python3
"""test-pages/serve-tests.py - local test bench for SecondLook.

Serves ./test-pages on 0.0.0.0:8000 with:
/hop/1 -> 302 -> /hop/2 -> 302 -> /hop/3    (real redirect chain)
/sink                                         (POST echo, cross-origin target)
/files/dummy.pdf                           (real PDF magic bytes)
/files/evil-invoice.pdf                     (MZ bytes, PDF name)
/files/plain.txt                             (honest file)
"""
import os
import socketserver
from http.server import SimpleHTTPRequestHandler

HERE = os.path.dirname(os.path.abspath(__file__))
PDF = b"%PDF-1.4\n1 0 obj<</Type/Catalog/Pages 2 0 R>>endobj\n" \
      b"2 0 obj<</Type/Pages/Kids[3 0 R]>>endobj\n" \
      b"3 0 obj<</Type/Page/Parent 2 0 R>>endobj\ntrailer<</Root 1 0 R>>\n"
MZ = b"MZ\x90\x00\x03\x00\x00\x00\x04\x00\x00\x00\xff\xff\x00\x00" \
     b"This is a Windows program wearing a PDF costume. " * 8
class Handler(SimpleHTTPRequestHandler):
```

```

def __init__(self, *a, **kw):
    super().__init__(*a, directory=HERE, **kw)

def do_GET(self):
    if self.path == "/hop/1":
        self.send_response(302)
        self.send_header("Location", "/hop/2")
        self.end_headers()
    elif self.path == "/hop/2":
        self.send_response(302)
        self.send_header("Location", "/hop/3")
        self.end_headers()
    elif self.path == "/hop/3":
        body = b"final destination - no more redirects"
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    elif self.path == "/files/dummy.pdf":
        self._bytes(PDF, "application/pdf")
    elif self.path == "/files/evil-invoice.pdf":
        self._bytes(MZ, "application/pdf")
    elif self.path == "/files/plain.txt":
        self._bytes(b"an honest little text file\n", "text/plain")
    else:
        super().do_GET()

def do_POST(self):
    if self.path == "/sink":
        length = int(self.headers.get("Content-Length") or 0)
        data = self.rfile.read(length) if length else b""
        body = b"credentials received: " + data
        self.send_response(200)
        self.send_header("Content-Type", "text/plain")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    else:
        self.send_response(404)
        self.end_headers()

def _bytes(self, payload, ctype):
    self.send_response(200)
    self.send_header("Content-Type", ctype)
    self.send_header("Content-Disposition", 'attachment')
    self.send_header("Content-Length", str(len(payload)))
    self.end_headers()
    self.wfile.write(payload)

def log_message(self, fmt, *args):
    print("[serve-tests]", fmt % args)

class Server(socketserver.ThreadingTCPServer):
    allow_reuse_address = True

if __name__ == "__main__":
    addr = ("0.0.0.0", 8000)
    print("SecondLook test bench on http://localhost:8000/ " +
          "(also http://127.0.0.1:8000/ - a DIFFERENT origin)")
    with Server(addr, Handler) as httpd:
        httpd.serve_forever()

```

12.3 The index and link farm

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>SecondLook Test Bench</title>
  <style>
    body { font: 15px/1.6 system-ui; margin: 40px auto; max-width: 560px; }
    a { color: #1a4a7a; }
    li { margin: 8px 0; }
    .tag { color: #888; font-size: 12px; }
  </style>
</head>
<body>
  <h1>SecondLook test bench</h1>
  <p>Start the server first: <code>python3 test-pages/serve-tests.py</code></p>
  <ul>
    <li><a href="/link-farm.html">link-farm.html</a>
      <span class="tag">Link Sniper + Redirect Detective + Trust Badge</span></li>
    <li><a href="/fake-login.html">fake-login.html</a>
      <span class="tag">Form Guardian (cross-origins password)</span></li>
    <li><a href="/ghost-click.html">ghost-click.html</a>
      <span class="tag">Ghost Click Blocker</span></li>
    <li><a href="/fake-download.html">fake-download.html</a>
      <span class="tag">Fake Download Button + Download Guard</span></li>
    <li><a href="/consent-banner.html">consent-banner.html</a>
      <span class="tag">Cookie Tamer</span></li>
    <li><a href="/popup-storm.html">popup-storm.html</a>
      <span class="tag">Tab Guard</span></li>
    <li><a href="/toggle-test.html">toggle-test.html</a>
      <span class="tag">the one-click cleanup demo</span></li>
  </ul>
</body>
</html>

```

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Link farm - every signal at once</title>
  <style>
    body { font: 15px/1.7 system-ui; margin: 40px auto; max-width: 620px; }
    li { margin: 10px 0; }
    .note { color: #777; font-size: 12px; }
  </style>
</head>
<body>
  <h1>Link farm</h1>
  <p>Hover each link and read the pill. The last two need the module's context or the redirect chain - watch the second-pass update.</p>
  <ul>
    <li><a href="https://www.wikipedia.org/">A boring, honest link</a>
      <div class="note">expect: green Nothing unusual</div></li>
    <li><a href="http://paypal-secure-login.com/verify">PayPal - verify your account</a>
      <div class="note">expect: amber, typo-squat + keyword + http</div></li>
    <li><a href="http://192.168.1.7:8081/login">Router panel (old firmware)</a>
      <div class="note">expect: amber, raw IP + unusual port</div></li>
    <li><a href="https://paypal.com.security-check.example.ru/signin">Continue to PayPal</a>
      <div class="note">expect: amber, brand in subdomain</div></li>
    <li><a href="https://xn--80ak6aa92e.com/">m\u00f6rning coffee</a>
      <div class="note">expect: amber, punycode/IDN host</div></li>
    <li><a href="http://micr0s0ft-support.top/download">Microsoft support tool</a>
      <div class="note">expect: amber, typo + suspicious TLD</div></li>
    <li><a href="https://user@evil.example/only-the-part-after-at">https://paypal.com/...</a>
      <div class="note">expect: amber, userinfo disguise</div></li>
    <li><a href="/hop/1">A shortlink-style chain (local)</a>
      <div class="note">trace with the right-click entry; the chain is

```

```

        real 302s</div></li>
<li><a href="#" id="zw">Sa<span>&#8203;</span>fe
    link</a>
    <div class="note">invisible zero-width char in the anchor text -
        hover after running the snippet below</div></li>
</ul>
<p></p>
<div class="note">A seal-shaped image with no link: Trust Badge flags
    it after ~800 ms. Click it for the explanation card.</div>
<script>
    // give the zero-width link a real destination
    document.getElementById('zw').href = 'https://example.com/';
</script>
</body>
</html>

```

12.4 Fake login, ghost click, fake download

test-pages/fake-login.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Fake login - Form Guardian bench</title>
    <style>
        body { font: 15px/1.6 system-ui; margin: 60px auto; max-width: 420px; }
        input, button { font: inherit; display: block; width: 100%;
            margin: 8px 0; padding: 8px; }
    </style>
</head>
<body>
    <h1>Sign in to localhost</h1>
    <p>This page is served from <b>localhost:8000</b> but posts your
        password to <b>127.0.0.1:8000</b> - a different origin. That is
        the phishing mechanic, on your machine, with zero risk.</p>
    <form method="POST" action="http://127.0.0.1:8000/sink">
        <input name="user" placeholder="user (anything)">
        <input name="pass" type="password" placeholder="password (anything)">
        <button type="submit">Sign in</button>
    </form>
    <p>Expect: the red full-page card names 127.0.0.1 as the
        destination. Continue lets it through once and remembers;
        Stop leaves the form untouched.</p>
</body>
</html>

```

test-pages/ghost-click.html

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <title>Ghost click - invisible overlay bench</title>
    <style>
        body { font: 15px/1.6 system-ui; margin: 40px auto; max-width: 520px; }
        #ghost { position: fixed; inset: 0; background: #000; opacity: 0.01;
            z-index: 2147483647; }
        #counter { font-weight: 700; color: #b91c1c; }
    </style>
</head>
<body>
    <h1>Ghost click bench</h1>
    <p>Click the big button. Before the module acts, the counter
        increments (the ghost ate your click). After ~1 second the ghost
        is neutralized and the button works.</p>
    <p>Ghost captures: <span id="counter">0</span></p>
    <button id="real" style="padding:16px 28px;font-size:18px">
        THE REAL BUTTON</button>
    <div id="ghost"></div>
    <script>
        let n = 0;

```

```

    const counter = document.getElementById('counter');
    document.getElementById('ghost').addEventListener('click', () => {
      counter.textContent = ++n;
    });
    document.getElementById('real').addEventListener('click', () => {
      counter.textContent = 'REAL CLICK LANDED';
    });
  </script>
</body>
</html>

```

test-pages/fake-download.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Download decoys - Fake Button + Download Guard bench</title>
  <style>
    body { font: 15px/1.6 system-ui; margin: 40px auto; max-width: 620px; }
    .advertisement { background: #faf3d9; padding: 10px;
      border: 1px dashed #c9b458; margin: 10px 0; }
    .corner { position: fixed; right: 12px; bottom: 12px;
      background: #ffd; padding: 10px; border: 1px solid #ccc; }
    a, button { font: inherit; padding: 10px 16px; display: inline-block;
      margin: 6px 4px; }
    main { border: 2px solid #4a9; padding: 16px; margin: 16px 0; }
  </style>
</head>
<body>
  <h1>Get the converter</h1>
  <div class="advertisement">
    <a href="https://elsewhere.example/x">Free Download</a>
    <a href="javascript:void(0)">Start Download Now</a>
    <span style="color:#888;font-size:11px">advertisement</span>
  </div>
  <div class="corner"><button>Download Now</button></div>
  <main>
    <a href="/files/dummy.pdf" download>Download dummy.pdf</a>
    <p>The honest one: real PDF bytes, direct link, main content.</p>
    <a href="/files/evil-invoice.pdf" download>invoice.pdf</a>
    <p>The lying one: MZ bytes wearing a .pdf name - Download Guard
      stops it and offers Undo.</p>
  </main>
</body>
</html>

```

12.5 Consent banner, popup storm, toggle test

test-pages/consent-banner.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Consent banner - Cookie Tamer bench</title>
  <style>
    body { font: 15px/1.6 system-ui; margin: 40px auto; max-width: 560px; }
    #banner { position: fixed; inset: auto 0 0 0; background: #111;
      color: #fff; padding: 18px 24px; display: flex; gap: 12px;
      align-items: center; z-index: 5000; }
    #banner button { font: inherit; padding: 8px 14px; }
    #banner .big { background: #2563eb; color: #fff; border: 0;
      font-weight: 700; }
    #banner .ghosty { background: none; color: #9bb; }
  </style>
</head>
<body>
  <h1>Cookie Tamer bench</h1>
  <p>This banner has an honest reject control. Cookie Tamer clicks it
    once and it disappears. Edit the HTML to remove the reject button
    and reload to see the parking behavior instead.</p>
  <p id="gone" style="color:#15803d;font-weight:700"></p>

```

```

<div id="banner">
  <span>We use cookies. You have no meaningful choice, but we must
    pretend you do.</span>
  <button class="big" onclick="console.log('ACCEPTED ALL')">Accept
    All</button>
  <button class="ghosty" id="reject"
    onclick="document.getElementById('banner').remove();
      document.getElementById('gone').textContent =
        'banner dismissed by an honest click';">Reject</button>
</div>
</body>
</html>

```

test-pages/popup-storm.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Popup storm - Tab Guard bench</title>
</head>
<body>
  <h1>Popup storm bench</h1>
  <button onclick="storm()" style="padding:14px 22px;font-size:16px">
    Open 6 tabs (storm)</button>
  <p>The first two land normally; the rest are held in the collapsed
    "Held for review" group.</p>
  <script>
    function storm() {
      for (let i = 0; i < 6; i++) {
        window.open('/popup-storm-target.html?n=' + i, '_blank');
      }
    }
  </script>
</body>
</html>

```

test-pages/popup-storm-target.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Storm tab</title>
</head>
<body>
  <h1>I am a storm tab</h1>
  <p>autonomous, loud, and now safely quarantined</p>
</body>
</html>

```

test-pages/toggle-test.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Toggle test - the one-click cleanup demo</title>
  <style>
    body { font: 15px/1.6 system-ui; margin: 40px auto; max-width: 600px; }
    table { border-collapse: collapse; width: 100%; }
    td, th { border: 1px solid #ccc; padding: 8px 10px;
      text-align: left; font-size: 13px; }
    .ok { color: #15803d; font-weight: 700; }
  </style>
</head>
<body>
  <h1>Toggle test - OFF must be real OFF</h1>
  <p>Open the SecondLook popup and flip modules while watching this
    page. Every row should pass. This is also the best single demo
    of the product: one click, every effect vanishes.</p>
  <table>
    <tr><th>Flip off</th><th>Should vanish instantly on this page</th></tr>
    <tr><td>Demo Greeting</td><td class="ok">the bottom-left chip</td></tr>
    <tr><td>Link Sniper</td><td class="ok">any verdict pill + hover

```

```

    reactivity</td></tr>
<tr><td>Ghost Click Blocker</td><td class="ok">the neutralized
  overlay comes back to life (click the real button on
  ghost-click.html in a second tab)</td></tr>
<tr><td>Cookie Tamer</td><td class="ok">a parked consent banner
  reappears (see consent-banner.html)</td></tr>
<tr><td>Webcam/Mic Indicator</td><td class="ok">the camera chip and
  REC badge (reload needed only to REMOVE the hook itself)</td></tr>
<tr><td>Safe Mode (site card)</td><td class="ok">network unfreezes
  immediately</td></tr>
</table>
<p>Then flip them back on: every effect returns without a reload.
  Record this as a screen capture - it is the store listing's best
  asset (Part 13).</p>
</body>
</html>

```

12.6 Load & test

- Start the bench: `python3 test-pages/serve-tests.py` and open `http://localhost:8000/`.
- Walk the index in order. On link-farm, hover every link and compare pill vs expectation note; right-click the /hop/1 link and trace the two real 302 hops; confirm the seal-shaped image gets flagged by Trust Badge.
- fake-login: submit the form, read the red card, Continue, submit again (silent - remembered), then Forget approvals in Options and confirm the card returns.
- ghost-click: watch the capture counter stop incrementing after ~1 s, and the real button's "REAL CLICK LANDED".
- fake-download: the three decoys get amber outlines; dummy.pdf downloads untouched; invoice.pdf is stopped with the undoable toast - click Undo and it arrives.
- consent-banner: reject clicked for you once; edit out the reject button and reload to see parking plus the Decide now restore.
- popup-storm: six tabs become two plus a collapsed grey group.
- toggle-test: run the full table, then record the screen capture for Part 13.

Page	Modules exercised	Key assertion
link-farm.html	P1, P2, P3	every pill matches its expectation note
fake-login.html	P4	red card names 127.0.0.1; remember + forget
ghost-click.html	P5	captures stop after persistence window
fake-download.html	P6, P7	decoys marked; MZ-as-PDF stopped, undoable
consent-banner.html	P9	honest reject clicked once; no-reject parks
popup-storm.html	P8	2 free, 4+ grouped
toggle-test.html	all	one-click cleanup, reversible

Part 13 - Ship It: README & Store Listing

13.1 Design analysis

The last mile is where security extensions die: a reviewer reads the permission list with no context, a user reads the store listing with no patience, and both decide in under a minute. This part is the complete text for both audiences, plus the final housekeeping: removing the dev-only demo module, packaging the zip, and the screenshot checklist. Nothing here is code you paste into the extension runtime except the README itself - everything else is paste-ready copy for the Web Store forms.

13.2 README.md

```
README.md
# SecondLook

Quality-of-life tools that quietly watch your back.

We don't tell you that the internet is dangerous. We tell you when
something deserves a second look.

## What it does

SecondLook is ten small tools in one extension:

- Link & Site Intel - Link Sniper (hover verdicts on links),
  Redirect Detective (traces redirect chains hop by hop), Trust Badge
  (flags trust seals that are just images).
- Form & Click Safety - Form Guardian (stops passwords being sent
  to a different domain than the page you are on), Ghost Click
  Blocker (neutralizes invisible click-stealing overlays), Fake
  Download Button (marks competing download decoys).
- Files & Downloads - Download Guard (checks the first bytes of
  downloaded files against their names).
- Annoyance & Privacy - Tab Guard (contains popup storms in a
  collapsible group), Cookie Tamer (parks consent banners - it never
  clicks Accept), Webcam/Mic Indicator (a chip whenever camera,
  microphone or screen sharing is in use).

## Privacy - the short version

- All analysis happens on this device. Nothing is uploaded, ever.
- Zero telemetry, zero analytics, zero remote code. No eval, no
  bundled scripts loaded from the network.
- Local storage holds: your settings, a verdict counter, a capped
  event log of hostnames only (never full URLs), and salted
  hashes of form destinations you chose to trust. Clear it any time
  from Settings -> Your data.
- Turning a module off is real: the extension unregisters that
  module's code from every page, not merely hides its warnings.
  Per-site exceptions leave the module dormant (no observers, no
  DOM footprint) on that site.
- Blocking is used in exactly one place: Safe Mode, a per-tab
  network freeze you trigger yourself, removed with one click.

## Honest limitations

- Form Guardian inspects native form submissions; JavaScript
  exfiltration via fetch/XHR is out of scope.
- Download Guard does not look inside archives.
- The camera/mic indicator detects use that starts after a page
  loads with the module enabled; it cannot see streams that began
  before, and unhooking already-loaded pages requires a reload.
- Cookie Tamer clicks only buttons that explicitly reject; when none
  exists it parks the banner and lets you decide.

## Install (development)
1. Chrome 116 or newer. `chrome://extensions` -> Developer mode.
```

2. Generate icons: ``python3 tools/make-icons.py`` (needs Pillow).
3. Load unpacked -> select this folder.
4. Optional: ``python3 test-pages/serve-tests.py`` and open `http://localhost:8000` for the full test bench.

Layout

manifest.json	MV3 manifest; content_scripts is [] by design
background/	service worker + router + bootstrap + guards
shared/	storage, settings, messaging, registry, bridge
engine/	verdict engine (the only URL scorer)
ui/	design tokens, components, popup, options, dashboard
modules/	one folder per module
test-pages/	local test bench (not shipped)

Troubleshooting

- A module does nothing on a site: check the popup row - it may be off globally or dormant for that site.
- Everything silent everywhere: the master switch in the popup.
- Weird behavior after editing code: reload the extension; the service worker console (`chrome://extensions -> SecondLook -> service worker`) tells the truth.

13.3 Remove the demo module

The demo chip is a development proof, not a product feature. Removal is three small edits and one deletion - and after each, reload the extension and confirm the popup still renders ten rows plus no DEV tag.

- Delete the folder `modules/demo-greeting/`.
- In `shared/module-registry.js`, delete the entire `demoGreeting` entry from `MODULES` (the first object in the array, marked with `devOnly: true`).
- In `ui/popup.js`, delete the two demo references: the `demoGreeting: eyeIcon()`, line inside `ICONS` and, if you copied the Part 11 version, nothing else - `iconFor` falls back safely.
- In `shared/settings.js`, delete the `demoGreeting: { enabled: true }`, line from `DEFAULTS`. Existing installs keep a stale key in storage harmlessly; the registry no longer iterates it.

13.4 Web Store listing copy

(store listing - paste into the developer dashboard forms)

Name:

SecondLook - a second look before you click

Short summary (132 chars max):

Hover verdicts on links, storm-proof tabs, honest cookie banners - quality-of-life tools that quietly watch your back. Local-only.

Category: Productivity

Detailed description:

SecondLook is a suite of ten quiet tools that watch the web pages you already use, and speak up only when something deserves a second look.

LINKS AND SITES

- Hover any link for a plain-language verdict: "Nothing unusual", or one sentence on what looks off (lookalike domains, disguised text, raw IP destinations).
- Right-click any link to trace its full redirect chain, hop by hop.
- Trust seals that are just images get marked - because anyone can paste a picture of a padlock.

FORMS AND CLICKS

- If a login form would send your password to a different domain than the page you are looking at, a full-page card stops it: Continue or Stop. Always reversible. Always your call.
- Invisible overlays that steal your clicks are quietly defused.
- Competing "Download" decoy buttons are marked so the real one stands out.

FILES

- A file named invoice.pdf whose bytes are actually a program never reaches your disk. Every stop has an Undo.

ANNOYANCE AND PRIVACY

- Popup storms get contained in a collapsed "Held for review" tab group instead of your face.
- Consent banners: we click only buttons that explicitly reject. If there is no honest reject button, the banner is parked - never accepted on your behalf.
- A clear indicator whenever a page uses your camera, microphone or screen sharing.

PRIVACY

- Everything runs on your device. No telemetry, no analytics, no remote code, no account.
- Every module can be turned off - and off means the code is really removed from pages, not just silenced.
- One blocking feature exists at all: Safe Mode, a per-tab network freeze you trigger yourself.

We don't tell you that the internet is dangerous. We tell you when something deserves a second look.

13.5 Permission justification (paste into the review form)

(review form - permission justifications)

storage: settings, local counters, and a capped hostname-only event log on the user's device. Nothing syncs, nothing uploads.

tabs: show the current site's state in our popup and contain popup-storm tab spam in a review group.

scripting: register/unregister each module's code on pages; this is how turning a module off truly removes it. No remote code - only files bundled in this package.

downloads: verify the first 16 bytes of a download against its file name, and stop (reversibly) documents whose bytes are programs.

cookies: the user-invoked "sweep this site's cookies" action, which preserves login and session cookies.

alarms: prune the local event log periodically. No network use.

contextMenus: "Where does this link really go?" and the per-tab Safe Mode toggle.

tabGroups: the collapsed "Held for review" group for popup storms.

declarativeNetRequest: Safe Mode only - a session rule scoped to one tab that the user enables and removes with a click. Our only blocking API, never used automatically.

webRequest: observe redirect hops to draw the chain a link really takes. Observer only; no blocking, no header modification.

http/https on all sites: the tool is site-agnostic by design - link and form checks must work everywhere the user browses. All analysis is local; no data leaves the browser.

13.6 Screenshot checklist

Store listings convert on their screenshots, and you already have every scene built. Capture at 1280x800, light mode, in this order: (1) the popup open on link-farm.html with an amber pill and a green pill visible; (2) the redirect trace card mid-hop-list; (3) the Form Guardian red card on fake-login.html; (4) fake-download.html with decoys outlined; (5) the toggle-test page mid-cleanup - modules flipping off with effects vanishing; (6) the dashboard with populated stats. The fifth one is the money shot: record it as a screen capture if the dashboard supports video listings, because one-click cleanup is the single most persuasive thing this extension does.

13.7 Packaging

The shipped zip contains the runtime tree and nothing else. Run from the parent folder of `secondlook/`:

```
(terminal - package for the store)
cd secondlook
python3 tools/make-icons.py          # ensure icons exist and are fresh
cd ..
zip -r secondlook-1.0.0.zip secondlook \
  -x "secondlook/test-pages/*" \
  -x "secondlook/tools/*" \
  -x "secondlook/README.md"          # store listing carries this now
```

Before uploading, walk one final pass with `test-pages/serve-tests.py` against the packaged build loaded from a fresh unpacked copy of the zip's contents, then submit. Review turnaround for permission-rich extensions is typically a few days; the justification block above answers every question they will ask, because it is the same honesty the code was built with.